

py4web Documentation

?

Massimo Di Pierro

Table of Contents

1 py4web ? 1
- 1.1 ? . . . 2

2 ? 3
- 2.1 ? . . . 3
- 2.2 ? . . . 3
- 2.3 ? . . . 4

3 ? 5
- 3.1 ? . . . 5
- 3.2 ? . . . 5
- 3.3 ? . . . 5
- 3.4 ? . . . 7
- 3.5 ? . . . 7
- 3.6 ? . . . 8
- 3.7 ? . . . 12

4 Dashboard 15
- 4.1 web ? . . . 15
- 4.2 ? . . . 16

5 ? 21
- 5.1 ? . . . 21
- 5.2 ? . . . 21
- 5.3 ? . . . 21
- 5.4 _scaffold ? . . . 24
- 5.5 ? . . . 26
- 5.6 ? . . . 26
- 5.7 ? . . . 27
- 5.8 ? . . . 27

6 ? 29
- 6.1 ? . . . 29
- 6.2 ? . . . 29
- 6.3 ? . . . 30
- 6.4 ? . . . 30
- 6.5 ? . . . 33
- 6.6 Session ? . . . 34
- 6.7 Condition ? . . . 36
- 6.8 URLsigner ? . . . 37
- 6.9 DAL ? . . . 38

6.10 Auth ? . . . 38
6.11 ? . . . 39
6.12 ? . . . 39
6.13 ? . . . 41
6.14 ? . . . 42
6.15 ? . . . 42

7 ? 45
7.1 DAL ? . . . 45
7.2 ? . . . 47
7.3 DAL ? . . . 48
7.4 “Table” ? . . . 52
7.5 “Field” ? . . . 55
7.6 ? . . . 59
7.7 “Table” ? . . . 60
7.8 ? . . . 64
7.9 `select` ? . . . 65
7.10 ? . . . 74
7.11 ? . . . 77
7.12 ? . . . 81
7.13 ? . . . 84
7.14 ? . . . 88
7.15 ? . . . 93

8 RestAPI 99
8.1 RestAPI ? . . . 100
8.2 RestAPI GET . . . 102
8.3 RestAPI ? . . . 102
8.4 RestAPI ? . . . 115

9 YATL ? 117
9.1 ? . . . 118
9.2 ? . . . 120
9.3 ? . . . 123

10 YATL ? 125
10.1 ? . . . 125
10.2 ? . . . 126
10.3 ? . . . 131
10.4 ?*DOM* . . . 132
10.5 ?? . . . 134

11 ? 137
11.1 Pluralize . . . 137
11.2 ? . . . 138

12 ? 139
12.1 Form ? . . . 140
12.2 ? . . . 140
12.3 ? . . . 142
12.4 ? . . . 144
12.5 ? . . . 145
12.6 ? . . . 147

13 ? 161
13.1 ? . . . 161
13.2 ? . . . 168

14 ? **171**
14.1 ? . . . 171
14.2 grid ? . . . 171
14.3 Grid ? . . . 174
14.4 ? . . . 176
14.5 ? . . . 176
14.6 Grid ? . . . 177
14.7 Grid ? . . . 177
14.8 ? . . . 178
14.9 ? . . . 178
14.10 ? . . . 179
14.11 ? . . . 179
14.12 ? . . . 180

15 ? **181**
15.1 ? . . . 181

16 ? **187**
16.1 ? . . . 187
16.2 ? . . . 188
16.3 Celery . . . 189
16.4 py4web ? . . . 189
16.5 htmx . . . 189
16.6 utils.js . . . 195
16.7 ? . . . 198

Chapter 1

py4web ?

PY4WEB ?

PY4WEB ?

?

Web2py ?

- ?
- ?
- ?

PY4WEB ?

?Chapter 15 ″ ?

- ?
- PY4WEB?
- PY4WEB?
- PY4WEB?
- PY4WEB?
- PY4WEB?
- PY4WEB?
- PY4WEB?
- PY4WEB?
- PY4WEB?
- PY4WEB?
- PY4WEB ?
- PY4WEB ?
- PY4WEB ?
- PY4WEB ?
- PY4WEB ?
- PY4WEB ?

PY4WEB ?

1.1 ?

?

?



?

?

2.1 ?

2.1.1 ?

?https://py4web.com/_documentation/static/index.html ?

2.1.2 Google group?

?https://groups.google.com/g/py4web?

2.1.3 Discord server?

?Discord server dedicated to py4web ?

2.1.4 ?

?

- ?Learn py4web ?
- UC Santa Cruz ?course 2020 by Luca de Alfaro ?
- Andrew Gavgavian ?py4web blog app ?
- ?Jim Steil ?South Breeze Enterprises ??? ?
- Alan Etkin ?py4web blog ?

2.1.5 GitHub ?

?https://github.com/web2py/py4web?

2.2 ?

?

2.2.1 ?

?

2.2.2 ?

?VS Code ?PyCharm ?

?

- ?**virtualenv** ??? ?
- ?**git** ?
- ?

2.2.3 ?

?

?

```
"args": ["run", "apps"],
"program": "your_full_path_to_py4web.py",
```

?

?

```
{
  "name": "py4web apps",
  "type": "debugpy",
  "request": "launch",
  "module": "py4web",
  "args": ["run", "apps", "-D", "--watch", "lazy"]
}
```

?*args* ?`apps` ?. ?

Tip ?`launch.json` ?`"gevent": true` ?

2.2.4 ?

?

2.3 ?

?

?specific README ?

?https://github.com/web2py/py4web ?

Chapter 3

?

3.1 ?

?

- py4web ?
- ?

py4web ?

3.2 ?

py4web ?

3.3 ?

?? ?

3.3.1 ?

?**virtualenv** ?here ?

?

Linux ?

```
python3 -m venv venv
. venv/bin/activate
python -m pip install --upgrade py4web --no-cache-dir
python py4web setup apps
python py4web set_password
python py4web run apps
```

?

Windows

```
run cmd.exe
In e.g. folder c:\py4web
python3 -m venv venv
"C:\py4web\venv\Scripts\activate.bat"
python -m pip install --upgrade py4web --no-cache-dir
```

```
cd venv\scripts
py4web.exe setup apps
py4web.exe set_password
py4web.exe run apps
```

?

3.3.2 ?

pip ?

?

```
python3 -m pip install --upgrade py4web --no-cache-dir --user
```

?`python3` ?`python3.8` ?

?

```
py4web setup apps
py4web set_password
py4web run apps
```

?*pip* ?*–user* ?

```
python3 py4web.py setup apps
python3 py4web.py set_password
python3 py4web.py run apps
```

3.3.3 ?

?*make* ?

```
git clone https://github.com/web2py/py4web.git
cd py4web
make assets
make test
python -m pip install .
py4web setup apps
py4web set_password
py4web run apps
```

?`py4web/assets` ?`make assets` ?

?

3.3.4 ?

?

?

```
git clone https://github.com/web2py/py4web.git
cd py4web
python3 -m pip install --upgrade -e .
```

?

Linux ?

```
./py4web.py setup apps
./py4web.py set_password
./py4web.py run apps
```

?**./** ?

Windows

```
python3 py4web.py setup apps
python3 py4web.py set_password
python3 py4web.py run apps
```

?./ ?

3.3.5 ?

?

?? ?

```
./py4web set_password
./py4web run apps
```

?

?

3.4 ?

?

```
python3 -m pip install --upgrade py4web
```

Warning ?Dashboard ?Default ?

```
py4web setup <path to apps_folder>
```

?

?

3.4.1 ?

?https://docs.astral.sh/uv/getting-started/installation/ ?

```
uv run py4web.py run apps
```

?

3.5 ?

?



?`apps` ?`run` ?**Dashboard** (`_dashboard`) ?**Default** (`_default`) ?

Dashboard ?

Default ?

Note ?**Dashboard** ?**Default** - ?_ ?

?

```
http://localhost:8000
http://localhost:8000/_dashboard
http://localhost:8000/{yourappname}/index
```

?`Control-C` ?

Note ?**Default** ?`apps/_default` ?

?`/index` ?

> **Warning** ?`Ctrl-C` ?`Ctrl-Break` ?`Ctrl-Fn-Pause` ?

3.6 ?

py4web ?

```
# py4web
```

```
Usage: py4web.py [OPTIONS] COMMAND [ARGS]...

  PY4WEB - a web framework for rapid development of efficient database
  driven web applications

  Type "./py4web.py COMMAND -h" for available options on commands

Options:
  -help, -h, --help  Show this message and exit.

Commands:
  call          Call a function inside apps_folder
  new_app       Create a new app copying the scaffolding one
  run           Run all the applications on apps_folder
  set_password  Set administrator's password for the Dashboard
  setup         Setup new apps folder or reinstall it
  shell         Open a python shell with apps_folder added to the path
  version       Show versions and exit
```

?**–help** ?**-h** ?

3.6.1 `call` ?

```
# py4web call -h
Usage: py4web.py call [OPTIONS] APPS_FOLDER FUNC

  Call a function inside apps_folder

Options:
  -Y, --yes           No prompt, assume yes to questions  [default: False]
  --args TEXT         Arguments passed to the program/function  [default: {}]
  -help, -h, --help   Show this message and exit.
```

?

```
# py4web call apps examples.test.myfunction --args '{"x": 100}'
```

myfunction ?

3.6.2 `new_app` ?

```
# py4web new_app -h
Usage: py4web.py new_app [OPTIONS] APPS_FOLDER APP_NAME

  Create a new app copying the scaffolding one

Options:
  -Y, --yes                 No prompt, assume yes to questions  [default:
                            False]

  -s, --scaffold_zip TEXT   Path to the zip with the scaffolding app
  -help, -h, --help         Show this message and exit.
```

?

3.6.3 `run` ?

```
# py4web run -h
Usage: py4web.py run [OPTIONS] APPS_FOLDER
```

```
  Run the applications on apps_folder

Options:
   -Y, --yes                       No prompt, assume yes to questions
   -H, --host TEXT                 Host listening IP  [default: 127.0.0.1]
   -P, --port INTEGER              Port number  [default: 8000]
   -A, --app_names TEXT            List of apps to run, comma separated (all if
                                    omitted or empty)
   -p, --password_file TEXT        File for the encrypted password  [default:
                                    password.txt]
   -Q, --quiet                     Suppress server output
   -R, --routes                    Write apps routes to file
   -s, --server
[default|wsgiref|tornado|wsgiref+threaded|rocket|waitress|gunicorn|gevent|gunicorn+gevent|gevent
                                    Web server to use (unavailable: waitress,
                                    gunicorn, gevent, gunicorn+gevent,
                                    gevent+websockets)
   -w, --number_workers INTEGER    Number of workers  [default: 0]
   -d, --dashboard_mode TEXT       Dashboard mode: demo, readonly, full, none
                                    [default: full]
   --watch [off|sync|lazy]         Watch python changes and reload apps
                                    automatically, modes: off, sync, lazy
                                    [default: lazy]
   --ssl_cert PATH                 SSL certificate file for HTTPS
   --ssl_key PATH                  SSL key file for HTTPS
   --errorlog TEXT                 Where to send error logs
                                    (:stdout|:stderr|tickets_only|{filename})
                                    [default: :stderr]
   -L, --logging_level INTEGER     The log level (0 - 50) [default: 30
                                    (=WARNING)]
   -D, --debug                     Debug switch
   -U, --url_prefix TEXT           Prefix to add to all URLs in and out
   -m, --mode TEXT                 default or development  [default: default]
   -help, -h, --help               Show this message and exit.
```

app_names ?

?py4web run --host 0.0.0.0 apps ?

url_prefix ?py4web run --url_prefix=/abracadabra --port 8000 apps ?

?py4web run --watch sync ?py4web run --watch off ?

Note --watch ?/apps ?

?server ?Rocket3 ?

logging_level ?**logging** ?

debug ?

3.6.4 set_password ?

```
# py4web set_password -h
Usage: py4web.py set_password [OPTIONS]

  Set administrator's password for the Dashboard

Options:
  --password TEXT           Password value (asked if missing)
  -p, --password_file TEXT  File for the encrypted password  [default:
                            password.txt]

  -h, -help, --help         Show this message and exit.
```

?--dashboard_mode ?demo ?none ?

```
py4web set_password
```

?

```
py4web set_password my_password_file.txt
```

?

```
py4web run -p my_password_file.txt apps
```

?

```
python3 -c "from pydal.validators import CRYPT;
open('password.txt','w').write(str(CRYPT()(input('password:'))[0]))"
password: *****
```

3.6.5 setup ?

```
# py4web setup -h
Usage: py4web.py setup [OPTIONS] APPS_FOLDER

  Setup new apps folder or reinstall it

Options:
  -Y, --yes           No prompt, assume yes to questions  [default: False]
  -help, -h, --help   Show this message and exit.
```

?

3.6.6 shell ?

```
# py4web shell -h
Usage: py4web.py shell [OPTIONS] APPS_FOLDER

  Open a python shell with apps_folder's parent added to the path

Options:
  -Y, --yes           No prompt, assume yes to questions  [default: False]
  -h, -help, --help   Show this message and exit.
```

Py4web ?

?

```
from apps.myapp import db
from py4web import Session, Cache, Translator, DAL, Field
from py4web.utils.auth import Auth
```

3.6.7 version ?

```
# py4web version -h
Usage: py4web.py version [OPTIONS]

  Show versions and exit

Options:
  -a, --all           List version of all modules
  -h, -help, --help   Show this message and exit.
```

?-all ?

3.7 ?

?deployment_tools ?

3.7.1 HTTPS

?

- ?

  https://www.section.io/engineering-education/how-to-get-ssl-https-for-localhost/.

- ?

?

```
"configurations": [
        {
            "name": "py4web",
            "type": "debugpy",
            "request": "launch",
            "module": "py4web",
            // or "program": "${workspaceFolder}/py4web.py", if you didn't install
py4web as a package
            "args": [
                "run",
                "apps",
                "--ssl_cert", "/path_to/localhost.crt",
                "--ssl_key", "/path_to/localhost.key",
                "--server", "rocketServer",
            ]
        }
    ]
```

?

3.7.2 WSGI

py4web ?

```
# py4web_wsgi.py
from py4web.core import wsgi
application = wsgi(apps_folder="apps")
```

?

```
gunicorn -w 4 py4web_wsgi:application
```

wsgi ?

3.7.3 ?

?Gcloud console ?

?

```
mkdir gae
cd gae
```

?

```
cp /path/to/py4web/development_tools/gcloud/* ./
```

?apps ?__init__.py ?

```
Makefile
apps
  __init__.py
  ... your apps ...
lib
app.yaml
main.py
```

?

```
make install-gcloud-linux
make setup
gcloud config set {your email}
gcloud config set {project id}
```

?

?

```
make deploy
```

?main.py ?

3.7.4 ?

?YouTube video ?detailed tutorial ?py4web/deployment_tools/pythonanywhere.com/bottle_app.py ?

3.7.5 ?

?deployment_tools/docker ?https://github.com/macneiln/docker-py4web-scaffold> ?

?

3.7.6 ?

?deployment_tools/ubuntu ?

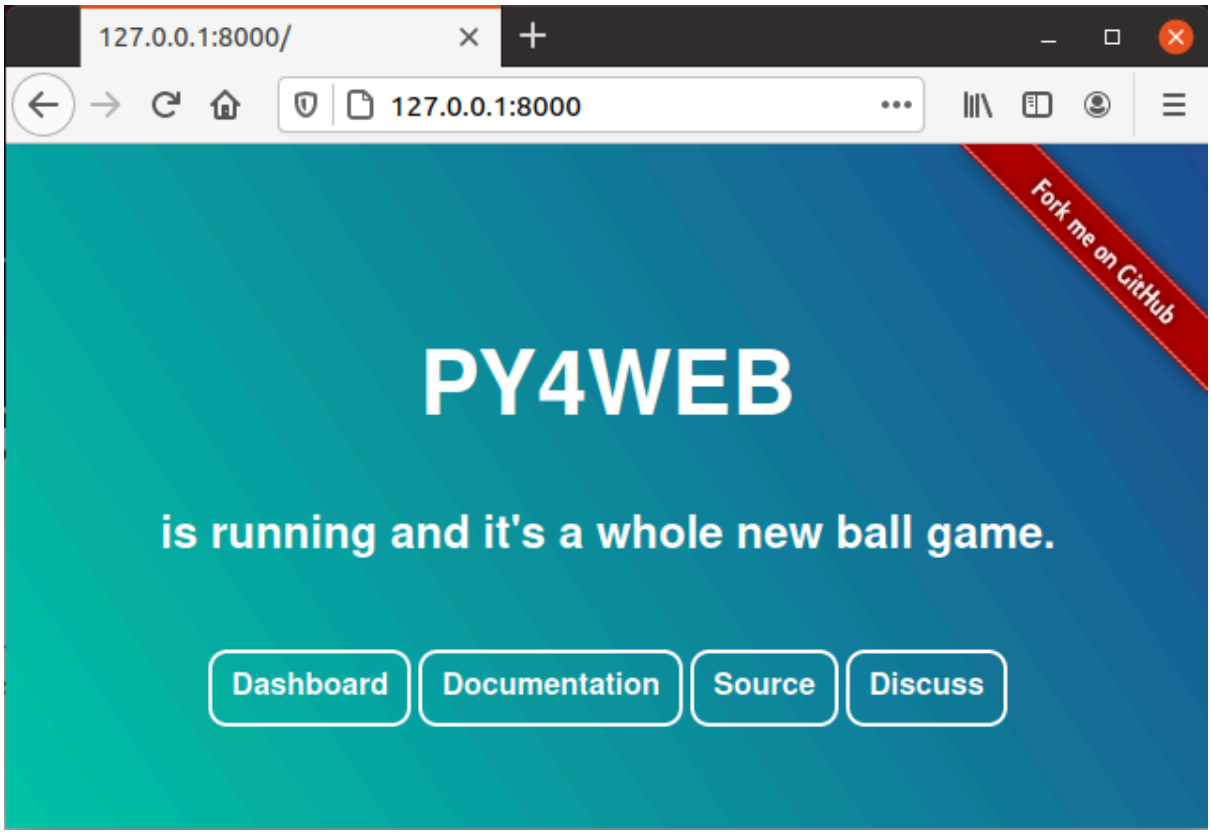
Dashboard

Dashboard ?

4.1 web ?

?http://127.0.0.1:8000 ?

?

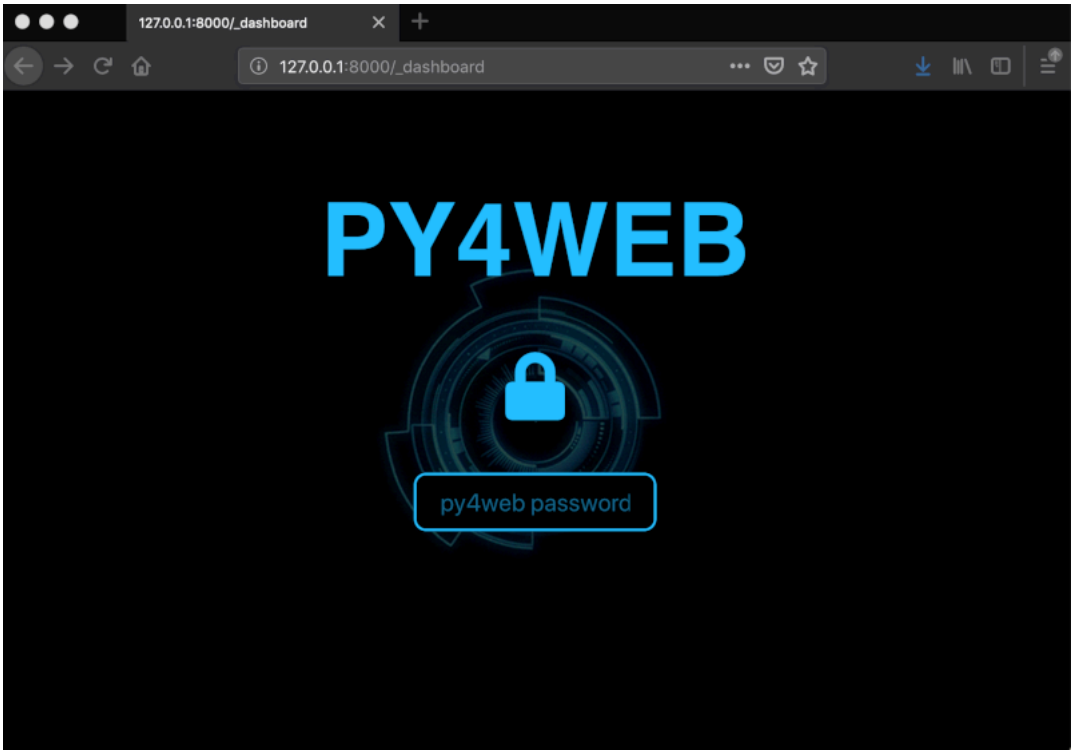


?

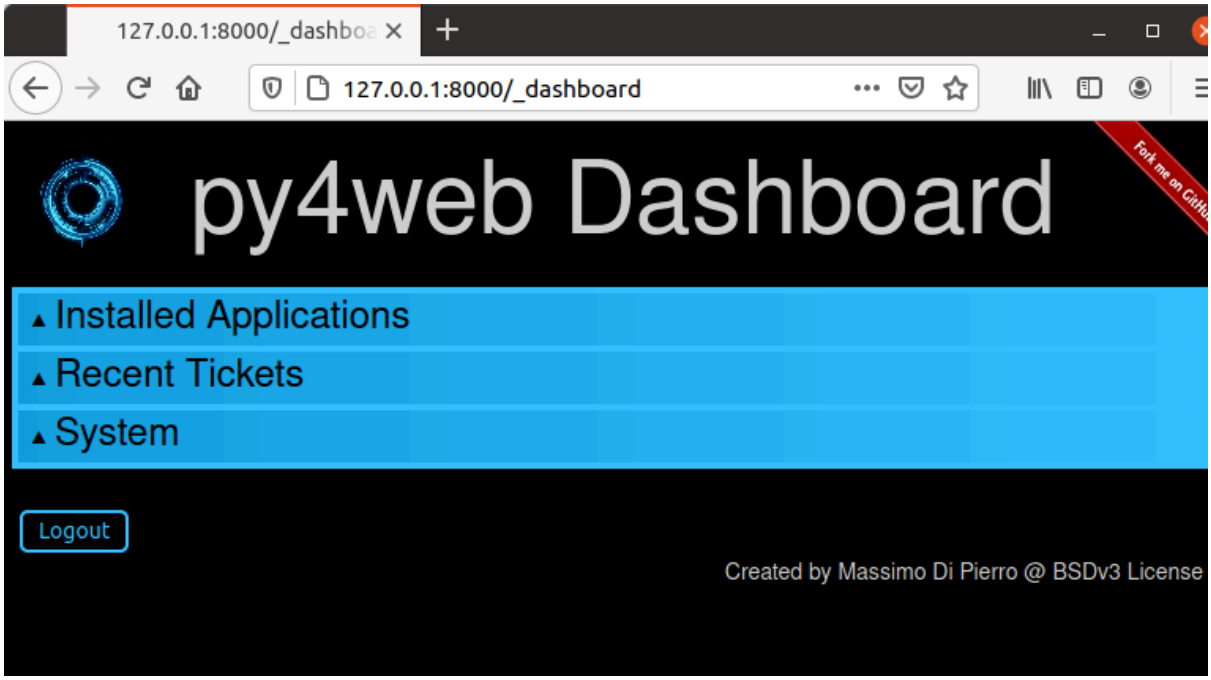
- Dashboard (http://127.0.0.1:8000/_dashboard), ?
- Documentation (http://127.0.0.1:8000/_documentation?version=1.20201112.1), ?
- Source (https://github.com/web2py/py4web), ?
- Discuss (https://groups.google.com/forum/#!forum/py4web), ?

4.2 ?

?Section 3.6.4 ?

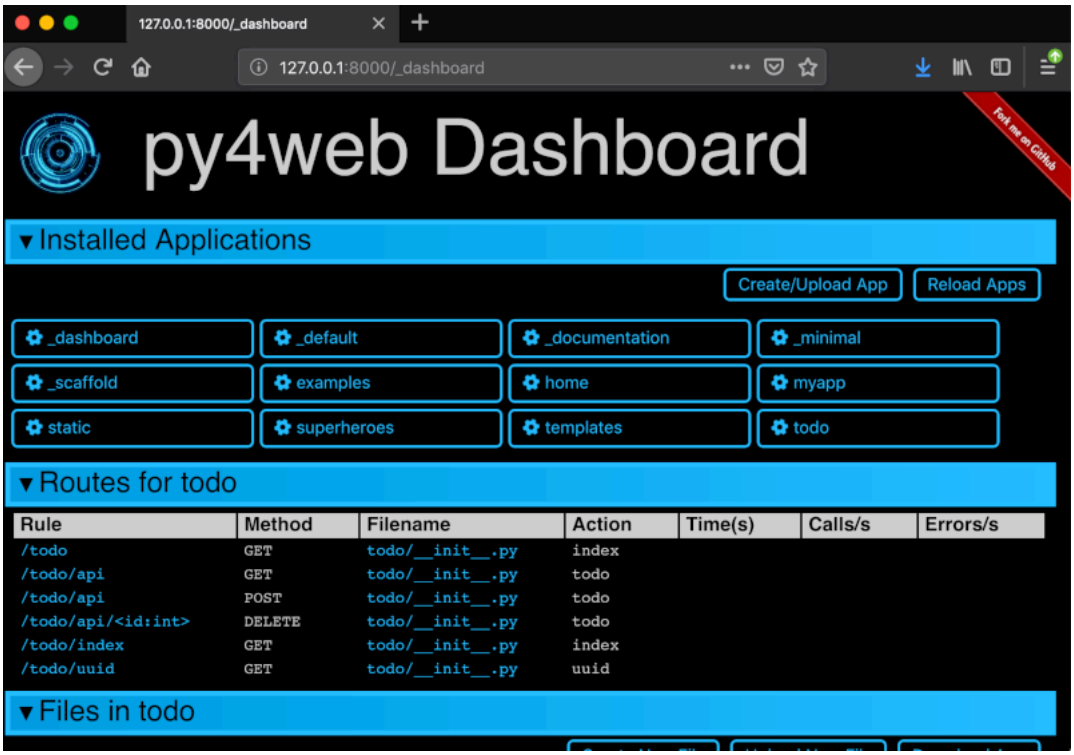


?

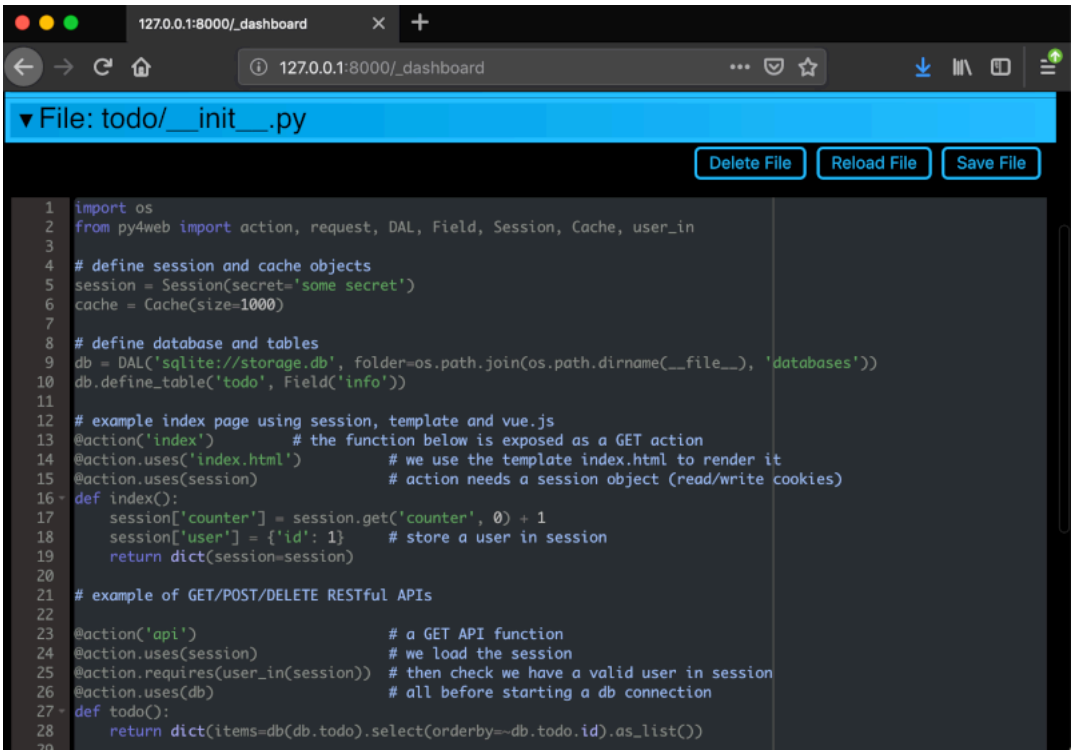


?

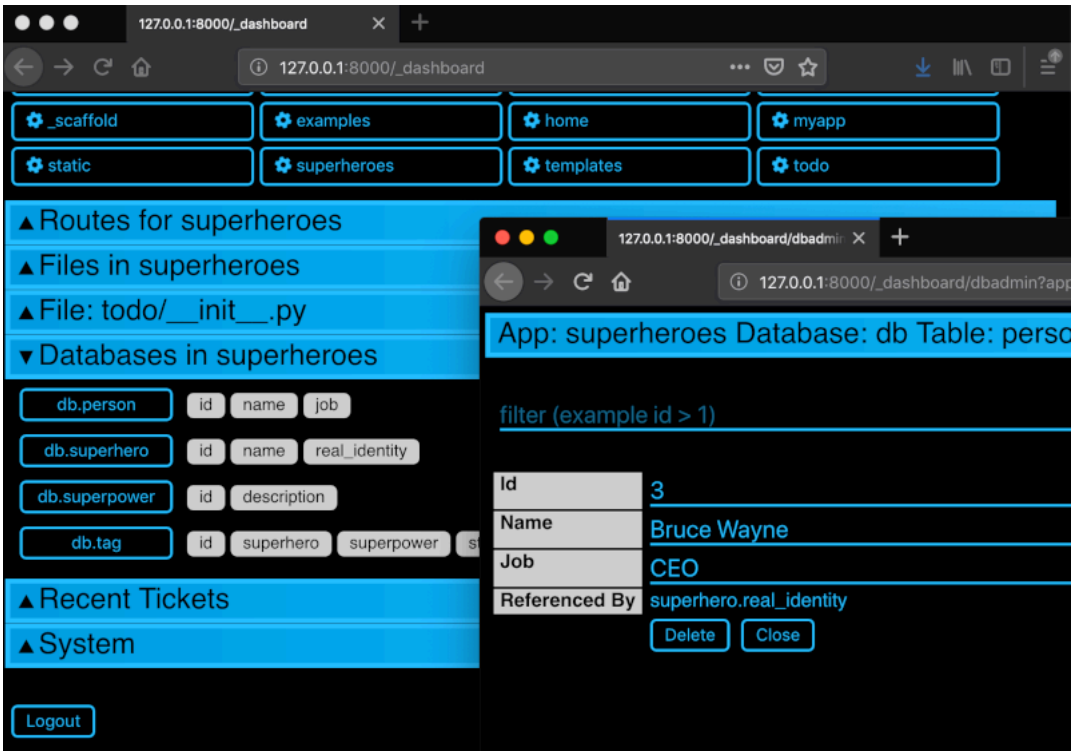
?



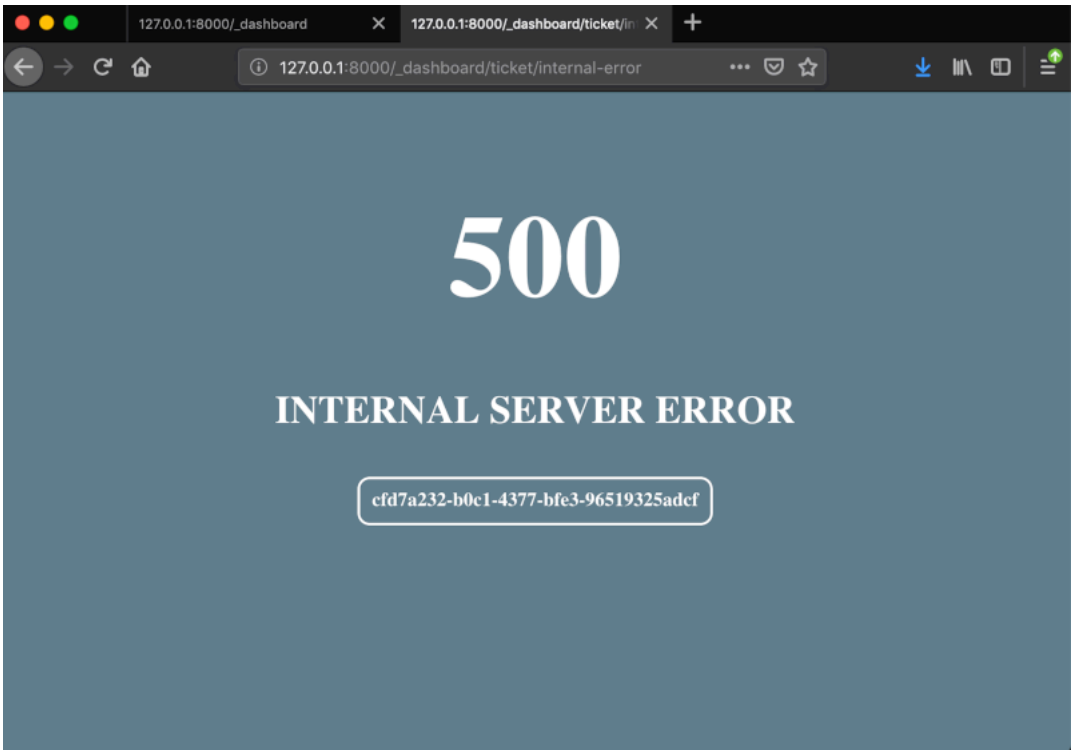
"Files" ?Section 3.6.3 ?*watch* ?



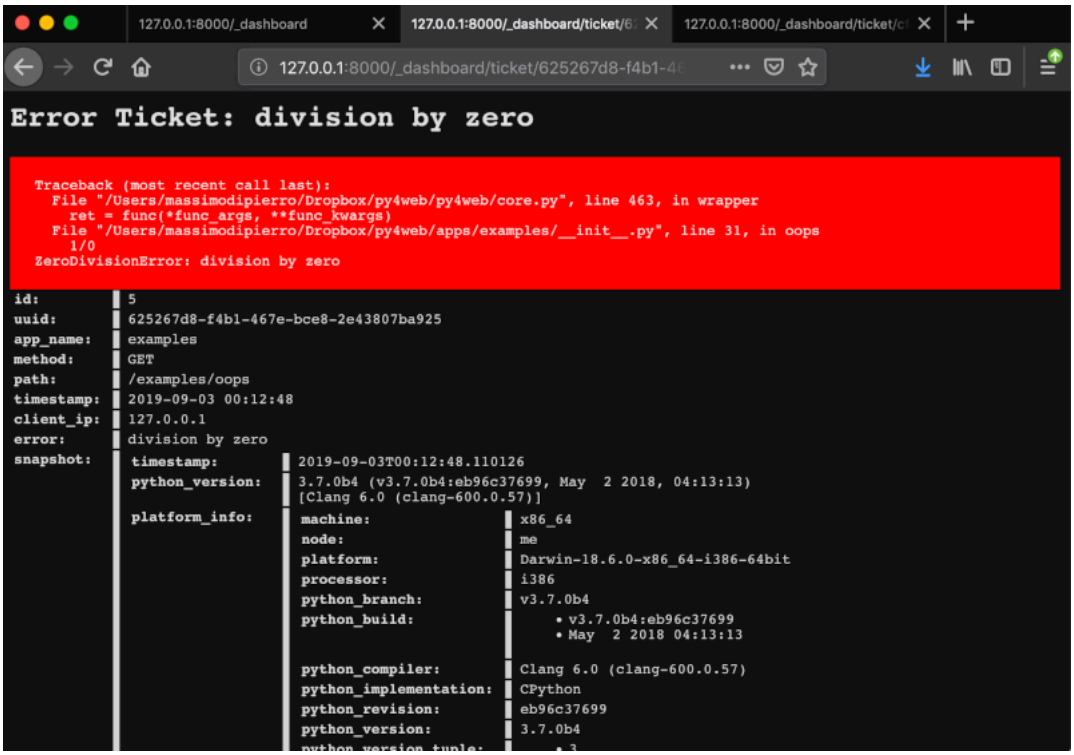
Dashboard ?


127.0.0.1:8000/_dashboard
_scaffold
examples
home
myapp
static
superheroes
templates
todo
Routes for superheroes
Files in superheroes
File: todo/__init__.py
Databases in superheroes
db.person
id
name
job
db.superhero
id
name
real_identity
db.superpower
id
description
db.tag
id
superhero
superpower
Recent Tickets
System
Logout
127.0.0.1:8000/_dashboard/dbadmin?app
App: superheroes Database: db Table: perso
filter (example id > 1)
Id
3
Name
Bruce Wayne
Job
CEO
Referenced By
superhero.real_identity
Delete
Close

?? ?


127.0.0.1:8000/_dashboard
127.0.0.1:8000/_dashboard/ticket/internal-error
500
INTERNAL SERVER ERROR
cfd7a232-b0c1-4377-bfe3-96519325adcf

? ?


127.0.0.1:8000/_dashboard
127.0.0.1:8000/_dashboard/ticket/
127.0.0.1:8000/_dashboard/ticket/625267d8-f4b1-4
Error Ticket: division by zero
Traceback (most recent call last):
File "/Users/massimodipierro/Dropbox/py4web/py4web/core.py", line 463, in wrapper
ret = func(*func_args, **func_kwargs)
File "/Users/massimodipierro/Dropbox/py4web/apps/examples/__init__.py", line 31, in oops
1/0
ZeroDivisionError: division by zero
id: 5
uuid: 625267d8-f4b1-467e-bce8-2e43807ba925
app_name: examples
method: GET
path: /examples/oops
timestamp: 2019-09-03 00:12:48
client_ip: 127.0.0.1
error: division by zero
snapshot:
timestamp: 2019-09-03T00:12:48.110126
python_version: 3.7.0b4 (v3.7.0b4:eb96c37699, May 2 2018, 04:13:13) [Clang 6.0 (clang-600.0.57)]
platform_info:
machine: x86_64
node: me
platform: Darwin-18.6.0-x86_64-i386-64bit
processor: i386
python_branch: v3.7.0b4
python_build:
• v3.7.0b4:eb96c37699
• May 2 2018 04:13:13
python_compiler: Clang 6.0 (clang-600.0.57)
python_implementation: CPython
python_revision: eb96c37699
python_version: 3.7.0b4

Chapter 5

?

5.1 ?

?

?`__init__.py` ?

Note ?

?**myapp** ?

```
mkdir apps/myapp
echo '' > apps/myapp/__init__.py
```

Tip ?\ )?

?*lazy watch* ?Section 3.6.3?

```
py4web run apps --watch off
```

py4web ?

5.2 ?

?`static` ?

```
mkdir apps/myapp/static
echo 'Hello World' > apps/myapp/static/hello.txt
```

?

```
http://localhost:8000/myapp/static/hello.txt
```

?`static` ?`static` ?

Important py4web ?ombott (One More BOTTle) web server -- ?bottlepy ?

5.3 ?

?`myapp/__init__.py` ?

```
import datetime
from py4web import action

@action('index')
def page():
    return "hello, now is %s" % datetime.datetime.now()
```

?

```
http://localhost:8000/myapp/index
```

?

```
http://localhost:8000/myapp
```

?

?myapp ?@action() ?/myapp ?_default ?_default ?myapp ?

5.3.1 ?

py4web ?__init__.py ?

```
@action('colors')
def colors():
    return {'colors': ['red', 'blue', 'green']}
```

?

```
http://localhost:8000/myapp/colors
```

?{"colors": ["red", "blue", "green"]} ?

?

5.3.2 ?

?

```
@action('color/<name>')
def color(name):
    if name in ['red', 'blue', 'green']:
        return 'You picked color %s' % name
    return 'Unknown color %s' % name
```

?

```
http://localhost:8000/myapp/color/red
```

?Bottle routes ?

- <name> ?
- <name:filter> ?
- <name:filter:config>

?:re ?

- :int ?
- :float ?
- :path ?
- :re[:exp] ?

?name ?

?

?

```
@action('color/<code:int>')
def color(code):
    return f'Color code: {code}'

@action('color/<name:path>')
def color(name):
    return f'Color name: {name}'
```

?

```
@action('color/<color_identifier:path>')
def color(color_identifier):
   try:
      msg = f'Color code: {int(color_identifier)}'
   except:
      msg = f'Color name: {color_identifier}'
   return msg
```

?method ?

```
@action('index', method=['GET','POST','DELETE'])
```

?

5.3.3 request ?

?request

```
from py4web import request

@action('paint')
def paint():
    if 'color' in request.query:
       return 'Painting in %s' % request.query.get('color')
    return 'You did not specify a color'
```

?

```
http://localhost:8000/myapp/paint?color=red
```

?Bottle request object ?

```
request.app_name
```

?

5.3.4 ?

?apps/myapp/templates/paint.html ?

```
<html>
 <head>
    <style>
      body {background:[[=color]]}
    </style>
 </head>
 <body>
    <h1>Color [[=color]]</h1>
```

```
  </body>
</html>
```

?

```
@action('paint')
@action.uses('paint.html')
def paint():
    return dict(color = request.query.get('color', 'green'))
```

?

?`@action.uses(...)` ?`action.uses` ?**fixtures** ?**fixtures** ?

?*templates* ?

?

?

- DAL ?
- Session ?
- Translator ?
- Auth ?

?

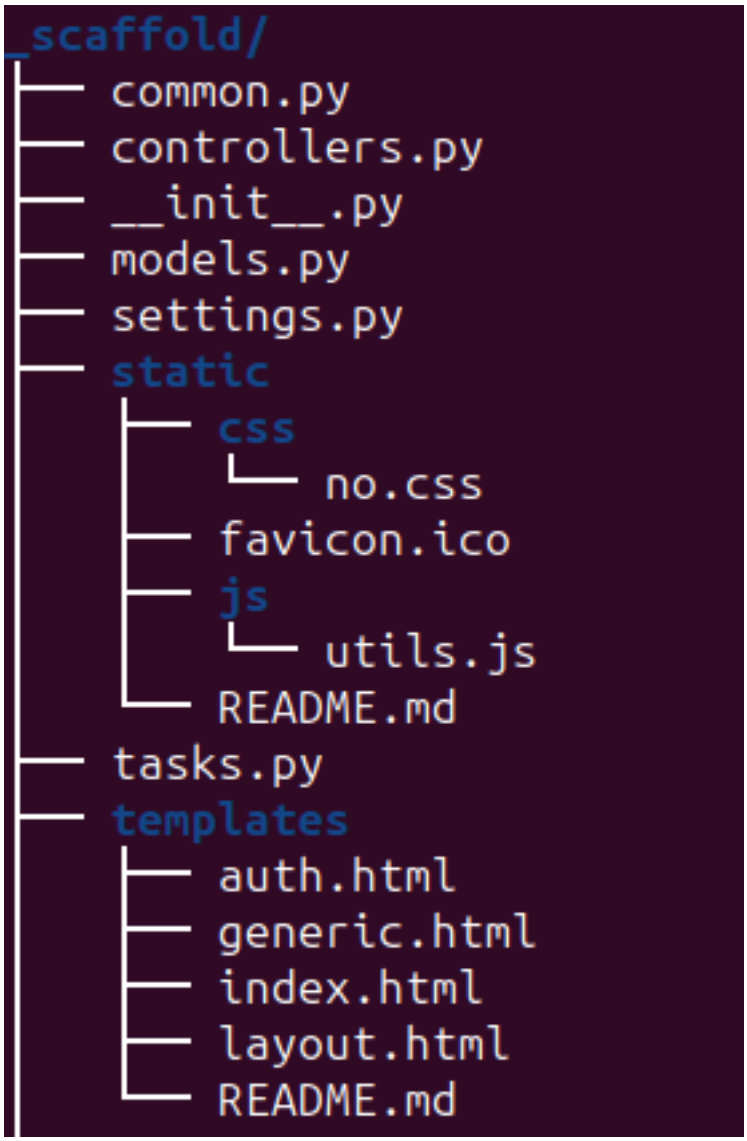
5.4 _scaffold ?

?`__init__.py` ?



?

?`_scaffold` ?



?

```
from py4web import action, request, response, abort, redirect, URL
from yatl.helpers import A
from . common import db, session, T, cache, auth


@action('welcome', method='GET')
@action.uses('generic.html', session, db, T, auth.user)
def index():
    user = auth.get_user()
    message = T('Hello {first_name}'.format(**user))
    return dict(message=message, user=user)
```

?

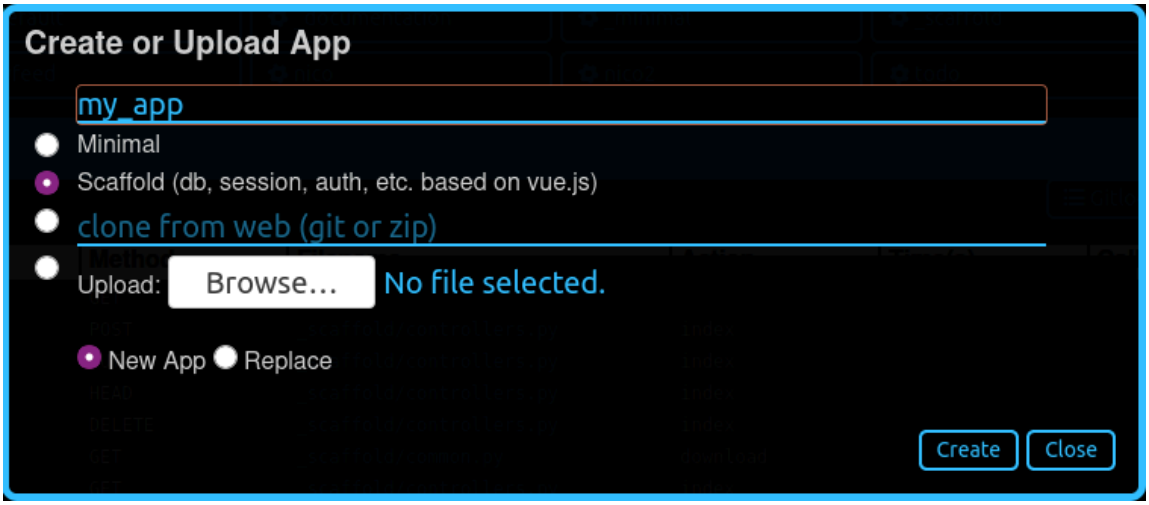
- request?response?abort ?ombott ?
- redirect ?URL ?
- ?A ?DIV ?SPAN ?IMG ?yatl.helpers ?
- db ?session ?T ?cache ?auth ?common.py ?
- @action.uses(auth.user) ?auth.get_user() ?common.py ?

?`settings.py` ?`templates` ?`models.py` ?`controllers.py` ?`common.py` ?
?`auth.html` ?

5.5 ?

?
?

- ?
- ?`Create/Upload App` ?`Create` ?



5.6 ?

?Section 3.6.3 ?`@app_watch_handler` ?
?`@app_watch_handler` ?
?

```
from py4web.core import app_watch_handler
import sass # https://github.com/sass/libsass-python

@app_watch_handler(
    ["static_dev/sass/all.sass",
     "static_dev/sass/main.sass",
     "static_dev/sass/overrides.sass"])
def sass_compile(changed_files):
    print(changed_files) # for info, files that changed, from a list of watched
files above
    ## ...
    compiled_css = sass.compile(filename=filep, include_paths=includes,
output_style="compressed")
    dest = os.path.join(app, "static/css/all.css")
    with open(dest, "w") as file:
        file.write(compiled)
```

?

```
import esprima # Python implementation of Esprima from Node.js
```

```
@app_watch_handler(
    ["static/js/index.js",
     "static/js/utils.js",
     "static/js/dbadmin.js"])
def validate_js(changed_files):
    for cf in changed_files:
        print("JS syntax validation: ", cf)
        with open(os.path.abspath(cf)) as code:
            esprima.parseModule(code.read())
```

?@app_watch_handler ?

5.7 ?

?

py4web ?

?

?myapp.example.com ?myapp ?

```
server {
   listen 80;
   server_name myapp.example.com;
   proxy_http_version 1.1;
   proxy_set_header Host $host;
   proxy_set_header X-PY4WEB-APPNAME /myapp;
   location / {
      proxy_pass http://127.0.0.1:8000/myapp$request_uri;
   }
}
```

?server ?**?** ?

- server_name ?myapp ?
- **proxy_http_version 1.1; ?**
  ?
- proxy_set_header Host $host; ?Host ?myapp.example.com
- **proxy_set_header X-PY4WEB-APPNAME /myapp; ?**
  ?myapp ?/ ?**?** ?
- **?proxy_pass http://127.0.0.1:8000/myapp$request_uri; ?$request_uri ?**
  ?127.0.0.1:8000 ?/myapp ?

?Auth ?Form ?Grid ?

5.8 ?

py4web ?

?

?

```
from py4web.core import ERROR_PAGES
```

```
ERROR_PAGES[404] = f"Page not found!"
```

?"*" ?

```
from py4web import URL
from py4web.core import ERROR_PAGES
from yatl.helpers import A

ERROR_PAGES["*"] = f"We have encountered an error! (try: {A('Main Page',
_href=URL("/",scheme=True))})"
```

?? ?? ?

Chapter 6

?

??? ??? ?

?

PY4WEB ?

PY4WEB ?

PY4WEB ?

?Section 6.12 ?

6.1 ?

?`@action.uses(...)` ?

?

```
preferred = action.uses(session, auth, T, flash)
```

?

```
@action('index')
@preferred
def index():
    return dict()
```

?

??? ?

6.2 ?

PY4WEB ?

```
from py4web import action
from py4web.core import Template

@action('index')
@action.uses(Template('index.html', delimiters='[[ ]]'))
def index():
    return dict(message="Hello world")
```

?

Template ?`index.html` ?`dict()` ?

?

```
@action.uses('index.html')
@action.uses(Template('index.html', delimiters='[[ ]]'))
```

?Section 6.14 ?

Warning ?? ?

?

```
@action.uses(session, db, 'index.html') # wrong
@action.uses('index.html', session, db) # right
```

?

?

```
def preferred(template, *optional):
    return action.uses(template, session, auth, T, flash, *optional)
```

?

```
@action('index')
@preferred('index.html')
def index():
    return dict()
```

?

6.3 ?

?

```
from py4web.utils.factories import Inject
my_var = "Example variable to be passed to a Template"

...

@action.uses('index.html', Inject(my_var=my_var))
def index():

    ...
```

?Section 10.5 ?

6.4 ?

?

```
from py4web import action, Translator
import os

T_FOLDER = os.path.join(os.path.dirname(__file__), 'translations')
T = Translator(T_FOLDER)

@action('index')
@action.uses(T)
def index(): return str(T('Hello world'))
```

?`hello world` ?`accept-language` ?

?Translator ?pluralize.Translator ?Fixture ?

?

```
from py4web import action, Session, Translator, DAL
from py4web.utils.dbstore import DBStore
import os
db = DAL('sqlite:memory')
session =  Session(storage=DBStore(db))
T_FOLDER = os.path.join(os.path.dirname(__file__), 'translations')
T = Translator(T_FOLDER)

@action('index')
@action.uses(session, T)
def index():
    counter = session.get('counter', -1)
    counter += 1
    session['counter'] = counter
    return str(T("You have been here {n} times").format(n=counter))
```

?T ?

```
@action('index')
@action.uses("index.html", session, T)
def index():
    return dict(T=T)
```

?

```
from py4web.utils.factories import Inject

@action('index')
@action.uses("index.html", session, Inject(T=T)
def index():
    return dict()
```

?translations/en.json:

```
{"You have been here {n} times":
  {
    "0": "This your first time here",
    "1": "You have been here once before",
    "2": "You have been here twice before",
    "3": "You have been here {n} times",
    "6": "You have been here more than 5 times"
  }
}
```

?

```
This your first time here
You have been here once before
You have been here twice before
You have been here 3 times
You have been here 4 times
You have been here 5 times
You have been here more than 5 times
```

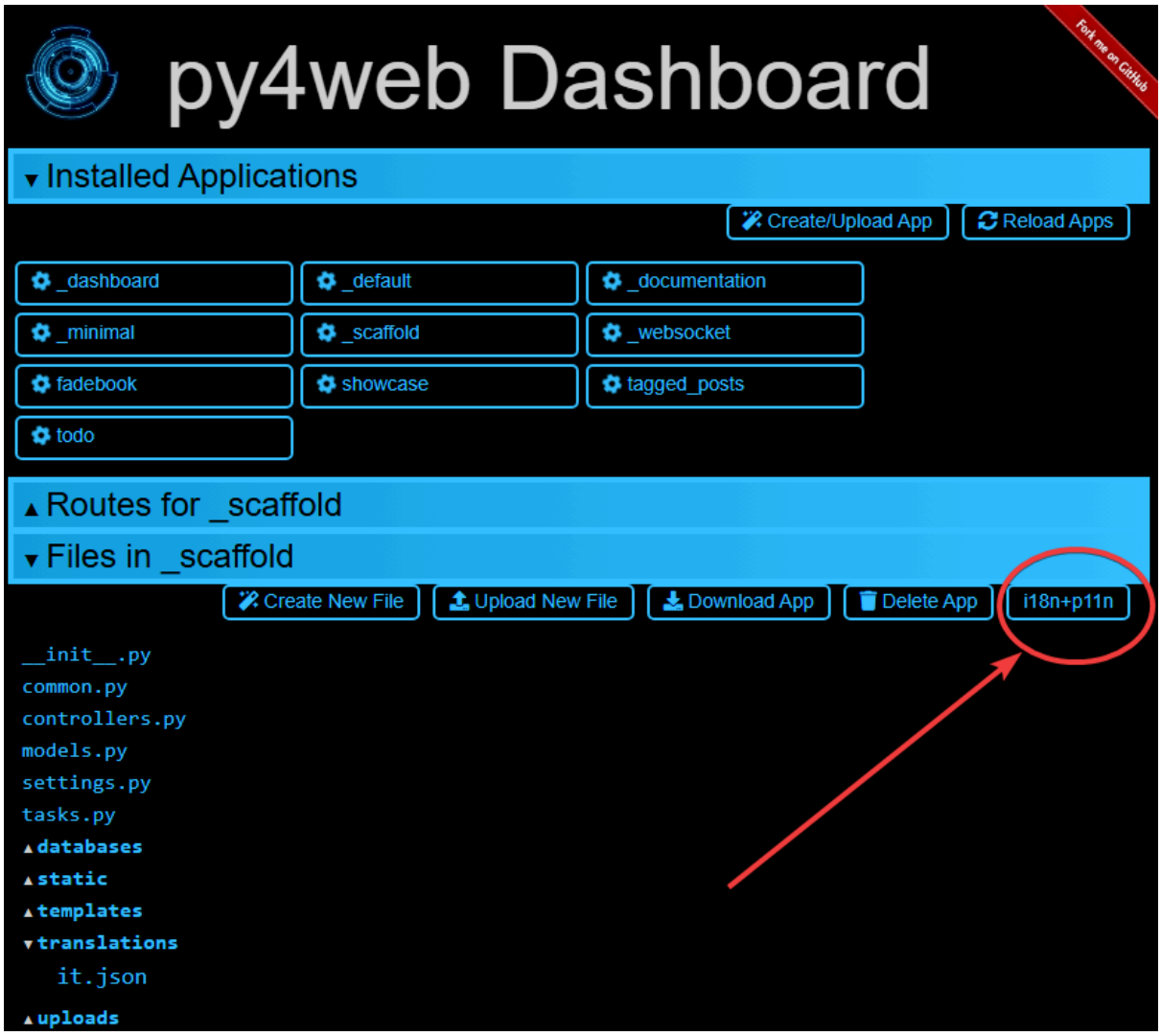
?translations/it.json ?

```
{"You have been here {n} times":
  {
    "0": "Non ti ho mai visto prima",
    "1": "Ti ho gia' visto",
```

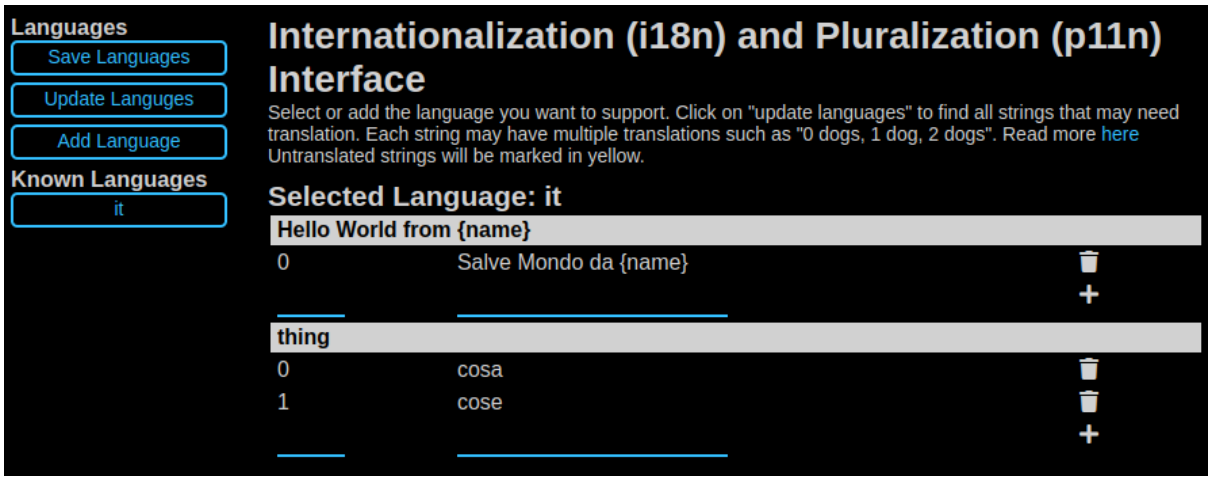
```
    "2": "Ti ho gia' visto 2 volte",
    "3": "Ti ho visto {n} volte",
    "6": "Ti ho visto piu' di 5 volte"
  }
}
```

?

?i18n+p11n ?



?



?https://github.com/web2py/pluralize ?

?

```
@action('index')
@action.uses("index.html", session, T)
def index():
    T.select(session.get("lang", "it"))
    return dict(T=T)
```

?

```
T.on_request = lambda *_: T.local.__dict__.update(tag="it",
language=T.languages["it"])
```

?`action.uses(T)` ?

6.5 ?

?flash ? ?

- ?
- ?
- ?
- ?

Flash helper ?

```
from py4web import Flash

flash = Flash()

@action('index')
@action.uses(flash)
def index():
    flash.set("Hello World", _class="info", sanitize=True)
    return dict()
```

?

```
<flash-alerts class="padded"
data-alert="[[=globals().get('flash','')]]"></flash-alerts>
```

?`py4web-flash` DIV ?

?

?`flash.set('something')` ?`flash={'message': 'hello world', 'class': 'info'}` ?

```
Q.flash({'message': 'hello world', 'class': 'info'});
```

py4web ?`info` ?`success`,`error`,`warning`,`default` ?`info` ?

?**examples** ?

6.6 Session ?

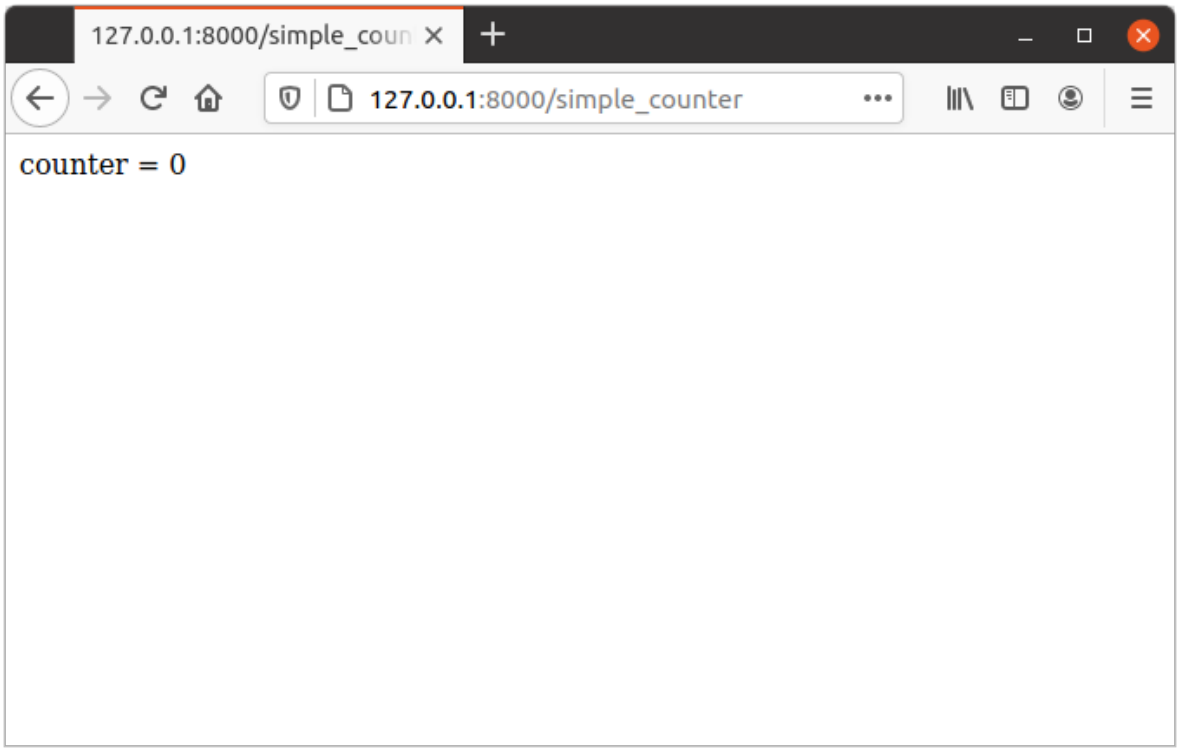
?

?

```
from py4web import Session, action
session = Session(secret='my secret key')

@action('index')
@action.uses(session)
def index():
    counter = session.get('counter', -1)
    counter += 1
    session['counter'] = counter
    return "counter = %i" % counter
```

?



?? ?

?JSON Web Token ?`__str__` ?

?

- ?
- ?

?

```
session = Session(secret='my secret key',
                  expiration=3600,
                  algorithm='HS256',
                  storage=None,
                  same_site='Lax',
                  name="{app_name}_sesson")
```

?

- secret ?
- expiration ?
- algorithm ?
- storage ?
- same_site ?? ?
- name ?

?

6.6.1 ?

?appname_session ?

Warning ?

?

6.6.2 ?

?

```
import memcache, time
conn = memcache.Client(['127.0.0.1:11211'], debug=0)
session = Session(storage=conn)
```

6.6.3 ?

?Redis ?

```
import redis
conn = redis.Redis(host='localhost', port=6379)
conn.set = lambda k, v, e, cs=conn.set, ct=conn.ttl: (cs(k, v), e and ct(e))
session = Session(storage=conn)
```

?get ?set ?set ?ttl ?set ?

6.6.4 ?

```
from py4web import Session, DAL
from py4web.utils.dbstore import DBStore
db = DAL('sqlite:memory')
session =  Session(storage=DBStore(db))
```

Warning ?`'sqlite:memory'` ?**?** ?

?

```
@action.uses(session)
@action.uses(db, session)
```

6.6.5 ?

?`Session` ?`storage` ?`get` ?`set` ?

```
import os
import json

class FSStorage:
    def __init__(self, folder):
        self.folder = folder
    def get(self, key):
        filename = os.path.join(self.folder, key)
        if os.path.exists(filename):
            with open(filename) as fp:
                return json.load(fp)
        return None
    def set(self, key, value, expiration=None):
        filename = os.path.join(self.folder, key)
        with open(filename, 'w') as fp:
            json.dump(value, fp)

session = Session(storage=FSStorage('/tmp/sessions'))
```

?

6.6.6 ?

?

```
session = Session(secret=settings.SESSION_SECRET_KEY,
                  name="app1_session")
```

"name" ?

?

```
session_app1 = Session(secret=settings.SESSION_SECRET_KEY,
                       name="app1_session")
...
@action.uses(session, session_app1)
...
```

6.7 Condition ?

?

```
@action("step1")
@action.uses(session)
def step1():
    session["step_completed"] = 1
    button = A("next", _href=URL("step2"))
    return locals()
```

```
@action("step2")
@action.uses(session, Condition(lambda: session.get("step_completed") == 1))
def step2():
    session["step_completed"] = 2
    button = A("next", _href=URL("step3"))
    return locals()

@action("step3")
@action.uses(session, Condition(lambda: session.get("step_completed") == 2))
def step3():
    session["step_completed"] = 3
    button = A("next", _href=URL("index"))
    return locals()
```

?on_request ?

?action.uses ?session ?

?

```
Condition(cond, exception=HTTP(400))
```

?

```
Condition(cond, on_false=lambda: redirect(URL('step1')))
```

?Tags ?Section 13.2 ?

```
groups = Tags(db.auth_user)

@action("payroll")
@action.uses(auth,
             Condition(lambda: 'employees' in groups.get(auth.user_id),
on_false=lambda: redirect('index')))
def payroll():
    return
```

6.8 URLsigner ?

?

```
from py4web.utils import URLSigner

# We build a URL signer.
url_signer = URLSigner(session)

@action('/somepath')
@action.uses(url_signer)
def somepath():
   # This controller signs a URL.
   return dict(signed_url = URL('/anotherpath', signer=url_signer))

@action('/anotherpath')
@action.uses(url_signer.verify())
def anotherpath():
   # The signature has been verified.
   return dict()
```

6.9 DAL ?

?DAL ?

PY4WEB ?**PyDAL** (Python Database Abstraction Layer?databases ?

```
from datetime import datetime
from py4web import action, request, DAL, Field
import os

DB_FOLDER = os.path.join(os.path.dirname(__file__), 'databases')
db = DAL('sqlite://storage.db', folder=DB_FOLDER, pool_size=1)
db.define_table('visit_log', Field('client_ip'), Field('timestamp', 'datetime'))
db.commit()

@action('index')
@action.uses(db)
def index():
    client_ip = request.environ.get('REMOTE_ADDR')
    db.visit_log.insert(client_ip=client_ip, timestamp=datetime.utcnow())
    return "Your visit was stored in database"
```

?index() action ?on_success ?on_error ?

6.10 Auth ?

auth ?auth.user ?session ?db ?

Auth ?

```
from py4web import action, redirect, Session, DAL, URL
from py4web.utils.auth import Auth
import os

session = Session(secret='my secret key')
DB_FOLDER = os.path.join(os.path.dirname(__file__), 'databases')
db = DAL('sqlite://storage.db', folder=DB_FOLDER, pool_size=1)
auth = Auth(session, db)
auth.enable()

@action('index')
@action.uses(auth)
def index():
    user = auth.get_user() or redirect(URL('auth/login'))
    return 'Welcome %s' % user.get('first_name')
```

Auth ?auth_user ?

?auth.enable() ?auth_user ?auth_user ?

?extra_fields ?auth_user ?

```
extra_fields = [
   Field("favorite_color"),
]
auth = Auth(session, db, extra_fields=extra_fields)
```

?*auth_user* ?

auth ?auth.enable() ?{appname}/auth/login ?auth.html ?_scaffold ?auth ?

```
auth.get_user()
```

?None ?auth/login ?

?auth.user ?

```
@action('index')
@action.uses(auth.user)
def index():
    user = auth.get_user()
    return 'Welcome %s' % user.get('first_name')
```

?auth/login ?

?auth ?Chapter 13 ?

6.11 ?

?

```
from py4web import action, request, DAL, Field
from py4web.utils.form import Form
import os

DB_FOLDER = os.path.join(os.path.dirname(__file__), 'databases')
db = DAL('sqlite://storage.db', folder=DB_FOLDER, pool_size=1)
db.define_table('thing', Field('name', writable=False))

@action('index')
@action.uses('generic.html', db)
def index():
    db.thing.name.writable = True
    form = Form(db.thing)
    return dict(form=form)
```

?

?Section 7.5.1 ?

6.12 ?

?

```
from py4web.core import Fixture

class MyFixture(Fixture):
    def on_request(self, context): pass
    def on_success(self, context): pass
    def on_error(self, context) pass
```

?on_request ?on_success ?on_error ?

?on_request ?on_error ?on_success ?

?auth.user ?on_request ?

?

```
request  -> A.on_request -> B.on_request -> C.on_request -> action
response <- A.on_success <- B.on_success <- C.on_success <-
```

?on_request ?on_success ?on_request ?on_success ?

?on_error ?on_success ?

context ?

- content['fixtures']: ?
- context['processed']: ?on_request ?
- context['exception']: ?
- context['output']: ?

on_success ?on_error ?context['exception'] ?context['output'] ?

?

```
class UpperCase(Fixture):
    def on_success(self, context):
        context['output'] = context['output'].upper()

upper_case = UpperCase()

@action('index')
@action.uses(upper_case)
def index(): return "hello world"
```

?context['output'] ?

?

```
class LogErrors(Fixture):
    def __init__(self, filename):
        self.filename = filename
    def on_error(self, context):
        with open(self.filename, "a") as stream:
            stream.write(str(context['exception']) + '\n')

errlog = LogErrors("myerrors.log")

@action('index')
@action.uses(errlog)
def index(): return 1/0
```

?__prerequisite__ ?__prerequisites__ ?__prerequisites__ ?action.uses ?

?Auth ?db ?session ?flash ?db ?session ?flash ?Auth ?

```
action.uses(auth)
```

?

```
action.uses(auth, session, db, flash)
```

?

?py4web.HTTP ?bottle.HTTResponse ?

?

?

6.12.1 ?

?__prerequisites__ ?

```
class AdminAccess(Fixture):

    def __init__(self, auth, admin_list, redirect_url=None):
```

```
        super().__init__()
        self.admin_list = admin_list
        self.auth = auth
        self.__prerequisites__ = [auth]
        # One thing to note here is that the URL function can only be called in a
        # request context (while serving a request).  Thus, we cannot store in
the fixture
        # initialization the full URL to redirect, but only the path.
        self.redirect_url = redirect_url or 'index'

    def on_request(self, context):
        if ((not self.auth.current_user)
            or self.auth.current_user.get('email') not in self.admin_list):
            redirect(URL(self.redirect_url))

    def on_error(self, context):
        redirect(URL(self.redirect_url))
```

?

```
admin_access = AdminAccess(auth, ['a@example.com',], 'index')

@action('/admin-only')
@action.uses('admin_only.html', admin_access)
def admin_only():
    return dict()
```

6.12.2 ?

Fixtures ?

```
class LocalStorageDemo(Fixture):

   def __init__(self):
       super().__init__()

   def on_request(self, context):
       Fixture.local_initialize(self)
       # We can check whether the local storage is valid.
       print(f"is_valid: {self.is_valid()}")
       content = str(uuid.uuid4())
       print(f"Storing content: {content}")
       self.local.my_content = content

   def on_success(self, context):
       # The line below is used only to show that the thread-local object is in
place.
       print(f"Retrieved: {self.local.my_content}")
```

?

```
Fixture.local_initialize(self)
```

?`self.local`?

6.13 ?

?**?**?

```
@action("index")
@action.uses(A,B)
```

```
def func(): return "Hello world"
```

fixture ?on_request ?on_success ?on_error ?

?

```
A.on_request()
B.on_request()
func()
B.on_success()
A.on_success()
```

?

?

```
@action.uses(A)
@action.uses(B)
```

?

```
@action.uses(A,B)
```

?action.uses ?action.uses ?action.uses ?

6.14 ?

py4web ?

```
import uuid
from py4web import Cache, action
cache = Cache(size=1000)

@action('hello/<name>')
@cache.memoize(expiration=60)
def hello(name):
    return "Hello %s your code is %s" % (name, uuid.uuid4())
```

?hello ?name ?

cache ?@action.uses ?

6.15 ?

_scaffold ?common.py ?

```
@unauthenticated()
def index():
    return dict()
```

?

```
@authenticated()
def index():
    return dict()
```

?

- @unauthenticated ?
- @authenticated ?

Warning ?

?

```
@authenticated.get()
def index():
    # only handle GET requests
    return dict()

@authenticated.post(path="index")
def index_form():
    # only handle POST requests
    return dict()
```

?

- `path` ?
- `template` ?
- `uses` ?

```
@authenticated(
   path="test",
   template="generic.html",
   uses=[Inject(message="Hello World")])
def example():
    return dict()
```

?`uses` ?Section 6.12.1

?

7.1 DAL ?

py4web ?**DAL**?pyDAL ?

Note pyDAL ?**?** ?**ORM** ?

pyDAL ?

- ?
- ?
- ?
- ?

Note py4web ?Section 7.5.1 ?Chapter 15 ?

7.1.1 ?

?

Note ?**SQLite** ?

Windows ?

?

?	?
SQLite	sqlite3 or pysqlite2 or zxJDBC (on Jython)
PostgreSQL	psycopg2 or zxJDBC (on Jython)
MySQL	pymysql or MySQLdb
Oracle	cx_Oracle
MSSQL	pyodbc or pypyodbc
FireBird	kinterbasdb or fdb or pyodbc
DB2	pyodbc
Informix	informixdb
Ingres	ingresdbi
Cubrid	cubriddb
Sybase	Sybase

Teradata	pyodbc
SAPDB	sapdb
MongoDB	pymongo
IMAP	imaplib

?? (Section 7.15)?

7.1.2 DAL: ?

py4web ?

DAL

?

```
db = DAL('sqlite://storage.sqlite')
```

Table

?? ?DAL.define_table ?

```
db.define_table('mytable', Field('myfield'))
```

? ?

insert ?truncate ?drop ?import_from_csv_file ?

Field

?? ?DAL.define_table ?

Rows

Rows ?Row ?

```
rows = db(db.mytable.myfield != None).select()
```

Row

?

```
for row in rows:
    print(row.myfield)
```

Query

Query ?

```
myquery = (db.mytable.myfield != None) | (db.mytable.myfield > 'A')
```

Set Set ?count ?select ?update ?delete ?

```
myset = db(myquery)
rows = myset.select()
myset.update(myfield='somevalue')
myset.delete()
```

? **?** ?? ?

```
myorder = db.mytable.myfield.upper() | db.mytable.id
db().select(db.table.ALL, orderby=myorder)
```

7.1.3 ?

pyDAL ?

```
>>> from pydal import DAL, Field
```

Note ?py4web.DAL ?? ?pydal.DAL ?

```
>>> from py4web import DAL, Field
```

7.1.4 ?

?Section 3.6.6 ?

Warning ?

?>>> ?

?showcase ?

```
>>> from apps.showcase.examples.models import db
>>> db.tables()
['auth_user', 'auth_user_tag_groups', 'person', 'superhero', 'superpower', 'tag',
'thing', 'user_token', 'dummy']
>>> rows = db(db.superhero.name != None).select()
>>> rows.first()
<Row {'id': 1, 'tag': <Set ("tag"."superhero" = 1)>, 'name': 'Superman',
'real_identity': 1}>
```

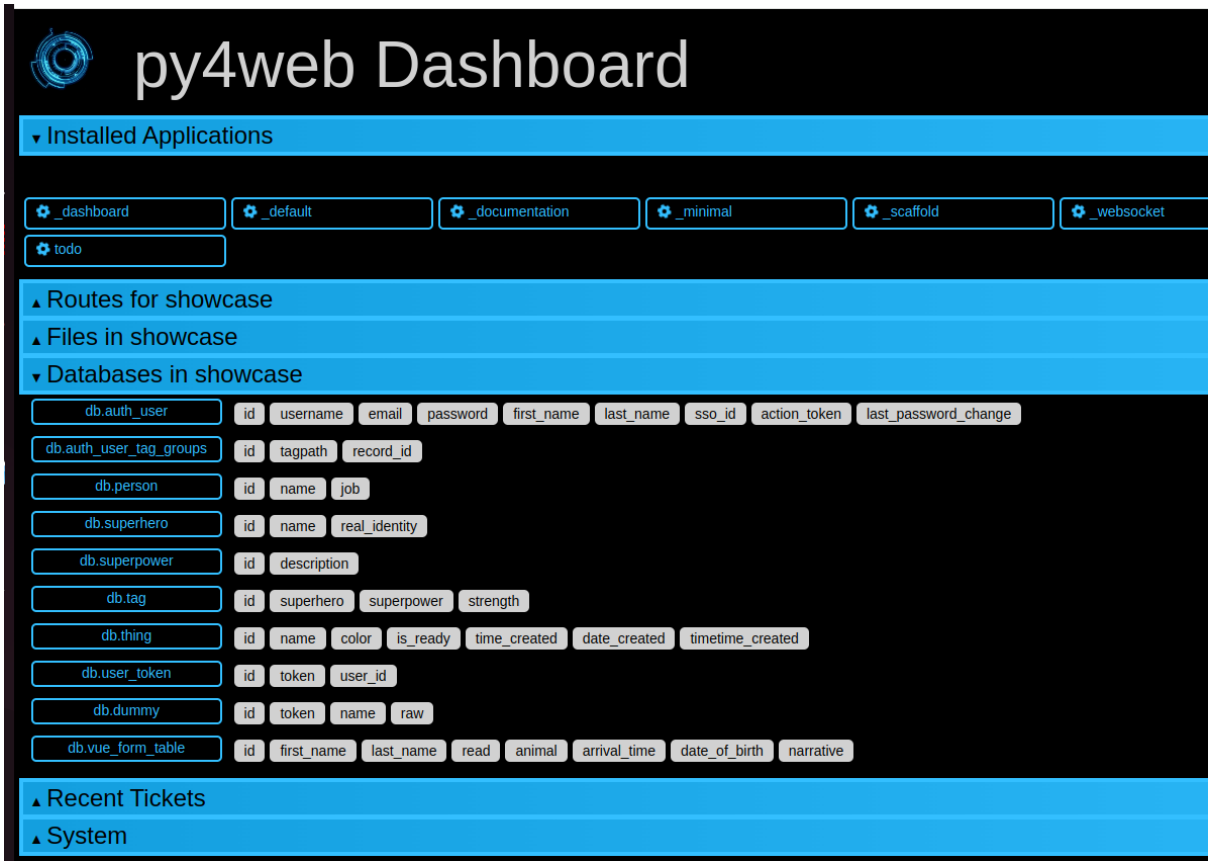
?

7.2 ?

?apps/showcase/__init__.py ?

```
from .examples.models import db
```

?



7.3 DAL ?

?

```
>>> db = DAL('sqlite://storage.sqlite')
```

?**?** ?db ?

?

```
>>> db._uri
sqlite://storage.sqlite
```

?

```
>>> db._dbname
sqlite
```

?`_uri` ?

DAL ?

7.3.1 DAL ?

```
DAL(uri='sqlite://dummy.db',
    pool_size=0,
    folder=None,
    db_codec='UTF-8',
    check_reserved=None,
    migrate=True,
```

```
    fake_migrate=False,
    migrate_enabled=True,
    fake_migrate_all=False,
    decode_credentials=False,
    driver_args=None,
    adapter_args=None,
    attempts=5,
    auto_import=False,
    bigint_id=False,
    debug=False,
    lazy_tables=False,
    db_uid=None,
    do_connect=True,
    after_connection=None,
    tables=None,
    ignore_field_case=True,
    entity_quoting=False,
    table_hash=None)
```

7.3.2 ?

?

```
db = DAL('sqlite://storage.sqlite')
```

db ?DAL ?

?	?
SQLite	`sqlite://storage.sqlite`
MySQL	`mysql://username:password@localhost/test?set_encoding=utf8mb4`
PostgreSQL	`postgres://username:password@localhost/test`
MSSQL (legacy)	`mssql://username:password@localhost/test`
MSSQL (>=2005)	`mssql3://username:password@localhost/test`
MSSQL (>=2012)	`mssql4://username:password@localhost/test`
FireBird	`firebird://username:password@localhost/test`
Oracle	`oracle://username/password@test`
DB2	`db2://username:password@test`
Ingres	`ingres://username:password@localhost/test`
Sybase	`sybase://username:password@localhost/test`
Informix	`informix://username:password@test`
Teradata	`teradata://DSN=dsn;UID=user;PWD=pass;DATABASE=test`
Cubrid	`cubrid://username:password@localhost/test`
SAPDB	`sapdb://username:password@localhost/test`
IMAP	`imap://user:password@server:port`
MongoDB	`mongodb://username:password@localhost/test`
Google/SQL	`google:sql://project:instance/database`
Google/NoSQL	`google:datastore`
Google/NoSQL/NDB	`google:datastore+ndb`

- ?
- ?
- ??set_encoding=utf8mb4 ?Invalid utf8 character string: ?
- ?+ndb ?
- ?None ?

?

```
db = DAL('...', do_connect=False)
```

?_select ?_insert ?_update ?_delete ?select ?_insert ?update ?delete ?Section 7.8.5 ?do_connect=False ?

?db_codec ?

```
db = DAL('...', db_codec='latin1')
```

?

7.3.3 ?

DAL ?pool_size?

?

py4web ?pool_size ?pool_size=10 ?pool_size=0 ?

?

pool_size ?

7.3.4 ?

?attempts ?

7.3.5 ?

?lazy_tables = True ?

Warning ?

7.3.6 “?

?**actions** ?

?**actions** ?

?

7.3.7 ?

DAL(...) ?

```
db = DAL(['mysql://...1', 'mysql://...2', 'mysql://...3'])
```

?

7.3.8 ?

check_reserved ?

check_reserved ?

?db ?

```
db = DAL('sqlite://storage.sqlite', check_reserved=['postgres', 'mssql'])
```

DAL ?

?SELECT ?INSERT ?UPDATE ?

?_nonreserved ?

```
check_reserved=['postgres', 'postgres_nonreserved']
```

?

?	check_reserved ?
PostgreSQL	`postgres(_nonreserved)`
MySQL	`mysql`
FireBird	`firebird(_nonreserved)`
MSSQL	`mssql`
Oracle	`oracle`

7.3.9 ?

DAL ?

`entity_quoting = True`

?

?

?

`ignore_field_case = False`

?

```
db = DAL(ignore_field_case=False)
db.define_table('table1', Field('column'), Field('COLUMN'))
query = db.table1.COLUMN != db.table1.column
```

7.3.10 ?

?**?** ?

?

```
DAL('postgres://user_name:user_password@server_addr/db_name',
    driver_args={'sslmode': 'require', 'sslrootcert': 'root.crt',
                 'sslcert': 'postgresql.crt', 'sslkey': 'postgresql.key'})
```

?`sslrootcert` ?`sslcert` ?`sslkey` ?

7.3.11 DAL ?

?

`folder` ?Section 7.6 ?

?

```
db = DAL('sqlite://storage.sqlite') # folder parameter not specified
db = DAL('sqlite:memory')           # in memory database
```

?

DAL ?Section 7.6 ?

`migrate = True` ?

`fake_migrate = False` ?**?** ?

`migrate_enabled = True` ?

`fake_migrate_all = False` ?

7.3.12 commit ?rollback

?

?action.uses() ?db ?authenticated ?unauthenticated ?

Tip ?action.uses ?db ?authenticated ?unauthenticated ?db.commit()

?commit ?rollback ?

?

```
>>> db.commit()
```

?

```
>>> db.person.insert(name="Bob")
2
```

???

```
>>> db.rollback()
```

?

```
>>> db.person.insert(name="Bob")
2
```

models?

```
try:
    execute models, controller function and view
except:
    rollback all connections
    log the traceback
    send a ticket to the visitor
else:
    commit all connections
    save cookies, sessions and return the page
```

7.4 “Table” ?

? ?define_table() ?

7.4.1 define_table ?

define_table() ?

```
define_table(tablename, *fields, **kwargs)
```

?Field ?Field ?Table ?rname ?redefine ?common_filter ?fake_migrate ?fields ?format ?migrate ?on_define ?plural ?polymodel ?primarykey ?sequence_name ?singular ?table_class ?trigger_name?

?

```
>>> db.define_table('person', Field('name'))
<Table person (id, name)>
```

?Table ?db.person ?define_table() ?

7.4.2 id : ?

?

?

7.4.3 plural ?singular

?plural ?singular ?

7.4.4 redefine

?

```
db.define_table('person', Field('name'))
db.define_table('person', Field('name'), redefine=True)
```

?

7.4.5 format: ?

?format ?

```
db.define_table('person', Field('name'), format='%(name)s')
```

?

```
db.define_table('person', Field('name'), format='%(name)s %(id)s')
```

?

```
db.define_table('person', Field('name'),
                format=lambda r: r.name or 'anonymous')
```

format ?

- ?
- ?db.othertable.otherfield.represent ?Form ?

7.4.6 rname: ?

rname ?rname ?rname ?rname = 'db1.dbo.table1'

7.4.7 primarykey: ?

primarykey ?Section 7.4.15 ?

7.4.8 migrate, fake_migrate

migrate ?Section 7.6

7.4.9 table_class

?

```
from pydal.objects import Table

class MyTable(Table):
    ...

db.define_table(..., table_class=MyTable)
```

7.4.10 sequence_name

?

?

7.4.11 trigger_name

?sequence_name ?

7.4.12 polymodel

?

7.4.13 on_define

on_define ?

?

```
db = DAL(lazy_tables=True)
db.define_table('person',
    Field('name'),
    Field('age', 'integer'),
    on_define=lambda table: [
        table.name.set_attributes(requires=IS_NOT_EMPTY(), default=''),
        table.age.set_attributes(requires=IS_INT_IN_RANGE(0, 120), default=30) ])
```

?on_define ?requires ?requires ?db.sometable.somefield == some_value ?sometable ?on_define ?

7.4.14 ?

?db.table.field.extra = {}

“extra” ?

```
db.table._extra = {}
```

7.4.15 ?

?

?

- ?
- ?

?migrate=False?

?Field('...', 'id') ?

?

```
db.define_table('account',
                Field('accnum', 'integer'),
                Field('acctype'),
                Field('accdesc'),
                primarykey=['accnum', 'acctype'],
                migrate=False)
```

- primarykey ?
- ?NOT NULL ?

- “?
- ?`reference tablename.fieldname` ?
- `update_record` ?

?

?`primarykey` ?

7.5 “Field” ?

?

```
Field(fieldname, type='string', length=None, default=DEFAULT,
      required=False, requires=DEFAULT,
      ondelete='CASCADE', notnull=False, unique=False,
      uploadfield=True, widget=None, label=None, comment=None,
      writable=True, readable=True, searchable=True, listable=True,
      update=None, authorize=None, autodelete=False, represent=None,
      uploadfolder=None, uploadseparate=None, uploadfs=None,
      compute=None, filter_in=None, filter_out=None,
      custom_qualifier=None, map_none=None, rname=None)
```

?

?`length` ?`uploadfield` ?`authorize` ?`autodelete` ?`ondelete` ?

- `length` ?`length` ??
- `default` ?`Form` ?
- `required` ?
- `requires` ?**?**?`Form` ?Chapter 12 ?Section 7.5.2 ?web2py ?

  Note `requires=...` ?`required=True` ?`notnull` ?`unique` ?`ondelete` ?

- `rname` ?
- `ondelete` ?`ondelete` ?
- `notnull=True` ?
- `unique=True` ?
- `uploadfield` ?`uploadfield` ?`uploadfield` ?Section 7.5.4 ?
- `uploadfolder` ?`settings.UPLOAD_FOLDER` ?`apps/{app_name}/uploads` ?`Field(... uploadfolder=settings.UPLOAD_FOLDER)` ?
- ?`uploadseparate` ?*uploadfolder* ?`uploadseparate` ?
- `uploadfs` ?

  ?`uploadfs` ?
- `autodelete` ?
- `label` ?
- `writable` ?
- `readable` ?
- `update` ?
- `compute` ?`dict` ?
- `authorize` ?

- `widget` ?? ?Section 12.4
- `represent` ?

7.5.1 ?

?`Field` ?

- `readable`
- `writable`
- `default`
- `filter_in`
- `filter_out`
- `label`
- `update`
- `requires`
- `widget`
- `represent`

?`Field`?`DAL` ?`Table` ?

?

7.5.2 ?

?	?
`string`	`IS_LENGTH(length)` ?
`text`	`IS_LENGTH(length)` ?
`blob`	`None` ?
`boolean`	`None`
`integer`	`IS_INT_IN_RANGE(-2**31, 2**31)`
`double`	`IS_FLOAT_IN_RANGE(-1e100, 1e100)`
`decimal(n,m)`	`IS_DECIMAL_IN_RANGE(-10**10, 10**10)`
`date`	`IS_DATE()`
`time`	`IS_TIME()`
`datetime`	`IS_DATETIME()`
`password`	`IS_LENGTH(length)` ?
`upload`	`None` ?
`reference <table>`	`IS_IN_DB(db, table.field, format)`
`list:string`	`None`
`list:integer`	`None`
`list:reference <table>`	`IS_IN_DB(db, table._id, format, multiple=True)`
`json`	`IS_EMPTY_OR(IS_JSON())` ?
`bigint`	`IS_INT_IN_RANGE(-2**63, 2**63)`
`big-id`	`None`
`big-reference`	`None`

Decimal ?`decimal` ?`decimal` ?`double`?

`big-id` ?`big-reference` ?`bigint_id` ?`id` ?`reference` ?`big-id` ?`big-reference` ?

`list:<type>` ?`ListProperty` ?`StringListProperty`?`|` ?`|` ?`||` ?Section 7.14.1 ?

json ?

blob ?

7.5.3 ?

?

```
>>> db.define_table('person', Field('name', default=''), format='%(name)s')
<Table person (id, name)>
>>> db.person._format = '%(name)s/%(id)s'
>>> db.person.name.default = 'anonymous'
```

?

?

```
>>> db.tables
['person']
```

?

```
>>> type(db.person)
<class 'pydal.objects.Table'>
```

?

```
>>> db.person is db['person']
True
```

?

```
>>> db.person.fields
['id', 'name']
```

?

```
>>> type(db.person.name)
<class 'pydal.objects.Field'>
>>> db.person.name is db.person['name']
True
```

?

```
>>> db.person.name.type
string
>>> db.person.name.unique
False
>>> db.person.name.notnull
False
>>> db.person.name.length
32
```

?

```
>>> db.person.name._table == db.person
True
>>> db.person.name._tablename == 'person'
True
>>> db.person.name._db == db
True
```

?validate ?

```
>>> db.person.name.validate('John')
```

```
('John', None)
```

?(value, error)?error ?

7.5.4 “?

?

```
db.define_table('myfile',
                Field('image', 'upload', default='path/to/file'))
```

?

?autodelete ?default ?

```
Field('image', 'upload', default=dict(data='<file_content>',
filename='<file_name>'))
```

?Form ?

```
with open(filename, 'rb') as stream:
    db.myfile.insert(image=db.myfile.image.store(stream, filename))
```

?store ?

```
with open(filename, 'rb') as stream:
    db.myfile.insert(image=stream)
```

?

?store ?db.myfile ?image ?

?store ?store ?

```
db.define_table('myfile',
                Field('image', 'upload', uploadfield='image_file'),
                Field('image_file', 'blob'))
with open(filename, 'rb') as stream:
    db.myfile.insert(image=db.myfile.image.store(stream, filename),
                     image_file=stream.read())
```

retrieve ?store ?

?Field('image', 'upload') ?

```
row = db(db.myfile).select().first()
(filename, fullname) = db.myfile.image.retrieve(row.image, nameonly=True)
```

?

```
(filename, stream) = db.myfile.image.retrieve(row.image)
```

?

?retrieve ?stream.close() ?

?retrieve ?

```
from contextlib import closing
import shutil
row = db(db.myfile).select().first()
(filename, stream) = db.myfile.image.retrieve(row.image)
with closing(stream) as src, closing(open(filename, 'wb')) as dest:
    shutil.copyfileobj(src, dest)
```

7.6 ?

?

```
db.define_table('person')
```

define_table ?db.person ?

?

Note ?folder ?db = DAL(..., adapter_args=dict(logfile='migrate.log')) ?

---

define_table ?

```
db.define_table('person', ..., migrate='person.table')
```

migrate ?define_table ?

?

DAL?define_table ?

```
db = DAL('sqlite://storage.sqlite', migrate=False)
```

?db.define_table ?

Note ?default ?unique ?notnull ?ondelete ?

?

```
db = DAL(..., migrate_enabled=False)
```

?

7.6.1 ?

?

SQLite ?

?

?

?

?

```
db.define_table(...., migrate=True, fake_migrate=True)
```

?fake_migrate=True ?

?

?

```
db = DAL(..., fake_migrate_all=True)
```

?

7.6.2 ?

?

```
if DAL.migrate_enabled and table.migrate:
   if DAL.fake_migrate_all or table.fake_migrate:
       perform fake migration
   else:
       perform migration
```

7.7 “Table” ?

7.7.1 insert

?

```
>>> db.person.insert(name="Alex")
1
>>> db.person.insert(name="Bob")
2
```

Insert() ?

?

```
>>> db.person.truncate()
```

?

```
>>> db.person.insert(name="Alex")
1
```

?truncate ?

```
>>> db.person.truncate('RESTART IDENTITY CASCADE')
```

?

py4web ?

```
>>> db.person.bulk_insert([{'name': 'Alex'}, {'name': 'John'}, {'name': 'Tim'}])
[3, 4, 5]
```

?

7.7.2 Query, Set, Rows

?

```
>>> db.define_table('person', Field('name'))
<Table person (id, name)>
>>> db.person.insert(name="Alex")
1
>>> db.person.insert(name="Bob")
2
>>> db.person.insert(name="Carl")
3
```

?

```
>>> person = db.person
```

?

```
>>> name = person.name
```

?== ?!= ?< ?> ?<= ?>= ?.like() ?.belongs() ?

```
>>> q = name == 'Alex'
```

?db ?

```
>>> s = db(q)
```

?

7.7.3 update_or_insert

?

```
db.define_table('person',
                Field('name'),
                Field('birthplace'))

db.person.update_or_insert(name='John', birthplace='Chicago')
```

?

?

```
db.person.update_or_insert(db.person.name == 'John',
                           name='John',
                           birthplace='Chicago')
```

?

?

```
db.person.update_or_insert((db.person.name == 'John') & (db.person.birthplace ==
'Chicago'),
                           name='John',
                           birthplace='Chicago',
                           pet='Rover')
```

7.7.4 validate_and_insert, validate_and_update

?

```
ret = db.mytable.validate_and_insert(field='value')
```

?

```
id = db.mytable.insert(field='value')
```

?ret["errors"] ?ret["errors"] ?form["errors"] ?ret["id"] ?

?

```
ret = db(query).validate_and_update(field='value')
```

?

```
num = db(query).update(field='value')
```

?ret["updated"] ?ret["errors"] ?

7.7.5 drop

?

```
db.person.drop()
```

7.7.6 QueryBuilder() ?

?

```
from py4web import DAL, Field
from pydal import QueryBuilder, QueryParseError

db = DAL("sqlite:memory")
db.define_table("thing", Field("name"), Field("solid", "boolean"))
builder = QueryBuilder(db.thing)
query = builder.parse('name is equal to Chair')
rows = db(query).select()
```

?

- name is null
- name is not null
- name == Chair
- name is Chair
- name is equal Chair
- name is equal to Chair
- name is equal to "Chair"
- name lower is equal to Chair
- not name lower is equal to Chair
- not (name lower is equal to Chair)
- name == Chair or name == Table
- name starts with C and name contains air
- name in Chair, Table, Glass
- name belongs Chair, Table, "Glass Top"
- solid is true
- solid is false

?

```
builder = QueryBuilder(db.thing,
            field_aliases={"id": "id", "nome": "name"},
            token_aliases={"non è nullo": "is not null", "è uguale a": "=="})
query = builder.parse('nome non è nullo')
query = builder.parse('nome è uguale a Tavolo')
```

?

?

```
builder = QueryBuilder(db.thing,
            field_aliases={str(T(field.name)): field name for field in db.thing},
            token_aliases={str(T(key):key) for key in QueryBuilder.token_ops})
```

?

```
{
   # boolean tokens
   "not",
   "and",
   "or",
```

```
    # field transformers
    "upper",
    "lower",
    # unary search expressions
    "is null",
    "is not null",
    "is true",
    "is false",
    # binary search expressions
    "==",
    "!=",
    "<",
    ">",
    "<=",
    ">=",
    "contains",
    "startswith", # notice "starts with" is an alias!
    # search in list
    "belongs",
}
```

?

7.7.7 ?

?

```
from py4web import DAL, Field
from pydal.tools.tags import Tags

db = DAL("sqlite:memory")
db.define_table("thing", Field("name"))
id1 = db.thing.insert(name="chair")
id2 = db.thing.insert(name="table")
properties = Tags(db.thing)
properties.add(id1, "color/red")
properties.add(id1, "style/modern")
properties.add(id2, "color/green")
properties.add(id2, "material/wood")

assert properties.get(id1) == ["color/red", "style/modern"]
assert properties.get(id2) == ["color/green", "material/wood"]

rows = db(properties.find(["style/modern"])).select()
assert rows.first().id == id1

rows = db(properties.find(["material/wood"])).select()
assert rows.first().id == id2

rows = db(properties.find(["color"])).select()
assert len(rows) == 2
```

`Tags` ?`find(["color"])` ?

?`Tags(table, tail="default")` ?

py4web ?`Tags` ?Section 13.2 ?

7.8 ?

7.8.1 executesql

DAL ?

```
>>> db.executesql('SELECT * FROM person;')
[(1, u'Massimo'), (2, u'Massimo')]
```

?

executesql ?placeholders ?as_dict ?fields ?colnames ?as_ordered_dict ?

placeholders ?

?as_dict ?as_dict = True ?

```
[{'field1': val1_row1, 'field2': val2_row1}, {'field1': val1_row2, 'field2':
val2_row2}]
```

as_ordered_dict ?as_dict ?

```
[OrderedDict([('field1', val1_row1), ('field2', val2_row1)]),
 OrderedDict([('field1', val1_row2), ('field2', val2_row2)])]
```

fields ?fields ?

?fields ?colnames ?

?fields ?colnames?fields ?fields ?colnames ?fields ?colnames ?

?fields ?colnames ?fields ?colnames ?

7.8.2 _lastsql

?db._lastsql ?

```
>>> rows = db().select(db.person.ALL)
>>> db._lastsql
SELECT person.id, person.name FROM person;
```

py4web ?

7.8.3 ?

?db._timings ?

7.8.4 ?

?

?

```
db = DAL('sqlite://storage.sqlite')
db.define_table('person', Field('name'))
db.executesql('CREATE INDEX IF NOT EXISTS myidx ON person (name);')
```

?

7.8.5 ?

?

?_insert

```
>>> print(db.person._insert(name='Alex'))
INSERT INTO "person"("name") VALUES ('Alex');
```

?_count

```
>>> print(db(db.person.name == 'Alex')._count())
SELECT COUNT(*) FROM "person" WHERE ("person"."name" = 'Alex');
```

?_select

```
>>> print(db(db.person.name == 'Alex')._select())
SELECT "person"."id", "person"."name" FROM "person" WHERE ("person"."name" =
'Alex');
```

?_delete

```
>>> print(db(db.person.name == 'Alex')._delete())
DELETE FROM "person" WHERE ("person"."name" = 'Alex');
```

?_update

```
>>> print(db(db.person.name == 'Alex')._update(name='Susan'))
UPDATE "person" SET "name"='Susan' WHERE ("person"."name" = 'Alex');
```

?db._lastsql ?

7.9 select ?

?s ?select ?

```
>>> rows = s.select()
```

?pydal.objects.Rows ?Row ?pydal.objects.Row ?Row ?

Rows ?

```
>>> for row in rows:
...     print(row.id, row.name)
...
1 Alex
```

?

```
>>> for row in db(db.person.name == 'Alex').select():
...     print(row.name)
...
Alex
```

select ?

```
>>> for row in db().select(db.person.id, db.person.name):
...     print(row.name)
...
Alex
Bob
Carl
```

?

```
>>> for row in db().select(db.person.ALL):
...     print(row.id, row.name)
...
```

```
1 Alex
2 Bob
3 Carl
```

?

?

```
>>> for row in db(db.person).select():
...     print(row.id, row.name)
...
1 Alex
2 Bob
3 Carl
```

?

?

```
>>> row = rows[0]
```

?

```
>>> row.name
Alex
>>> row['name']
Alex
>>> row('person.name')
Alex
```

?

?

```
rows.compact = False
```

?

```
rows[i].name
```

?

```
rows[i].person.name
```

?

Row ?

```
row.delete_record()
```

?

```
row.update_record(name="new value")
```

7.9.1 ?

Python ?

?

```
for row in db(db.table).select():
    ...
```

?

```
for row in db(db.table).iterselect():
    ...
```

?

7.9.2 ?

?

```
rows = db(query).select()
repr_row = rows.render(0)
```

?

```
for row in rows.render():
    print(row.myfield)
```

?

```
for row in rows[0:10].render():
    print(row.myfield)
```

?

```
repr_row = row.render(0, fields=[db.mytable.myfield])
```

?

7.9.3 ?

DAL ?

```
myrecord = db.mytable[id]
```

?`id` ?`id` ?`None` ?

```
myrecord = db(db.mytable.id == id).select().first()
```

?

```
del db.mytable[id]
```

?

```
db(db.mytable.id == id).delete()
```

?`id` ?

?*versioning* ?

?

```
db.mytable[None] = dict(myfield='somevalue')
```

?

```
db.mytable.insert(myfield='somevalue')
```

?

?`db.table[0] = ...` ?

?

```
db.mytable[id] = dict(myfield='somevalue')
```

?

```
db(db.mytable.id == id).update(myfield='somevalue')
```

?

7.9.4 ?Row

?

```
record = db.mytable(id)
record = db.mytable(db.mytable.id == id)
record = db.mytable(id, myfield='somevalue')
```

?db.mytable[id] ?id ?str(id) ?None ?None ?

7.9.5 ?select

?

```
db.define_table('thing',
                Field('name'),
                Field('owner_id', 'reference person'))
```

?

```
things = db(db.thing).select()
```

?

```
things = db(db.thing._id != None).select()
```

?_id ?db.thing._id ?db.thing.id ?

?

```
for thing in things:
    print(thing.name, thing.owner_id.name)
```

?thing.owner_id.name ?

?

```
person = db.person(id)
for thing in person.thing.select(orderby=db.thing.name):
    print(person.name, 'owns', thing.name)
```

?person.thing ?

```
db(db.thing.owner_id == person.id)
```

?person ?thing ?thing ?person ?

7.9.6 orderby, groupby, limitby, distinct, having, orderby_on_limitby, join, left, cache

select ?

orderby

?

```
>>> for row in db().select(db.person.ALL, orderby=db.person.name):
...     print(row.name)
...
Alex
Bob
Carl
```

?

```
>>> for row in db().select(db.person.ALL, orderby=~db.person.name):
...     print(row.name)
...
Carl
Bob
Alex
```

?

```
>>> for row in db().select(db.person.ALL, orderby='<random>'):
...     print(row.name)
...
Carl
Alex
Bob
```

Google NoSQL ?`orderby='<random>'` ?

```
import random
rows = db(...).select().sort(lambda row: random.random())
```

?

```
>>> for row in db().select(db.person.name, orderby=db.person.name|db.person.id):
...     print(row.name)
...
Alex
Bob
Carl
```

groupby, having

?`groupby` ?`orderby` ?

```
>>> for row in db().select(db.person.ALL,
...                        orderby=db.person.name,
...                        groupby=db.person.name):
...     print(row.name)
...
Alex
Bob
Carl
```

?`having` ?`groupby` ?`having` ?

```
db(query1).select(db.person.ALL, groupby=db.person.name, having=query2)
```

?

distinct

?`distinct=True` ?

?

```
>>> for row in db().select(db.person.name, distinct=True):
...     print(row.name)
...
Alex
Bob
Carl
```

?`distinct` ?

```
>>> for row in db().select(db.person.name, distinct=db.person.name):
...     print(row.name)
...
Alex
Bob
Carl
```

limitby

?`limitby=(min, max)` ?

```
>>> for row in db().select(db.person.ALL, limitby=(0, 2)):
...     print(row.name)
...
Alex
Bob
```

orderby_on_limitby

?`orderby_on_limitby = False` ?

?

?Section 7.11.1 ?Section 7.11.2 ?Section 7.11.3 ?

cache, cacheable

?

```
rows = db(query).select(cache=(cache.get, 3600), cacheable=True)
```

?Section 7.9.17 ?

7.9.7 ?

?&" ?

```
>>> rows = db((db.person.name=='Alex') & (db.person.id > 3)).select()
>>> for row in rows: print row.id, row.name
>>> len(rows)
0
```

?|" ?

```
>>> rows = db((db.person.name == 'Alex') | (db.person.id > 3)).select()
>>> for row in rows: print row.id, row.name
1 Alex
```

?

```
>>> rows = db((db.person.name != 'Alex') | (db.person.id > 3)).select()
>>> for row in rows: print row.id, row.name
2 Bob
3 Carl
```

?~" ?

```
>>> rows = db(~(db.person.name == 'Alex') | (db.person.id > 3)).select()
>>> for row in rows: print row.id, row.name
2 Bob
3 Carl
```

?and" ?or" ?&" ?|" ?and" ?or" ?~" ?~ ?

?

```
>>> query = db.person.name != 'Alex'
>>> query &= db.person.id > 3
>>> query |= db.person.name == 'John'
```

7.9.8 count, isempty, delete, update

?

```
>>> db(db.person.name != 'William').count()
3
```

?count ?distinct ?select ?count ?cache ?select ?

?isempty ?count ?

```
>>> db(db.person).isempty()
False
```

?

```
>>> db(db.person.id > 3).delete()
0
```

delete ?

?

```
>>> db(db.person.id > 2).update(name='Ken')
1
```

update ?

7.9.9 ?

?

```
db.define_table('person',
                Field('name'),
                Field('visits', 'integer', default=0))

db(db.person.name == 'Massimo').update(visits = db.person.visits + 1)
```

?

```
db.define_table('person',
                Field('name'),
                Field('visits', 'integer', default=0),
                Field('clicks', 'integer', default=0))

db(db.person.visits == db.person.clicks + 1).delete()
```

7.9.10 case

?

```
>>> condition = db.person.name.startswith('B')
>>> yes_or_no = condition.case('Yes', 'No')
>>> for row in db().select(db.person.name, yes_or_no):
...     print(row.person.name, row[yes_or_no])  # could be row(yes_or_no) too
...
Alex No
Bob Yes
Ken No
```

7.9.11 update_record

?update_record ?

```
>>> row = db(db.person.id == 2).select().first()
>>> row.update_record(name='Curt')
<Row {'id': 2, 'name': 'Curt'}>
```

update_record ?

```
>>> row.update(name='Curt')
```

?update ?update_record ?

?update_record() ?

```
>>> row = db(db.person.id > 2).select().first()
>>> row.name = 'Philip'
>>> row.update_record() # saves above change
<Row {'id': 3, 'name': 'Philip'}>
```

?row ?update ?Field('modified_on', update=datetime.datetime.utcnow) ?row.update_record() ?row.update_record() ?row ?? ?update ?auth.signature ?

?id ?cacheable ?True``? ?

7.9.12 ?

?tablename, fieldname ?value?

?

```
db[tablename].insert(**{fieldname:value})
```

?

```
db(db[tablename]._id == id).update(**{fieldname:value})
```

?table._id ?table.id?

7.9.13 commit ?rollback

?

```
rows = db(query).select()
first_row = rows.first()
last_row = rows.last()
```

?

```
first_row = rows[0] if len(rows) else None
last_row = rows[-1] if len(rows) else None
```

?first() ?last() ?orderby=db.table_name.id ?

7.9.14 as_dict ?as_list

?as_dict() ?as_list() ?

```
rows = db(query).select()
rows_list = rows.as_list()
first_row_dict = rows.first().as_dict()
```

?

```
rows = db(query).select()
session.rows = rows  # not allowed!
session.rows = rows.as_list()  # allowed!
```

7.9.15 ?

?

```
>>> print(rows1)
person.name
Max
Tim

>>> print(rows2)
person.name
John
Tim
```

?+?

```
>>> rows3 = rows1 + rows2
>>> print(rows3)
person.name
Max
Tim
John
Tim
```

?|?

```
>>> rows3 = rows1 | rows2
>>> print(rows3)
person.name
Max
Tim
John
```

?&?

```
>>> rows3 = rows1 & rows2
>>> print(rows3)
person.name
Tim
```

7.9.16 `find`, `exclude`, `sort`

?`find`?`exclude` ?`sort` ?`find` ?`exclude` ?`sort` ?

?

?

```
>>> db.define_table('person', Field('name'))
<Table person (id, name)>
>>> db.person.insert(name='John')
1
>>> db.person.insert(name='Max')
2
>>> db.person.insert(name='Alex')
3
>>> rows = db(db.person).select()
>>> for row in rows.find(lambda row: row.name[0]=='M'):
...     print(row.name)
```

```
...
Max
>>> len(rows)
3
>>> for row in rows.exclude(lambda row: row.name[0]=='M'):
...     print(row.name)
...
Max
>>> len(rows)
2
>>> for row in rows.sort(lambda row: row.name):
...     print(row.name)
...
Alex
John
```

?

```
>>> rows = db(db.person).select()
>>> rows = rows.find(lambda row: 'x' in row.name).sort(lambda row: row.name)
>>> for row in rows:
...     print(row.name)
...
Alex
Max
```

`sort` ?`reverse=True` ?

`find` ?`limitby` ?`select` ?

7.9.17 ?

select ?`cache` ?*(key, callback, expiration)* ?`cache.get` ?`cache` ?

?

```
def cache_db_select():
    logs = db().select(db.log.ALL, cache=(cache.get, 60))
    return dict(logs=logs)
```

`select` ?`cacheable` ?`cacheable=True` ?`Rows` ?`Rows` ?`update_record` ?`delete_record` ?

?`cacheable` ?

```
rows = db(query).select(cacheable=True)
```

?`cache` ?`cacheable=False` ?`cache` ?`cacheable=True` ?

```
rows = db(query).select(cache=(cache.get, 3600), cacheable=True)
```

7.10 ?

7.10.1 ?

DAL ?`compute` ?`compute` ?

```
>>> db.define_table('item',
...                 Field('unit_price', 'double'),
...                 Field('quantity', 'integer'),
...                 Field('total_price',
...                       compute=lambda r: r['unit_price'] * r['quantity']))
<Table item (id, unit_price, quantity, total_price)>
```

```
>>> rid = db.item.insert(unit_price=1.99, quantity=5)
>>> db.item[rid]
<Row {'total_price': '9.95', 'unit_price': 1.99, 'id': 1L, 'quantity': 5}>
```

?

- ?
- ?

?

7.10.2 ?

?

7.10.3 ?

py4web ?
?

```
db.define_table('item',
                Field('unit_price', 'double'),
                Field('quantity', 'integer'))
```

?total_price ?

```
db.item.total_price = Field.Virtual(lambda row: row.item.unit_price *
row.item.quantity)
```

?total_price ?Field.Virtual?

?

```
for row in db(db.item).select():
    print(row.total_price)
```

?

```
db.item.discounted_total = \
    Field.Method(lambda row, discount=0.0:
                 row.item.unit_price * row.item.quantity * (100.0 - discount /
100))
```

?row.discounted_total ?Method ?row ?self ?

?item ?

```
for row in db(db.item).select(): print(row.discounted_total())
```

?discount ?

```
for row in db(db.item).select(): print(row.discounted_total(15))
```

?

```
db.define_table('item',
                Field('unit_price', 'double'),
                Field('quantity', 'integer'),
                Field.Virtual('total_price', lambda row: ...),
                Field.Method('discounted_total', lambda row, discount=0.0: ...))
```

?db.table.fields ?

7.10.4 ?

?

```
db.define_table('item',
                Field('unit_price', 'double'),
                Field('quantity', 'integer'))
```

?total_price ?

```
class MyVirtualFields:
    def total_price(self):
        return self.item.unit_price * self.item.quantity

db.item.virtualfields.append(MyVirtualFields())
```

?self ?self.item.unit_price ?virtualfields ?

?

```
db.define_table('item',
                Field('unit_price', 'double'))

db.define_table('order_item',
                Field('item', 'reference item'),
                Field('quantity', 'integer'))

class MyVirtualFields:
    def total_price(self):
        return self.order_item.item.unit_price * self.order_item.quantity

db.order_item.virtualfields.append(MyVirtualFields())
```

?self.order_item.item.unit_price ?self ?

?

```
rows = db(db.order_item.item == db.item.id).select()

class MyVirtualFields:
    def total_price(self):
        return self.item.unit_price * self.order_item.quantity

rows.setvirtualfields(order_item=MyVirtualFields())

for row in rows:
    print(row.order_item.total_price)
```

?self.item.unit_price ?self.order_item.quantity ?setvirtualfields ?

```
class MyVirtualFields1:
    def discounted_unit_price(self):
        return self.item.unit_price * 0.90

class MyVirtualFields2:
    def total_price(self):
        return self.item.unit_price * self.order_item.quantity
    def discounted_total_price(self):
        return self.item.discounted_unit_price * self.order_item.quantity

rows.setvirtualfields(item=MyVirtualFields1(),
                      order_item=MyVirtualFields2())

for row in rows:
```

```
    print(row.order_item.discounted_total_price)
```

?

```
db.define_table('item',
                Field('unit_price', 'double'),
                Field('quantity', 'integer'))

class MyVirtualFields:
    def lazy_total_price(self):
        def lazy(self=self):
            return self.item.unit_price * self.item.quantity
        return lazy

db.item.virtualfields.append(MyVirtualFields())

for item in db(db.item).select():
    print(item.lazy_total_price())
```

?

```
class MyVirtualFields:
    def lazy_total_price(self):
        return lambda self=self: self.item.unit_price * self.item.quantity
```

7.11 ?

7.11.1 ?

?

```
>>> db.define_table('person',
...                 Field('name'))
<Table person (id, name)>
>>> db.person.insert(name='Alex')
1
>>> db.person.insert(name='Bob')
2
>>> db.person.insert(name='Carl')
3
>>> db.define_table('thing',
...                 Field('name'),
...                 Field('owner_id', 'reference person'))
<Table thing (id, name, owner_id)>
```

?`Field('owner_id', 'reference person')` ?`Field('owner_id', db.person)`?

?

?

```
>>> db.thing.insert(name='Boat', owner_id=1)
1
>>> db.thing.insert(name='Chair', owner_id=1)
2
>>> db.thing.insert(name='Shoes', owner_id=2)
3
```

?

```
>>> for row in db(db.thing.owner_id == 1).select():
...     print(row.name)
```

```
...
Boat
Chair
```

?

```
>>> for person in db().select(db.person.ALL):
...     print(person.name)
...     for thing in person.thing.select():
...         print('    ', thing.name)
...
Alex
     Boat
     Chair
Bob
     Shoes
Carl
```

7.11.2 ?

?

```
>>> rows = db(db.person.id == db.thing.owner_id).select()
>>> for row in rows:
...     print(row.person.name, 'has', row.thing.name)
...
Alex has Boat
Alex has Chair
Bob has Shoes
```

?

```
row.name
```

?

```
row.person.name
```

?

```
row.thing.name
```

?

```
>>> rows = db(db.person).select(join=db.thing.on(db.person.id ==
db.thing.owner_id))
>>> for row in rows:
...     print(row.person.name, 'has', row.thing.name)
...
Alex has Boat
Alex has Chair
Bob has Shoes
```

?

```
db.define_table('thing',
                Field('name'),
                Field('owner_id1', 'reference person'),
                Field('owner_id2', 'reference person'))

rows = db(db.person).select(
        join=[db.person.with_alias('owner_id1').on(db.person.id ==
db.thing.owner_id1),
              db.person.with_alias('owner_id2').on(db.person.id ==
```

```
db.thing.owner_id2)])
```

join ?db.table.on(...) ?

7.11.3 ?

?

```
>>> rows = db().select(db.person.ALL, db.thing.ALL,
...                    left=db.thing.on(db.person.id == db.thing.owner_id))
>>> for row in rows:
...     print(row.person.name, 'has', row.thing.name)
...
Alex has Boat
Alex has Chair
Bob has Shoes
Carl has None
```

?

```
left = db.thing.on(...)
```

?db.thing.on ?

?db.mytable.on(...) ?left ?

7.11.4 ?

?

```
>>> count = db.person.id.count()
>>> for row in db(db.person.id == db.thing.owner_id
...               ).select(db.person.name, count, groupby=db.person.name):
...     print(row.person.name, row[count])
...
Alex 2
Bob 1
```

?count ?

Field ?count ?distinct ?

7.11.5 ?

?

?

```
>>> db.define_table('person',
...                 Field('name'))
<Table person (id, name)>
>>> db.person.bulk_insert([dict(name='Alex'), dict(name='Bob'),
dict(name='Carl')])
[1, 2, 3]
>>> db.define_table('thing',
...                 Field('name'))
<Table thing (id, name)>
>>> db.thing.bulk_insert([dict(name='Boat'), dict(name='Chair'),
dict(name='Shoes')])
[1, 2, 3]
>>> db.define_table('ownership',
...                 Field('person', 'reference person'),
...                 Field('thing', 'reference thing'))
<Table ownership (id, person, thing)>
```

?

```
>>> db.ownership.insert(person=1, thing=1)  # Alex owns Boat
1
>>> db.ownership.insert(person=1, thing=2)  # Alex owns Chair
2
>>> db.ownership.insert(person=2, thing=3)  # Bob owns Shoes
3
```

?

```
>>> db.ownership.insert(person=3, thing=1)  # Curt owns Boat too
4
```

?

```
>>> persons_and_things = db((db.person.id == db.ownership.person) &
...                         (db.thing.id == db.ownership.thing))
```

?

```
>>> for row in persons_and_things.select():
...     print(row.person.name, 'has', row.thing.name)
...
Alex has Boat
Alex has Chair
Bob has Shoes
Curt has Boat
```

?

```
>>> for row in persons_and_things(db.person.name == 'Alex').select():
...     print(row.thing.name)
...
Boat
Chair
```

?

```
>>> for row in persons_and_things(db.thing.name == 'Boat').select():
...     print(row.person.name)
...
Alex
Curt
```

?Section 13.2 ?

7.11.6 ?

?

```
db.define_table('person',
                Field('name'),
                Field('father_id', 'reference person'),
                Field('mother_id', 'reference person'))
```

?

```
db.define_table('person',
                Field('name'),
                Field('father_id', db.person),   # wrong!
                Field('mother_id', db['person']))   # wrong!
```

?`db.tablename` ?`'reference tablename'` ?

?

?with_alias ?Field ?Expression ?

?

```
>>> fid, mid = db.person.bulk_insert([dict(name='Massimo'), dict(name='Claudia')])
>>> db.person.insert(name='Marco', father_id=fid, mother_id=mid)
3
>>> Father = db.person.with_alias('father')
>>> Mother = db.person.with_alias('mother')
>>> type(Father)
<class 'pydal.objects.Table'>
>>> str(Father)
'person AS father'
>>> rows = db().select(db.person.name, Father.name, Mother.name,
...                    left=(Father.on(Father.id == db.person.father_id),
...                          Mother.on(Mother.id == db.person.mother_id)))
>>> for row in rows:
...     print(row.person.name, row.father.name, row.mother.name)
...
Massimo None None
Claudia None None
Marco Massimo Claudia
```

?

?

```
>>> db.define_table('person',
...                 Field('name'),
...                 Field('father', 'reference person'),
...                 Field('mother', 'reference person'))
<Table person (id, name, father, mother)>
>>> fid, mid = db.person.bulk_insert([dict(name='Massimo'), dict(name='Claudia')])
>>> db.person.insert(name='Marco', father=fid, mother=mid)
3
>>> father = db.person.with_alias('father')
>>> mother = db.person.with_alias('mother')
>>> rows = db().select(db.person.name, father.name, mother.name,
...                    left=(father.on(father.id==db.person.father),
...                          mother.on(mother.id==db.person.mother)))
>>> for row in rows:
...     print(row.person.name, row.father.name, row.mother.name)
...
Massimo None None
Claudia None None
Marco Massimo Claudia
```

?

7.12 ?

py4web ?

```
>>> db.define_table('log', Field('event'),
...                        Field('event_time', 'datetime'),
...                        Field('severity', 'integer'))
<Table log (id, event, event_time, severity)>
```

?

```
>>> import datetime
```

```
>>> now = datetime.datetime.now()
>>> db.log.insert(event='port scan', event_time=now, severity=1)
1
>>> db.log.insert(event='xss injection', event_time=now, severity=2)
2
>>> db.log.insert(event='unauthorized login', event_time=now, severity=3)
3
```

7.12.1 like, ilike, regexp, startswith, endswith, contains, upper, lower

?like ?

```
>>> for row in db(db.log.event.like('port%')).select():
...     print(row.event)
...
port scan
```

?

like ?like ?

```
db.mytable.myfield.like('value', case_sensitive=False)
```

?ilike ?

```
db.mytable.myfield.ilike('value')
```

py4web ?

```
db.mytable.myfield.startswith('value')
db.mytable.myfield.endswith('value')
db.mytable.myfield.contains('value')
```

?

```
db.mytable.myfield.like('value%')
db.mytable.myfield.like('%value')
db.mytable.myfield.like('%value%')
```

?contains ?list:<type> ?Section 7.14.1 ?

?contains ?all ?

```
db.mytable.myfield.contains(['value1', 'value2'], all=True)
```

?

```
db.mytable.myfield.contains(['value1', 'value2'], all=False)
```

?regexp ?like ?

upper ?lower ?

```
>>> for row in db(db.log.event.upper().like('PORT%')).select():
...     print(row.event)
...
port scan
```

7.12.2 year, month, day, hour, minutes, seconds

date ?day ?month ?year ?hour ?minutes ?seconds ?

```
>>> for row in db(db.log.event_time.year() > 2018).select():
...     print(row.event)
...
```

```
port scan
xss injection
unauthorized login
```

7.12.3 belongs

belongs ?

```
>>> for row in db(db.log.severity.belongs((1, 2))).select():
...     print(row.event)
...
port scan
xss injection
```

DAL ?_select ?select ?

```
>>> bad_days = db(db.log.severity == 3)._select(db.log.event_time)
>>> for row in db(db.log.event_time.belongs(bad_days)).select():
...     print(row.severity, row.event)
...
1 port scan
2 xss injection
3 unauthorized login
```

?

```
db.define_table('person', Field('name'))
db.define_table('thing',
                Field('name'),
                Field('owner_id', 'reference person'))

db(db.thing.owner_id.belongs(db.person.name == 'Jonathan')).select()
```

?db.thing.owner_id ?_select ?

?

```
lazy = db(db.person.name == 'Jonathan').nested_select(db.person.id)

db(db.thing.id == 1).update(owner_id = lazy)
```

?lazy ?id ?

7.12.4 sum, avg, min, max and len

?count ?sum ?

```
>>> sum = db.log.severity.sum()
>>> print(db().select(sum).first()[sum])
6
```

?avg ?min ?max ?

```
>>> max = db.log.severity.max()
>>> print(db().select(max).first()[max])
3
```

len ?

```
>>> for row in db(db.log.event.len() > 13).select():
...     print(row.event)
...
unauthorized login
```

?

```
>>> exp = (db.log.event.len() + 1).sum()
>>> db().select(exp).first()[exp]
43
```

7.12.5 ?

?

```
db(db.thing).select(distinct = db.thing.name[:3])
```

7.12.6 coalesce ?coalesce_zero

?COALESCE ?coalesce ?

```
>>> db.define_table('sysuser', Field('username'), Field('fullname'))
<Table sysuser (id, username, fullname)>
>>> db.sysuser.insert(username='max', fullname='Max Power')
1
>>> db.sysuser.insert(username='tim', fullname=None)
2
>>> coa = db.sysuser.fullname.coalesce(db.sysuser.username)
>>> for row in db().select(coa):
...     print(row[coa])
...
Max Power
tim
```

?coalesce_zero ?

```
>>> db.define_table('sysuser', Field('username'), Field('points'))
<Table sysuser (id, username, points)>
>>> db.sysuser.insert(username='max', points=10)
1
>>> db.sysuser.insert(username='tim', points=None)
2
>>> exp = db.sysuser.points.coalesce_zero().sum()
>>> db().select(exp).first()[exp]
10
>>> type(exp)
<class 'pydal.objects.Expression'>
>>> print(exp)
SUM(COALESCE("sysuser"."points",'0'))
```

7.13 ?

7.13.1 CSV?

?

```
>>> rows = db(db.person.id == db.thing.owner_id).select()
>>> print(rows)
person.id,person.name,thing.id,thing.name,thing.owner_id
1,Alex,1,Boat,1
1,Alex,2,Chair,1
2,Bob,3,Shoes,2
```

?

```
with open('test.csv', 'wb') as dumpfile:
    dumpfile.write(str(db(db.person).select()))
```

?Rows ?

```
with open('test.csv', 'w', encoding='utf-8', newline='') as dumpfile:
    dumpfile.write(str(db(db.person).select()))
```

?

```
rows = db(db.person).select()
with open('test.csv', 'wb') as dumpfile:
    rows.export_to_csv_file(dumpfile)
```

?

```
with open('test.csv', 'rb') as dumpfile:
    db.person.import_from_csv_file(dumpfile)
```

?

```
rows = db(db.person).select()
with open('test.csv', 'w', encoding='utf-8', newline='') as dumpfile:
    rows.export_to_csv_file(dumpfile)
```

?

```
with open('test.csv', 'r', encoding='utf-8', newline='') as dumpfile:
    db.person.import_from_csv_file(dumpfile)
```

?

7.13.2 CSV?

?

?

```
with open('somefile.csv', 'w', encoding='utf-8', newline='') as dumpfile:
    db.export_to_csv_file(dumpfile)
```

?

```
with open('somefile.csv', 'r', encoding='utf-8', newline='') as dumpfile:
    db.import_from_csv_file(dumpfile)
```

?

?

```
TABLE tablename
field1,field2,field3,...
```

?\r\n\r\n ?

```
END
```

?

?

?uuid ?uuid "?

7.13.3 CSV ?

?

```
db.define_table('person',
                Field('name'))

db.define_table('thing',
                Field('name'),
                Field('owner_id', 'reference person'))

# usage example
if db(db.person).isempty():
    nid = db.person.insert(name='Massimo')
    db.thing.insert(name='Chair', owner_id=nid)
```

?

?

?

```
import datetime
import uuid

now = datetime.datetime.utcnow

db.define_table('person',
                Field('uuid', length=64),
                Field('modified_on', 'datetime', default=now, update=now),
                Field('name'))

db.define_table('thing',
                Field('uuid', length=64),
                Field('modified_on', 'datetime', default=now, update=now),
                Field('name'),
                Field('owner_id', length=64))

db.person.uuid.default = db.thing.uuid.default = lambda:str(uuid.uuid4())

db.thing.owner_id.requires = IS_IN_DB(db, 'person.uuid', '%(name)s')

# usage example
if db(db.person).isempty():
    nid = str(uuid.uuid4())
    db.person.insert(uuid=nid, name='Massimo')
    db.thing.insert(name='Chair', owner_id=nid)
```

?uuid?

?

```
from py4web import response

def export():
    s = StringIO.StringIO()
    db.export_to_csv_file(s)
    response.set_header('Content-Type', 'text/csv')
    return s.getvalue()
```

?

```
from yatl.helpers import FORM, INPUT

def import_and_sync():
    form = FORM(INPUT(_type='file', _name='data'), INPUT(_type='submit'))
    if form.process().accepted:
        db.import_from_csv_file(form.vars.data.file, unique=False)
```

```
        # for every table
        for tablename in db.tables:
            table = db[tablename]
            # for every uuid, delete all but the latest
            items = db(table).select(table.id, table.uuid,
                                     orderby=~table.modified_on,
                                     groupby=table.uuid)
            for item in items:
                db((table.uuid == item.uuid) & (table.id != item.id)).delete()
    return dict(form=form)
```

?

?

?

7.13.4 HTML ?

Rows ?`xml` ?

```
>>> rows = db(db.person.id == db.thing.owner_id).select()
>>> print(rows.xml())
```

```
<table>
<thead>
<tr><th>person.id</th><th>person.name</th>
    <th>thing.id</th><th>thing.name</th>
    <th>thing.owner_id</th>
</tr>
</thead>
<tbody>
<tr class="w2p_odd odd">
    <td>1</td><td>Alex</td>
    <td>1</td><td>Boat</td>
    <td>1</td>
</tr>
<tr class="w2p_even even">
    <td>1</td><td>Alex</td>
    <td>2</td><td>Chair</td>
    <td>1</td>
</tr>
<tr class="w2p_odd odd">
    <td>2</td><td>Bob</td>
    <td>3</td>
    <td>Shoes</td>
    <td>2</td>
</tr>
</tbody>
</table>
```

?`TAG` XML helper ?`*<iterable>` ?

```
>>> rows = db(db.person).select()
>>> print(TAG.result(*[TAG.row(*[TAG.field(r[f], _name=f) for f in
db.person.fields]) for r in rows]))
```

```
<result>
<row><field name="id">1</field><field name="name">Alex</field></row>
<row><field name="id">2</field><field name="name">Bob</field></row>
<row><field name="id">3</field><field name="name">Carl</field></row>
</result>
```

Warning ?*TAG* XML helper ?Section 10.3.1 ?`Tags` ?Section 13.2 ?

7.13.5 Data representation

`Rows.export_to_csv_file` ?represent ?`True` ?represent ?

?`colnames` ?

`export_to_csv_file` ?`import_from_csv_file` ?`delimiter` ?`quotechar` ?`quoting` ?`csv.QUOTE_MINIMAL`?

?

```
import csv
rows = db(query).select()
with open('/tmp/test.txt', 'wb') as oufile:
    rows.export_to_csv_file(oufile,
                            delimiter='|',
                            quotechar='"',
                            quoting=csv.QUOTE_NONNUMERIC)
```

?

```
"hello"|35|"this is the text description"|"2013-03-03"
```

?

7.14 ?

7.14.1 `list:<type>` ?`contains`

py4web ?

```
list:string
list:integer
list:reference <table>
```

?

?`list:string` ?`StringListProperty` ?`ListProperty(int)` ?`|` ?`[1, 2, 3]` ?`|1|2|3|` ?

?`|` ?`||` ?

?`list:string` ?

```
>>> db.define_table('product',
...                 Field('name'),
...                 Field('colors', 'list:string'))
<Table product (id, name, colors)>
>>> db.product.colors.requires = IS_IN_SET(('red', 'blue', 'green'))
>>> db.product.insert(name='Toy Car', colors=['red', 'green'])
1
>>> products = db(db.product.colors.contains('red')).select()
>>> for item in products:
...     print(item.name, item.colors)
...
Toy Car ['red', 'green']
```

`list:integer` ?

?`insert` ?

?`list:<type>` ?`contains(value)` ?value ?`contains` ?string ?text ?`LIKE`

'%value%' ?

list:reference ?contains(value) ?

```
>>> db.define_table('tag',
...                 Field('name'),
...                 format='%(name)s')
<Table tag (id, name)>
>>> db.define_table('product',
...                 Field('name'),
...                 Field('tags', 'list:reference tag'))
<Table product (id, name, tags)>
>>> a = db.tag.insert(name='red')
>>> b = db.tag.insert(name='green')
>>> c = db.tag.insert(name='blue')
>>> db.product.insert(name='Toy Car', tags=[a, b, c])
1
>>> products = db(db.product.tags.contains(b)).select()
>>> for item in products:
...     print(item.name, item.tags)
...
Toy Car [1, 2, 3]
>>> for item in products:
...     print(item.name, db.product.tags.represent(item.tags))
...
Toy Car red, green, blue
```

?list:reference tag ?

```
requires = IS_IN_DB(db, db.tag._id, db.tag._format, multiple=True)
```

?SELECT/OPTION ?

?represent ?forms ?

?list:reference ?list:integer ?list:string ?IS_IN_SET ?IS_IN_DB ?

7.14.2 “?

?define_table ?

```
>>> db.define_table('person', Field('name'), Field('gender'))
<Table person (id, name, gender)>
>>> db.define_table('doctor', db.person, Field('specialization'))
<Table doctor (id, name, gender, specialization)>
```

?

```
now = datetime.datetime.utcnow

signature = db.Table(db, 'signature',
                     Field('is_active', 'boolean', default=True),
                     Field('created_on', 'datetime', default=now),
                     Field('created_by', db.auth_user, default=auth.user_id),
                     Field('modified_on', 'datetime', update=now),
                     Field('modified_by', db.auth_user, update=auth.user_id))

db.define_table('payment', Field('amount', 'double'), signature)
```

?

?Auth ?

```
auth = Auth(db)
db.define_table('payment', Field('amount', 'double'), auth.signature)
```

?

7.14.3 filter_in ?filter_out

?

?

```
>>> import json
>>> db.define_table('anyobj',
...                 Field('name'),
...                 Field('data', 'text'))
<Table anyobj (id, name, data)>
>>> db.anyobj.data.filter_in = lambda obj: json.dumps(obj)
>>> db.anyobj.data.filter_out = lambda txt: json.loads(txt)
>>> myobj = ['hello', 'world', 1, {2: 3}]
>>> aid = db.anyobj.insert(name='myobjname', data=myobj)
>>> row = db.anyobj[aid]
>>> row.data
['hello', 'world', 1, {'2': 3}]
```

?SQLCustomType ?Section 7.14.7 ?

7.14.4 ?

PY4WEB ?

?

```
db.mytable._before_insert
db.mytable._after_insert
db.mytable._before_update
db.mytable._after_update
db.mytable._before_delete
db.mytable._after_delete
```

?

?

```
>>> db.define_table('person', Field('name'))
<Table person (id, name)>
>>> def pprint(callback, *args):
...     print("%s%s" % (callback, args))
...
>>> db.person._before_insert.append(lambda f: pprint('before_insert', f))
>>> db.person._after_insert.append(lambda f, i: pprint('after_insert', f, i))
>>> db.person.insert(name='John')
before_insert(<OpRow {'name': 'John'}>,)
after_insert(<OpRow {'name': 'John'}>, 1)
1
>>> db.person._before_update.append(lambda s, f: pprint('before_update', s, f))
>>> db.person._after_update.append(lambda s, f: pprint('after_update', s, f))
>>> db(db.person.id == 1).update(name='Tim')
before_update(<Set ("person"."id" = 1)>, <OpRow {'name': 'Tim'}>)
after_update(<Set ("person"."id" = 1)>, <OpRow {'name': 'Tim'}>)
1
>>> db.person._before_delete.append(lambda s: pprint('before_delete', s))
>>> db.person._after_delete.append(lambda s: pprint('after_delete', s))
>>> db(db.person.id == 1).delete()
before_delete(<Set ("person"."id" = 1)>,)
after_delete(<Set ("person"."id" = 1)>,)
1
```

?f ?OpRow ?i ?s ?OpRow ?f.name ?f['name'] ?

?None ?False ?_before_* ?True ?

?

?Set ?update_naive ?

?

?

7.14.5 ?

?

```
auth.enable_record_versioning(db)
```

?Auth ?

?

```
db.define_table('stored_item',
                Field('name'),
                Field('quantity', 'integer'),
                Field('is_active', 'boolean',
                      writable=False, readable=False, default=True))
```

?is_active ?

?

?

```
db.stored_item._enable_record_versioning()
```

?

```
db.stored_item._enable_record_versioning(archive_db=db,
                                         archive_name='stored_item_archive',
                                         current_record='current_record',
                                         is_active='is_active')
```

archive_db=db ?stored_item ?archive_name ?stored_item ?stored_item ?current_record ?

?stored_item_archive ?is_active ?common_filter ?stored_item ?is_active ?_enable_record_versioning ?is_active ?common_filter ?

7.14.6 ?

?

```
db.define_table('blog_post',
                Field('subject'),
                Field('post_text', 'text'),
                Field('is_public', 'boolean'),
                common_filter = lambda query: db.blog_post.is_public == True)
```

?

```
db.blog_post._common_filter = lambda query: ...
```

?

?

```
db.blog_post._common_filter = None
```

?

```
db(query, ignore_common_filters=True)
```

?

7.14.7 ?Field ?

?filter_in ?filter_out ?

```
>>> def ip2int(sv):
...     "Convert an IPV4 to an integer."
...     sp = sv.split('.'); assert len(sp) == 4 # IPV4 only
...     iip = 0
...     for i in map(int, sp): iip = (iip<<8) + i
...     return iip
...
>>> def int2ip(iv):
...     "Convert an integer to an IPV4."
...     assert iv > 0
...     iv = (iv,); ov = []
...     for i in range(3):
...         iv = divmod(iv[0], 256)
...         ov.insert(0, iv[1])
...     ov.insert(0, iv[0])
...     return '.'.join(map(str, ov))
...
>>> from pydal import SQLCustomType
>>> ipv4 = SQLCustomType(type='string', native='integer',
...                      encoder=lambda x : str(ip2int(x)), decoder=int2ip)
>>> db.define_table('website',
...                 Field('name'),
...                 Field('ipaddr', type=ipv4))
<Table website (id, name, ipaddr)>
>>> db.website.insert(name='wikipedia', ipaddr='91.198.174.192')
1
>>> db.website.insert(name='google', ipaddr='172.217.11.174')
2
>>> db.website.insert(name='youtube', ipaddr='74.125.65.91')
3
>>> db.website.insert(name='github', ipaddr='207.97.227.239')
4
>>> rows = db(db.website.ipaddr > '100.0.0.0').select(orderby=~db.website.ipaddr)
>>> for row in rows:
...     print(row.name, row.ipaddr)
...
github 207.97.227.239
google 172.217.11.174
```

SQLCustomType ?type ?native ?encoder ?decoder ?

?

?

7.14.8 ?

DAL ?

```
from pydal import DAL, Field
db = DAL('sqlite://storage.sqlite', folder='path/to/app/databases')
```

?

?db.define_table ?

?

```
from py4web import DAL, Field
db = DAL('sqlite://storage.sqlite', folder='path/to/app/databases',
auto_import=True)
```

?

7.14.9 ?

?

?

```
db_a = DAL('postgres://...')
db_b = DAL('postgres://...')
```

?

```
DAL.distributed_transaction_commit(db_a, db_b)
```

?Exception?

?

7.14.10 ?

?

```
db = DAL('sqlite://storage.sqlite')
```

?

```
db = DAL('postgres://username:password@localhost/mydb')
```

?

7.15 ?

7.15.1 ?

?

?

```
ConnectionPool
BaseAdapter extends ConnectionPool
Row
DAL
Reference
Table
Expression
Field
Query
Set
Rows
```

?BaseAdapter ?Table ?Set ?

?

```
db.mytable.insert(myfield='myvalue')
```

?

```
Table.insert(myfield='myvalue')
```

?

```
db._adapter.insert(db.mytable, db.mytable._listify(dict(myfield='myvalue')))
```

?db.mytable._listify ?(field, value) ?adapter ?insert ?db._adapter ?

```
query = db._adapter._insert(db.mytable, list_of_fields)
db._adapter.execute(query)
```

?

BaseAdapter ?

?

```
SQLiteAdapter extends BaseAdapter
JDBCSQLiteAdapter extends SQLiteAdapter
MySQLAdapter extends BaseAdapter
PostgreSQLAdapter extends BaseAdapter
JDBCPostgreSQLAdapter extends PostgreSQLAdapter
OracleAdapter extends BaseAdapter
MSSQLAdapter extends BaseAdapter
MSSQL2Adapter extends MSSQLAdapter
MSSQL3Adapter extends MSSQLAdapter
MSSQL4Adapter extends MSSQLAdapter
FireBirdAdapter extends BaseAdapter
FireBirdEmbeddedAdapter extends FireBirdAdapter
InformixAdapter extends BaseAdapter
DB2Adapter extends BaseAdapter
IngresAdapter extends BaseAdapter
IngresUnicodeAdapter extends IngresAdapter
GoogleSQLAdapter extends MySQLAdapter
NoSQLAdapter extends BaseAdapter
GoogleDatastoreAdapter extends NoSQLAdapter
CubridAdapter extends MySQLAdapter (experimental)
TeradataAdapter extends DB2Adapter (experimental)
SAPDBAdapter extends BaseAdapter (experimental)
CouchDBAdapter extends NoSQLAdapter (experimental)
IMAPAdapter extends NoSQLAdapter (experimental)
MongoDBAdapter extends NoSQLAdapter (experimental)
VerticaAdapter extends MSSQLAdapter (experimental)
SybaseAdapter extends MSSQLAdapter (experimental)
```

?BaseAdapter ?

?

```
class MySQLAdapter(BaseAdapter):

    # specify a driver to use
    driver = globals().get('pymysql', None)

    # map py4web types into database types
    types = {
        'boolean': 'CHAR(1)',
        'string': 'VARCHAR(%(length)s)',
        'text': 'LONGTEXT',
        ...
        }

    # connect to the database using driver
    def __init__(self, db, uri, pool_size=0, folder=None, db_codec ='UTF-8',
                 credential_decoder=lambda x:x, driver_args={},
```

```
                adapter_args={}):
        # parse uri string and store parameters in driver_args
        ...
        # define a connection function
        def connect(driver_args=driver_args):
            return self.driver.connect(**driver_args)
        # place it in the pool
        self.pool_connection(connect)
        # set optional parameters (after connection)
        self.execute('SET FOREIGN_KEY_CHECKS=1;')
        self.execute("SET sql_mode='NO_BACKSLASH_ESCAPES';")

    # override BaseAdapter methods as needed
    def lastrowid(self, table):
        self.execute('select last_insert_id();')
        return int(self.cursor.fetchone()[0])
```

?

?db ?

```
db = DAL('mysql://...')
```

DAL() ?

couchdb	pydal.adapters.couchdb.CouchDB
cubrid	pydal.adapters.mysql.Cubrid
db2:ibm_db_dbi	pydal.adapters.db2.DB2IBM
db2:pyodbc	pydal.adapters.db2.DB2Pyodbc
firebird	pydal.adapters.firebird.FireBird
firebird_embedded	pydal.adapters.firebird.FireBirdEmbedded
google:MySQLdb	pydal.adapters.google.GoogleMySQL
google:datastore	pydal.adapters.google.GoogleDatastore
google:datastore+ndb	pydal.adapters.google.GoogleDatastore
google:psycopg2	pydal.adapters.google.GooglePostgres
google:sql	pydal.adapters.google.GoogleSQL
informix	pydal.adapters.informix.Informix
informix-se	pydal.adapters.informix.InformixSE
ingres	pydal.adapters.ingres.Ingres
ingresu	pydal.adapters.ingres.IngresUnicode
jdbc:postgres	pydal.adapters.postgres.JDBCPostgre
jdbc:sqlite	pydal.adapters.sqlite.JDBCSQLite
jdbc:sqlite:memory	pydal.adapters.sqlite.JDBCSQLite
mongodb	pydal.adapters.mongo.Mongo
mssql	pydal.adapters.mssql.MSSQL1
mssql2	pydal.adapters.mssql.MSSQL1N
mssql3	pydal.adapters.mssql.MSSQL3
mssql3n	pydal.adapters.mssql.MSSQL3N
mssql4	pydal.adapters.mssql.MSSQL4
mssql4n	pydal.adapters.mssql.MSSQL4N
mssqln	pydal.adapters.mssql.MSSQL1N
mysql	pydal.adapters.mysql.MySQL
oracle	pydal.adapters.oracle.Oracle
postgres	pydal.adapters.postgres.Postgre
postgres2	pydal.adapters.postgres.PostgreNew
postgres2:psycopg2	pydal.adapters.postgres.PostgrePsycoNew

postgres3	pydal.adapters.postgres.PostgreBoolean
postgres3:psycopg2	pydal.adapters.postgres.PostgrePsycoBoolean
postgres:psycopg2	pydal.adapters.postgres.PostgrePsyco
pytds	pydal.adapters.mssql.PyTDS
sapdb	pydal.adapters.sap.SAPDB
spatialite	pydal.adapters.sqlite.Spatialite
spatialite:memory	pydal.adapters.sqlite.Spatialite
sqlite	pydal.adapters.sqlite.SQLite
sqlite:memory	pydal.adapters.sqlite.SQLite
sybase	pydal.adapters.mssql.Sybase
teradata	pydal.adapters.teradata.Teradata
vertica	pydal.adapters.mssql.Vertica

?

?

```
import MySQLdb as mysqldb
from pydal.adapters.mysql import SQLAdapter
SQLAdapter.driver = mysqldb
```

?`mysqldb` ?`.connect()` ?

```
db = DAL(..., driver_args={}, adapter_args={})
```

?`adapter_args` ?

```
from pydal.adapters.mysql import MySQL
assert "mysqldv" in MySQL.drivers
db = DAL(..., driver_args={}, adapter_args={"driver": "mysqldb"})
```

7.15.2 SQLite

SQLite ?`fake_migrate=True` ?

SQLite ?

7.15.3 MySQL

MySQL ?`fake_migrate=True` ?`fake_migrate=False` ?

7.15.4 Google SQL

Google SQL ?`py4web_filesystem` ?

7.15.5 MSSQL (Microsoft SQL Server)

?`limitby=(a, b)` ?`OFFSET ... ROWS ... FETCH NEXT ... ROWS ONLY` ?`mssql://` ?`text` ?`varchar(max)` ?`text` ?`mssql3://` ?`mssql4://` ?

MSSQL ?

```
db = DAL('mssql://....')
for key in db._adapter.types:
    if ' ON DELETE %(on_delete_action)s' in db._adapter.types[key]:
        db._adapter.types[key] =
db._adapter.types[key].replace('%(on_delete_action)s', 'NO ACTION')
```

MSSQL ?

```
db(query).select(distinct=True)
```

?

```
db(query).select(distinct=db.mytable.myfield)
```

7.15.6 Oracle

Oracle ?`db(...).select(limitby=(a, b))` ?

7.15.7 Google NoSQL (Datastore)

Google NoSQL (Datastore) ?

?`run_in_transaction` ?

?`ListStringProperty`?`ListProperty` ?

Chapter 8

RestAPI

?[CIT0801] ?[CIT0802] ?

?

?

Note ?

?

```
# in superheroes/__init__.py
import os
from py4web import action, request, Field, DAL
from pydal.restapi import RestAPI, Policy

# database definition
DB_FOLDER = os.path.join(os.path.dirname(__file__), 'databases')
if not os.path.isdir(DB_FOLDER):
    os.mkdir(DB_FOLDER)
db = DAL('sqlite://storage.sqlite', folder=DB_FOLDER)
db.define_table(
    'person',
    Field('name'),
    Field('job'))
db.define_table(
    'superhero',
    Field('name'),
    Field('real_identity', 'reference person'))
db.define_table(
    'superpower',
    Field('description'))
db.define_table(
    'tag',
    Field('superhero', 'reference superhero'),
    Field('superpower', 'reference superpower'),
    Field('strength', 'integer'))

# add example entries in db
if not db(db.person).count():
    db.person.insert(name='Clark Kent', job='Journalist')
    db.person.insert(name='Peter Park', job='Photographer')
    db.person.insert(name='Bruce Wayne', job='CEO')
    db.superhero.insert(name='Superman', real_identity=1)
    db.superhero.insert(name='Spiderman', real_identity=2)
    db.superhero.insert(name='Batman', real_identity=3)
```

[CIT0801] https://en.wikipedia.org/wiki/Representational_state_transfer
[CIT0802] https://graphql.org/

```
    db.superpower.insert(description='Flight')
    db.superpower.insert(description='Strength')
    db.superpower.insert(description='Speed')
    db.superpower.insert(description='Durability')
    db.tag.insert(superhero=1, superpower=1, strength=100)
    db.tag.insert(superhero=1, superpower=2, strength=100)
    db.tag.insert(superhero=1, superpower=3, strength=100)
    db.tag.insert(superhero=1, superpower=4, strength=100)
    db.tag.insert(superhero=2, superpower=2, strength=50)
    db.tag.insert(superhero=2, superpower=3, strength=75)
    db.tag.insert(superhero=2, superpower=4, strength=10)
    db.tag.insert(superhero=3, superpower=2, strength=80)
    db.tag.insert(superhero=3, superpower=3, strength=20)
    db.tag.insert(superhero=3, superpower=4, strength=70)
    db.commit()

# policy definitions
policy = Policy()
policy.set('superhero', 'GET', authorize=True, allowed_patterns=['*'])
policy.set('*', 'GET', authorize=True, allowed_patterns=['*'])

# for security reasons we disabled here all methods but GET at the policy level,
# to enable any of them just set authorize = True
policy.set('*', 'PUT', authorize=False)
policy.set('*', 'POST', authorize=False)
policy.set('*', 'DELETE', authorize=False)

@action('api/<tablename>/', method = ['GET', 'POST'])
@action('api/<tablename>/<rec_id>', method = ['GET', 'PUT', 'DELETE'])
@action.uses(db)
def api(tablename, rec_id=None):
    return RestAPI(db, policy)(request.method,
                               tablename,
                               rec_id,
                               request.GET,
                               request.POST
                               )

@action("index")
def index():
    return "RestAPI example"
```

8.1 RestAPI ?

?`authorize` ?

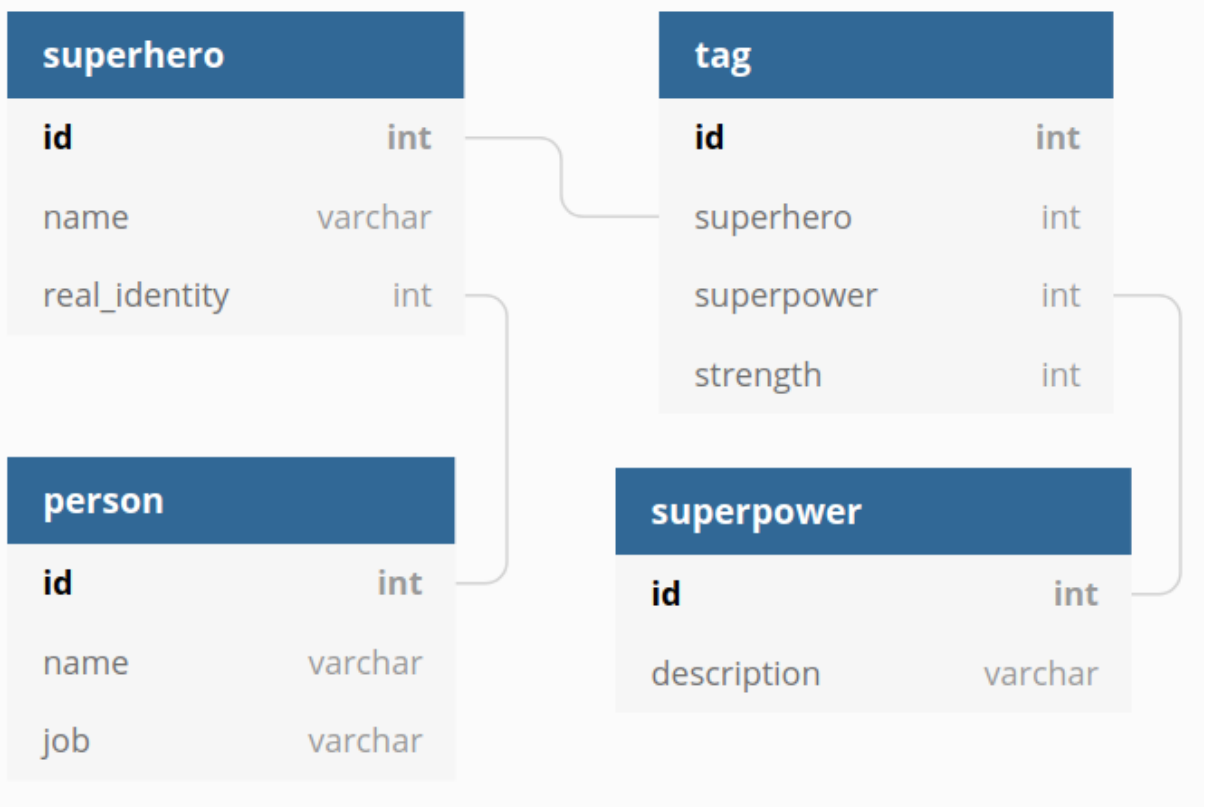
?

```
/superheroes/api/{tablename}
/superheroes/api/{tablename}/{rec_id}
```

?`person` ?


127.0.0.1:8000/superheroes/api/person
{
"api_version": "0.1",
"code": 200,
"count": 3,
"items": [
{
"id": 1,
"job": "Journalist",
"name": "Clark Kent"
},
{
"id": 2,
"job": "Photographer",
"name": "Peter Park"
},
{
"id": 3,
"job": "CEO",
"name": "Bruce Wayne"
}
],
"status": "success",
"timestamp": "2021-09-23T21:08:04.359685"
}

superhero ?


superhero
id int
name varchar
real_identity int
tag
id int
superhero int
superpower int
strength int
person
id int
name varchar
job varchar
superpower
id int
description varchar

Note ?request.POST ?regular HTML-form ?JavaScript FormData ?

8.2 RestAPI GET

?`{something}.eq=value` ?`eq=` ?`gt=` ?`not.`

`{something}` ?

- ?

  ?

  ```
  /superheroes/api/superhero?name.eq=Superman
  ```

- ?

  ?

  ```
  /superheroes/api/superhero?real_identity.name.eq=Clark Kent
  ```

- ?

  ?

  ```
  /superheroes/api/superhero?superhero.tag.strength.gt=90
  ```

  ?`tag` ?`superhero` ?`strength` ?

- ?

  ?

  ```
  /superheroes/api/superhero?superhero.tag.superpower.description.eq=Flight
  ```

Hint ?

?tag ?superpower ?description ?tag ?superhero ?superhero ?

?

```
@offset=10
@limit=10
@order=name
@model=true
@lookup=real_identity
```

?`@model` ?`@lookup` ?

8.3 RestAPI ?

?

URL:

```
/superheroes/api/superhero
```

?

```
{
    "count": 3,
```

```
    "status": "success",
    "code": 200,
    "items": [
        {
            "real_identity": 1,
            "name": "Superman",
            "id": 1
        },
        {
            "real_identity": 2,
            "name": "Spiderman",
            "id": 2
        },
        {
            "real_identity": 3,
            "name": "Batman",
            "id": 3
        }
    ],
    "timestamp": "2019-05-19T05:38:00.132635",
    "api_version": "0.1"
}
```


URL:

```
/superheroes/api/superhero?@model=true
```

?


```
{
    "count": 3,
    "status": "success",
    "code": 200,
    "items": [
        {
            "real_identity": 1,
            "name": "Superman",
            "id": 1
        },
        {
            "real_identity": 2,
            "name": "Spiderman",
            "id": 2
        },
        {
            "real_identity": 3,
            "name": "Batman",
            "id": 3
        }
    ],
    "timestamp": "2021-01-04T07:03:38.466030",
    "model": [
        {
            "regex": "[1-9]\\d*",
            "name": "id",
            "default": null,
            "required": false,
            "label": "Id",
            "post_writable": true,
            "referenced_by": [
                "tag.superhero"
            ],
            "unique": false,
```

```
            "type": "id",
            "options": null,
            "put_writable": true
        },
        {
            "regex": null,
            "name": "name",
            "default": null,
            "required": false,
            "label": "Name",
            "post_writable": true,
            "unique": false,
            "type": "string",
            "options": null,
            "put_writable": true
        },
        {
            "regex": null,
            "name": "real_identity",
            "default": null,
            "required": false,
            "label": "Real Identity",
            "post_writable": true,
            "references": "person",
            "unique": false,
            "type": "reference",
            "options": null,
            "put_writable": true
        }
    ],
    "api_version": "0.1"
}
```


URL:

```
/superheroes/api/superhero?@lookup=real_identity
```

?


```
{
    "count": 3,
    "status": "success",
    "code": 200,
    "items": [
        {
            "real_identity": {
                "name": "Clark Kent",
                "job": "Journalist",
                "id": 1
            },
            "name": "Superman",
            "id": 1
        },
        {
            "real_identity": {
                "name": "Peter Park",
                "job": "Photographer",
                "id": 2
            },
            "name": "Spiderman",
            "id": 2
        },
        {
```

```
            "real_identity": {
                "name": "Bruce Wayne",
                "job": "CEO",
                "id": 3
            },
            "name": "Batman",
            "id": 3
        }
    ],
    "timestamp": "2019-05-19T05:38:00.178974",
    "api_version": "0.1"
}
```


URL:

```
/superheroes/api/superhero?@lookup=identity:real_identity
```

?

?


```
{
    "count": 3,
    "status": "success",
    "code": 200,
    "items": [
        {
            "real_identity": 1,
            "name": "Superman",
            "id": 1,
            "identity": {
                "name": "Clark Kent",
                "job": "Journalist",
                "id": 1
            }
        },
        {
            "real_identity": 2,
            "name": "Spiderman",
            "id": 2,
            "identity": {
                "name": "Peter Park",
                "job": "Photographer",
                "id": 2
            }
        },
        {
            "real_identity": 3,
            "name": "Batman",
            "id": 3,
            "identity": {
                "name": "Bruce Wayne",
                "job": "CEO",
                "id": 3
            }
        }
    ],
    "timestamp": "2019-05-19T05:38:00.123218",
    "api_version": "0.1"
}
```


URL:

```
/superheroes/api/superhero?@lookup=identity!:real_identity[name,job]
```

?

?


```
{
    "count": 3,
    "status": "success",
    "code": 200,
    "items": [
        {
            "name": "Superman",
            "identity.job": "Journalist",
            "identity.name": "Clark Kent",
            "id": 1
        },
        {
            "name": "Spiderman",
            "identity.job": "Photographer",
            "identity.name": "Peter Park",
            "id": 2
        },
        {
            "name": "Batman",
            "identity.job": "CEO",
            "identity.name": "Bruce Wayne",
            "id": 3
        }
    ],
    "timestamp": "2021-01-04T07:03:38.559918",
    "api_version": "0.1"
}
```


URL:

```
/superheroes/api/superhero?@lookup=superhero.tag
```

?


```
{
    "count": 3,
    "status": "success",
    "code": 200,
    "items": [
        {
            "real_identity": 1,
            "name": "Superman",
            "superhero.tag": [
                {
                    "strength": 100,
                    "superhero": 1,
                    "id": 1,
                    "superpower": 1
                },
                {
                    "strength": 100,
                    "superhero": 1,
                    "id": 2,
                    "superpower": 2
                },
                {
                    "strength": 100,
                    "superhero": 1,
                    "id": 3,
                    "superpower": 3
```

```
                },
                {
                    "strength": 100,
                    "superhero": 1,
                    "id": 4,
                    "superpower": 4
                }
            ],
            "id": 1
        },
        {
            "real_identity": 2,
            "name": "Spiderman",
            "superhero.tag": [
                {
                    "strength": 50,
                    "superhero": 2,
                    "id": 5,
                    "superpower": 2
                },
                {
                    "strength": 75,
                    "superhero": 2,
                    "id": 6,
                    "superpower": 3
                },
                {
                    "strength": 10,
                    "superhero": 2,
                    "id": 7,
                    "superpower": 4
                }
            ],
            "id": 2
        },
        {
            "real_identity": 3,
            "name": "Batman",
            "superhero.tag": [
                {
                    "strength": 80,
                    "superhero": 3,
                    "id": 8,
                    "superpower": 2
                },
                {
                    "strength": 20,
                    "superhero": 3,
                    "id": 9,
                    "superpower": 3
                },
                {
                    "strength": 70,
                    "superhero": 3,
                    "id": 10,
                    "superpower": 4
                }
            ],
            "id": 3
        }
    ],
    "timestamp": "2019-05-19T05:38:00.201988",
    "api_version": "0.1"
```

```
}
```

URL:

```
/superheroes/api/superhero?@lookup=superhero.tag.superpower
```

?


```
{
    "count": 3,
    "status": "success",
    "code": 200,
    "items": [
        {
            "real_identity": 1,
            "name": "Superman",
            "superhero.tag.superpower": [
                {
                    "strength": 100,
                    "superhero": 1,
                    "id": 1,
                    "superpower": {
                        "id": 1,
                        "description": "Flight"
                    }
                },
                {
                    "strength": 100,
                    "superhero": 1,
                    "id": 2,
                    "superpower": {
                        "id": 2,
                        "description": "Strength"
                    }
                },
                {
                    "strength": 100,
                    "superhero": 1,
                    "id": 3,
                    "superpower": {
                        "id": 3,
                        "description": "Speed"
                    }
                },
                {
                    "strength": 100,
                    "superhero": 1,
                    "id": 4,
                    "superpower": {
                        "id": 4,
                        "description": "Durability"
                    }
                }
            ],
            "id": 1
        },
        {
            "real_identity": 2,
            "name": "Spiderman",
            "superhero.tag.superpower": [
                {
                    "strength": 50,
                    "superhero": 2,
```

```
                    "id": 5,
                    "superpower": {
                        "id": 2,
                        "description": "Strength"
                    }
                },
                {
                    "strength": 75,
                    "superhero": 2,
                    "id": 6,
                    "superpower": {
                        "id": 3,
                        "description": "Speed"
                    }
                },
                {
                    "strength": 10,
                    "superhero": 2,
                    "id": 7,
                    "superpower": {
                        "id": 4,
                        "description": "Durability"
                    }
                }
            ],
            "id": 2
        },
        {
            "real_identity": 3,
            "name": "Batman",
            "superhero.tag.superpower": [
                {
                    "strength": 80,
                    "superhero": 3,
                    "id": 8,
                    "superpower": {
                        "id": 2,
                        "description": "Strength"
                    }
                },
                {
                    "strength": 20,
                    "superhero": 3,
                    "id": 9,
                    "superpower": {
                        "id": 3,
                        "description": "Speed"
                    }
                },
                {
                    "strength": 70,
                    "superhero": 3,
                    "id": 10,
                    "superpower": {
                        "id": 4,
                        "description": "Durability"
                    }
                }
            ],
            "id": 3
        }
    ],
    "timestamp": "2019-05-19T05:38:00.322494",
```

```
    "api_version": "0.1"
}
```

URL ?

```
/superheroes/api/superhero?
@lookup=powers:superhero.tag[strength].superpower[description]
```

?


```
{
    "count": 3,
    "status": "success",
    "code": 200,
    "items": [
        {
            "real_identity": 1,
            "name": "Superman",
            "powers": [
                {
                    "strength": 100,
                    "superpower": {
                        "description": "Flight"
                    }
                },
                {
                    "strength": 100,
                    "superpower": {
                        "description": "Strength"
                    }
                },
                {
                    "strength": 100,
                    "superpower": {
                        "description": "Speed"
                    }
                },
                {
                    "strength": 100,
                    "superpower": {
                        "description": "Durability"
                    }
                }
            ],
            "id": 1
        },
        {
            "real_identity": 2,
            "name": "Spiderman",
            "powers": [
                {
                    "strength": 50,
                    "superpower": {
                        "description": "Strength"
                    }
                },
                {
                    "strength": 75,
                    "superpower": {
                        "description": "Speed"
                    }
                },
                {
```

```
                    "strength": 10,
                    "superpower": {
                        "description": "Durability"
                    }
                }
            ],
            "id": 2
        },
        {
            "real_identity": 3,
            "name": "Batman",
            "powers": [
                {
                    "strength": 80,
                    "superpower": {
                        "description": "Strength"
                    }
                },
                {
                    "strength": 20,
                    "superpower": {
                        "description": "Speed"
                    }
                },
                {
                    "strength": 70,
                    "superpower": {
                        "description": "Durability"
                    }
                }
            ],
            "id": 3
        }
    ],
    "timestamp": "2019-05-19T05:38:00.309903",
    "api_version": "0.1"
}
```


URL ?

```
/superheroes/api/superhero?
@lookup=powers!:superhero.tag[strength].superpower[description]
```

?


```
{
    "count": 3,
    "status": "success",
    "code": 200,
    "items": [
        {
            "real_identity": 1,
            "name": "Superman",
            "powers": [
                {
                    "strength": 100,
                    "description": "Flight"
                },
                {
                    "strength": 100,
                    "description": "Strength"
                },
                {
```

```
                    "strength": 100,
                    "description": "Speed"
                },
                {
                    "strength": 100,
                    "description": "Durability"
                }
            ],
            "id": 1
        },
        {
            "real_identity": 2,
            "name": "Spiderman",
            "powers": [
                {
                    "strength": 50,
                    "description": "Strength"
                },
                {
                    "strength": 75,
                    "description": "Speed"
                },
                {
                    "strength": 10,
                    "description": "Durability"
                }
            ],
            "id": 2
        },
        {
            "real_identity": 3,
            "name": "Batman",
            "powers": [
                {
                    "strength": 80,
                    "description": "Strength"
                },
                {
                    "strength": 20,
                    "description": "Speed"
                },
                {
                    "strength": 70,
                    "description": "Durability"
                }
            ],
            "id": 3
        }
    ],
    "timestamp": "2019-05-19T05:38:00.355181",
    "api_version": "0.1"
}
```


URL ?

```
/superheroes/api/superhero?
@lookup=powers!:superhero.tag[strength].superpower[description],
identity!:real_identity[name]
```

?


```
{
    "count": 3,
    "status": "success",
```

```
    "code": 200,
    "items": [
        {
            "name": "Superman",
            "identity.name": "Clark Kent",
            "powers": [
                {
                    "strength": 100,
                    "description": "Flight"
                },
                {
                    "strength": 100,
                    "description": "Strength"
                },
                {
                    "strength": 100,
                    "description": "Speed"
                },
                {
                    "strength": 100,
                    "description": "Durability"
                }
            ],
            "id": 1
        },
        {
            "name": "Spiderman",
            "identity.name": "Peter Park",
            "powers": [
                {
                    "strength": 50,
                    "description": "Strength"
                },
                {
                    "strength": 75,
                    "description": "Speed"
                },
                {
                    "strength": 10,
                    "description": "Durability"
                }
            ],
            "id": 2
        },
        {
            "name": "Batman",
            "identity.name": "Bruce Wayne",
            "powers": [
                {
                    "strength": 80,
                    "description": "Strength"
                },
                {
                    "strength": 20,
                    "description": "Speed"
                },
                {
                    "strength": 70,
                    "description": "Durability"
                }
            ],
            "id": 3
        }
```

```
    ],
    "timestamp": "2021-01-04T07:31:34.974953",
    "api_version": "0.1"
}
```


URL:

```
/superheroes/api/superhero?name.eq=Superman
```

?


```
{
    "count": 1,
    "status": "success",
    "code": 200,
    "items": [
        {
            "real_identity": 1,
            "name": "Superman",
            "id": 1
        }
    ],
    "timestamp": "2019-05-19T05:38:00.405515",
    "api_version": "0.1"
}
```


URL:

```
/superheroes/api/superhero?real_identity.name.eq=Clark Kent
```

?


```
{
    "count": 1,
    "status": "success",
    "code": 200,
    "items": [
        {
            "real_identity": 1,
            "name": "Superman",
            "id": 1
        }
    ],
    "timestamp": "2019-05-19T05:38:00.366288",
    "api_version": "0.1"
}
```


URL:

```
/superheroes/api/superhero?not.real_identity.name.eq=Clark Kent
```

?


```
{
    "count": 2,
    "status": "success",
    "code": 200,
    "items": [
        {
            "real_identity": 2,
            "name": "Spiderman",
            "id": 2
        },
        {
```

```
            "real_identity": 3,
            "name": "Batman",
            "id": 3
        }
    ],
    "timestamp": "2019-05-19T05:38:00.451907",
    "api_version": "0.1"
}
```

URL:

```
/superheroes/api/superhero?superhero.tag.superpower.description=Flight
```

?

```
{
    "count": 1,
    "status": "success",
    "code": 200,
    "items": [
        {
            "real_identity": 1,
            "name": "Superman",
            "id": 1
        }
    ],
    "timestamp": "2019-05-19T05:38:00.453020",
    "api_version": "0.1"
}
```

8.4 RestAPI ?

?

api_version RestAPI ?

timestamp ISO 8601 ?

status RestAPI ?

code HTTP ?

?

count ?

items ?

errors ?

models ?

message ?

Chapter 9

YATL ?

py4web ?

- yatl (Yet Another Template Language) ?
- Renoir ?

?`Renoir` ?`Renoir` ?`yatl` ?

py4web ?`[[ ... ]]` ?

?

Warning ?

```
[[items = ['a', 'b', 'c']]] # this gives "Internal Server Error"
[[items = ['a', 'b', 'c'] ]] # this works
```

?

?`[[ ... ]]` ?`pass` ?

? ?`pass` ?`pass` ?

?

```
[[
if i == 0:
response.write('i is 0')
else:
response.write('i is not 0')
pass
]]
```

?`pass` ?`pass` ?

py4web ?

```
<html><body>
[[for x in range(10):]][[=x]] hello <br />[[pass]]
</body></html>
```

py4web ?

```
response.write("""<html><body>""", escape=False)
for x in range(10):
    response.write(x)
    response.write(""" hello <br />""", escape=False)
response.write("""</body></html>""", escape=False)
```

`response.write` ?

?

?

```
[[=x]]
```

?

```
response.write(x)
```

?x ?XML ?True ?Section 10.2.1?

?H1 helper ?

```
[[=H1(i)]]
```

?

```
response.write(H1(i))
```

?H1 ?H1 ?

?response.write(obj, escape=True) ?True ?obj ?.xml() ?escape ?__str__ ?escape ?True?H1 ?.xml() ?

?

Note ?bottle.response ?yatl.template.DummyResponse ?response.write ?

9.1 ?

py4web ?Section 5.5 ?new_app/template/index.html ?

9.1.1 for...in

?

```
[[items = ['a', 'b', 'c'] ]]
<ul>
[[for item in items:]]<li>[[=item]]</li>[[pass]]
</ul>
```

?

```
<ul>
<li>a</li>
<li>b</li>
<li>c</li>
</ul>
```

?items ?

9.1.2 while

?

```
[[k = 3]]
<ul>
[[while k > 0:]]<li>[[=k]][[k = k - 1]]</li>[[pass]]
</ul>
```

?

```
<ul>
<li>3</li>
<li>2</li>
<li>1</li>
</ul>
```

9.1.3 if...elif...else

?

```
[[
import random
k = random.randint(0, 100)
]]
<h2>
[[=k]]
[[if k % 2:]]is odd[[else:]]is even[[pass]]
</h2>
```

?

```
<h2>
45 is odd
</h2>
```

?else ?if ?pass ?pass ?else ?

?elif ?

```
[[
import random
k = random.randint(0, 100)
]]
<h2>
[[=k]]
[[if k % 4 == 0:]]is divisible by 4
[[elif k % 2 == 0:]]is even
[[else:]]is odd
[[pass]]
</h2>
```

?

```
<h2>
64 is divisible by 4
</h2>
```

9.1.4 try...except...else...finally

?try...except ?

```
[[try:]]
Hello [[= 1 / 0]]
[[except:]]
division by zero
[[else:]]
no division by zero
[[finally:]]
<br />
[[pass]]
```

?

```
Hello division by zero
<br />
```

?

9.1.5 def...return

py4web ?

```
[[def itemize1(link): return LI(A(link, _href="http://" + link))]]
<ul>
[[=itemize1('www.google.com')]]
</ul>
```

?

```
<ul>
<li><a href="http://www.google.com">www.google.com</a></li>
</ul>
```

itemize1 ?

?

```
[[def itemize2(link):]]
<li><a href="http://[[=link]]">[[=link]]</a></li>
[[return]]
<ul>
[[itemize2('www.google.com')]]
</ul>
```

?itemize2 ?itemize2 ?

?return ?

9.2 ?

?extend ?include ?block ?super ?

?

9.2.1 extend ?include

?

?

?**layout template** ?

?

```
<!--minimalist_page.html-->
[[extend 'layout.html']]
<h1>Hello World</h1>
[[include 'page.html']]
```

?[[include]] ?

```
<!--layout.html-->
<html>
  <head>
    <title>Page Title</title>
  </head>
```

```
  <body>
    [[include]]
  </body>
</html>
```

?[[include]] ?[[include]] ?[[extend]] ?extend ?include ?

?

[[extend ...]] ?? ?

```
<!--index.html-->
[[sidebar_enabled=True]]
[[extend 'layout.html']]
<h1>Home Page</h1>
```

?

```
<!--layout.html-->
[[include]]
[[if sidebar_enabled:]]
    <div id="sidebar">
        Sidebar Content
    </div>
[[pass]]
```

?sidebar_enabled ?extend ?sidebar_enabled ?

?

9.2.2 ?

extend ?include ?extend ?include ?extend ?include ?

?include ?[[include ...]] ?if...else ?

```
[[if some_condition:]]
   [[include 'this_template.html']]
[[else:]]
   [[include 'that_template.html']]
[[pass]]
```

?some_condition ?

?include —— ?if...else ?[[extend ...]] ?

?

9.2.3 ?

?

```
<!--layout.html-->
<html>
  <body>
    [[include]]
    <div class="sidebar">
      [[if 'mysidebar' in globals():]][[mysidebar()]][[else:]]
        my default sidebar
      [[pass]]
    </div>
  </body>
</html>
```

?

```
[[def mysidebar():]]
   my new sidebar!!!
[[return]]
[[extend 'layout.html']]
   Hello World!!!
```

?[[extend...]] ?[[include]] ?= ?

?

```
<html>
  <body>
    Hello World!!!
    <div class="sidebar">
      my new sidebar!!!
    </div>
  </body>
</html>
```

?response.write ?[[mysidebar()]] ?[[=mysidebar()]] ?

9.2.4 block ?super

?[[block ...]] ?

?default/index.html ?[[extend 'layout.html']] ?

?**?** ?

?

```
<html>
  <body>
    [[include]]
    <div class="sidebar">
      [[block mysidebar]]
        my default sidebar (this content to be replaced)
      [[end]]
    </div>
  </body>
</html>
```

?default/index.html ?

```
[[extend 'layout.html']]
Hello World!!!
[[block mysidebar]]
my new sidebar!!!
[[end]]
```

?

```
<html>
  <body>
    Hello World!!!
    <div class="sidebar">
        my new sidebar!!!
    </div>
  </body>
</html>
```

?

?[[extend ...]] ?extend ?

?[[super]] ?

```
[[extend 'layout.html']]
Hello World!!!
[[block mysidebar]]
[[super]]
my new sidebar!!!
[[end]]
```

?

```
<html>
  <body>
    Hello World!!!
    <div class="sidebar">
        my default sidebar
        my new sidebar!
    </div>
  </body>
</html>
```

9.3 ?

9.3.1 ?

?_scaffold ?

```
 <!DOCTYPE html>
 <html>
   <head>
     <base href="[[=URL('static')]]/">
     <meta name="viewport" content="width=device-width, initial-scale=1">
     <link rel="shortcut icon"
href="
     <link rel="stylesheet" href="css/no.css">
     <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.14.0/css/all.min.css"
integrity="sha512-1PKOgIY59xJ8Co8+NE6FZ+LOAZKjy+KY8iq0G4B3CyeY6wYHN3yt9PW0XpSriVlkMXe40PTKnXrLnZ
crossorigin="anonymous" />
     <style>.py4web-validation-error{margin-top:-16px;
font-size:0.8em;color:red}</style>
     [[block page_head]]<!-- individual pages can customize header here -->[[end]]
   </head>
   <body>
     <header>
       <!-- Navigation bar -->
       <nav class="black">
         <!-- Logo -->
         <a href="[[=URL('index')]]">
           <b>py4web
<script>document.write(window.location.href.split('/')[3]);</script></b>
         </a>
         <!-- Do not touch this -->
         <label for="hamburger">?</label>
         <input type="checkbox" id="hamburger">
         <!-- Left menu ul/li -->
         [[block page_left_menu]][[end]]
         <!-- Right menu ul/li -->
         <ul>
           [[if globals().get('user'):]]
           <li>
```

```
            <a class="navbar-link is-primary">
              [[=globals().get('user',{}).get('email')]]
            </a>
            <ul>
              <li><a href="[[=URL('auth/profile')]]">Edit Profile</a></li>
              <li><a href="[[=URL('auth/change_password')]]">Change
Password</a></li>
              <li><a href="[[=URL('auth/logout')]]">Logout</a></li>
            </ul>
          </li>
          [[else:]]
          <li>
            Login
            <ul>
              <li><a href="[[=URL('auth/register')]]">Sign up</a></li>
              <li><a href="[[=URL('auth/login')]]">Log in</a></li>
            </ul>
          </li>
          [[pass]]
        </ul>
      </nav>
    </header>
    <!-- beginning of HTML inserted by extending template -->
    <center>
      <div>
        <!-- Flash alert messages, first optional one in data-alert -->
        <flash-alerts class="padded"
data-alert="[[=globals().get('flash','')]]"></flash-alerts>
      </div>
      <main class="padded">
        <!-- contect injected by extending page -->
        [[include]]
      </main>
    </center>
    <!-- end of HTML inserted by extending template -->
    <footer class="black padded">
      <p>
        Made with py4web
      </p>
    </footer>
  </body>
  <!-- You've gotta have utils.js -->
  <script src="js/utils.js"></script>
  [[block page_scripts]]<!-- individual pages can add scripts here -->[[end]]
 </html>
```

?

- ?
- ?no.css ??
- ?[[include]] ?
- ?
- ?
- ?

?

9.3.2 ?

?

YATL ?

10.1 ?

?

```
[[=DIV('this', 'is', 'a', 'test', _id='123', _class='myclass')]]
```

?

```
<div id="123" class="myclass">thisisatest</div>
```

?Section 5.5 ?new_app/template/index.html ?

DIV ?**helper class** ? ?

?

- **?**?? ?? ?thisisatest
- **?**?_class ?_id
- **?** ?

helper ?* ?** ?

```
[[
contents = ['this', 'is', 'a', 'test']
attributes = {'_id':'123', '_class':'myclass'}
=DIV(*contents, **attributes)
]]
```

?

?

A, BEAUTIFY, BODY, CAT, CODE, DIV, EM, FORM, H1, H2, H3, H4, H5, H6, HEAD, HTML, IMG, INPUT, LABEL, LI, METATAG, OL, OPTION, P, PRE, SELECT, SPAN, STRONG, TABLE, TAG, TAGGER, THEAD, TBODY, TD, TEXTAREA, TH, TT, TR, UL, XML, xmlescape, I, META, LINK, TITLE, STYLE, SCRIPT

?

```
[[=DIV(STRONG(I("hello ", "<world>")), _class="myclass")]]
```

?

```
<div class="myclass"><strong><i>hello &lt;world&gt;</i></strong></div>
```

?__str__ ?xml ?Section 3.6.6 ?

```
>>> from yatl.helpers import *
>>>
>>> str(DIV("hello world"))
'<div>hello world</div>'
>>> DIV("hello world").xml()
'<div>hello world</div>'
```

py4web ?

helper ?

```
>>> a = DIV(SPAN('a', 'b'), 'c')
>>> print(a)
<div><span>ab</span>c</div>
>>> del a[1]
>>> a.append(STRONG('x'))
>>> a[0][0] = 'y'
>>> print(a)
<div><span>yb</span><strong>x</strong></div>
```

helpers ?

```
>>> a = DIV(SPAN('a', 'b'), 'c')
>>> a['_class'] = 's'
>>> a[0]['_class'] = 't'
>>> print(a)
<div class="s"><span class="t">ab</span>c</div>
```

?`a.children` ?`a.attributes` ?`i` ?`a[i]` ?`a.children[i]` ?`s` ?`a[s]` ?`a.attributes[s]` ?

?

```
DIV('text', _data-role='collapsible')
```

?

?`**` ?

```
>>> print(DIV('text', **{'_data-role': 'collapsible'}))
<div data-role="collapsible">text</div>
```

?

```
>>> print(TAG['soap:Body']('whatever', **{'_xmlns:m':'http://www.example.org'}))
<soap:Body xmlns:m="http://www.example.org">whatever</soap:Body>
```

10.2 ?

10.2.1 XML

XML ?**?** ?

?

```
>>> print(DIV("<strong>hello</strong>"))
<div>&lt;strong&gt;hello&lt;/strong&gt;</div>
```

?XML ?

```
>>> print(DIV(XML("<strong>hello</strong>")))
<div><strong>hello</strong></div>
```

?

```
>>> print(XML('<script>alert("unsafe!")</script>'))
<script>alert("unsafe!")</script>
```

?XML helper ?

```
>>> print(XML('<script>alert("unsafe!")</script>', sanitize=True))
&lt;script&gt;alert(&quot;unsafe!&quot;)&lt;/script&gt;
```

?XML ?permitted_tags ?allowed_attributes ?XML helper ?

```
XML(text, sanitize=False,
    permitted_tags=['a', 'b', 'blockquote', 'br/', 'i', 'li',
        'ol', 'ul', 'p', 'cite', 'code', 'pre', 'img/',
        'h1', 'h2', 'h3', 'h4', 'h5', 'h6', 'table', 'tr', 'td',
        'div', 'strong', 'span'],
    allowed_attributes={'a': ['href', 'title', 'target'],
        'img': ['src', 'alt'], 'blockquote': ['type'], 'td': ['colspan']})
```

10.2.2 A

?

```
>>> print(A('<click>', XML('<strong>me</strong>'),
            _href='http://www.py4web.com'))
<a href="http://www.py4web.com">&lt;click&gt;<strong>me</strong></a>
```

10.2.3 BODY

?

```
>>> print(BODY('<hello>', XML('<strong>world</strong>'), _bgcolor='red'))
<body bgcolor="red">&lt;hello&gt;<strong>world</strong></body>
```

10.2.4 CAT

?

```
>>> print(CAT('Here is a ', A('link', _href='target'), ', and here is some ',
STRONG('bold text'), '.'))
Here is a <a href="target">link</a>, and here is some <strong>bold text</strong>.
```

10.2.5 DIV

?

```
>>> print(DIV('<hello>', XML('<strong>world</strong>'), _class='test', _id=0))
<div id="0" class="test">&lt;hello&gt;<strong>world</strong></div>
```

10.2.6 EM

?

```
>>> print(EM('<hello>', XML('<strong>world</strong>'), _class='test', _id=0))
<em id="0" class="test">&lt;hello&gt;<strong>world</strong></em>
```

10.2.7 FORM

?Chapter 12 ?

```
>>> print(FORM(INPUT(_type='submit'), _action='', _method='post'))
<form action="" method="post"><input type="submit"/></form>
```

10.2.8 H1, H2, H3, H4, H5, H6

?

```
>>> print(H1('<hello>', XML('<strong>world</strong>'), _class='test', _id=0))
<h1 id="0" class="test">&lt;hello&gt;<strong>world</strong></h1>
```

10.2.9 HEAD

?

```
>>> print(HEAD(TITLE('<hello>', XML('<strong>world</strong>'))))
<head><title>&lt;hello&gt;<strong>world</strong></title></head>
```

10.2.10 HTML

?

```
>>> print(HTML(BODY('<hello>', XML('<strong>world</strong>'))))
<html><body>&lt;hello&gt;<strong>world</strong></body></html>
```

10.2.11 I

?

```
>>> print(I('<hello>', XML('<strong>world</strong>'), _class='test', _id=0))
<i id="0" class="test">&lt;hello&gt;<strong>world</strong></i>
```

10.2.12 IMG

?

```
>>> print(IMG(_src='http://example.com/image.png', _alt='test'))
<img alt="test" src="http://example.com/image.png"/>
```

?

```
>>> print(A(IMG(_src=URL('static', 'logo.png'), _alt="My Logo"),
... _href=URL('default', 'index')))
<a href="/default/index"><img alt="My Logo" src="/static/logo.png"/></a>
```

10.2.13 INPUT

?<input.../> ?/> ?> ?_type ?

```
>>> print(INPUT(_name='test', _value='a'))
<input name="test" value="a"/>
```

?_checked ?

```
>>> for v in ['a', 'b', 'c']:
...     print(INPUT(_type='radio', _name='test', _value=v, _checked=v=='b'), v)
...
<input name="test" type="radio" value="a"/> a
<input checked="checked" name="test" type="radio" value="b"/> b
<input name="test" type="radio" value="c"/> c
```

?

```
>>> print(INPUT(_type='checkbox', _name='test', _value='a', _checked=True))
<input checked="checked" name="test" type="checkbox" value="a"/>
>>> print(INPUT(_type='checkbox', _name='test', _value='a', _checked=False))
<input name="test" type="checkbox" value="a"/>
```

10.2.14 LABEL

?

```
>>> print(LABEL('<hello>', XML('<strong>world</strong>'), _class='test', _id=0))
<label id="0" class="test">&lt;hello&gt;<strong>world</strong></label>
```

10.2.15 LI

?UL ?OL ?

```
>>> print(LI('<hello>', XML('<strong>world</strong>'), _class='test', _id=0))
<li id="0" class="test">&lt;hello&gt;<strong>world</strong></li>
```

10.2.16 OL

?

```
>>> print(OL(LI('<hello>'), LI(XML('<strong>world</strong>')), _class='test',
_id=0))
<ol class="test" id="0"><li>&lt;hello&gt;</li><li><strong>world</strong></li></ol>
```

10.2.17 OPTION

?SELECT ?

```
>>> print(OPTION('<hello>', XML('<strong>world</strong>'), _value='a'))
<option value="a">&lt;hello&gt;<strong>world</strong></option>
```

?_selected ?

```
>>> print(OPTION('Thank You', _value='ok', _selected=True))
<option selected="selected" value="ok">Thank You</option>
```

10.2.18 P

?

```
>>> print(P('<hello>', XML('<strong>world</strong>'), _class='test', _id=0))
<p id="0" class="test">&lt;hello&gt;<strong>world</strong></p>
```

10.2.19 PRE

?<pre>...</pre> ?CODE helper ?

```
>>> print(SELECT(OPTION('first', _value='1'), OPTION('second', _value='2'),
_class='test', _id=0))
<pre id="0" class="test">&lt;hello&gt;<strong>world</strong></pre>
```

10.2.20 SCRIPT

?

```
>>> print(SCRIPT('console.log("hello world");', _type='text/javascript'))
<script type="text/javascript">console.log("hello world");</script>
```

10.2.21 SELECT

?<select>...</select> ?OPTION helper ?

```
>>> print(SELECT(OPTION('first', _value='1'), OPTION('second', _value='2'),
... _class='test', _id=0))
<select class="test" id="0"><option value="1">first</option><option
value="2">second</option></select>
```

10.2.22 SPAN

?DIV ?? ?

```
>>> print(SPAN('<hello>', XML('<strong>world</strong>'), _class='test', _id=0))
<span id="0" class="test">&lt;hello&gt;<strong>world</strong></span>
```

10.2.23 STYLE

?SCRIPT ?

```
>>> print(STYLE(XML('body {color: white}')))
<style>body {color: white}</style>
```

?

```
>>> print(STYLE(_src='style.css'))
<style src="style.css"></style>
```

10.2.24 TABLE, TR, TD

?THEAD ?TBODY helpers?

```
>>> print(TABLE(TR(TD('a'), TD('b')), TR(TD('c'), TD('d'))))
<table><tr><td>a</td><td>b</td></tr><tr><td>c</td><td>d</td></tr></table>
```

TR ?TD ?

?* ?

?

```
>>> table = [['a', 'b'], ['c', 'd']]
>>> print(TABLE(TR(*map(TD, table[0])), TR(*map(TD, table[1]))))
<table><tr><td>a</td><td>b</td></tr><tr><td>c</td><td>d</td></tr></table>
```

?

```
>>> table = [['a', 'b'], ['c', 'd']]
>>> print(TABLE(*[TR(*map(TD, rows)) for rows in table]))
<table><tr><td>a</td><td>b</td></tr><tr><td>c</td><td>d</td></tr></table>
```

10.2.25 TBODY

?

```
>>> print(TBODY(TR(TD('<hello>')), _class='test', _id=0))
<tbody id="0" class="test"><tr><td>&lt;hello&gt;</td></tr></tbody>
```

10.2.26 TEXTAREA

?<textarea>...</textarea>?

```
>>> print(TEXTAREA('<hello>', XML('<strong>world</strong>'), _class='test',
... _cols="40", _rows="10"))
<textarea class="test" cols="40"
rows="10">&lt;hello&gt;<strong>world</strong></textarea>
```

10.2.27 TH

?TD?

```
>>> print(TH('<hello>', XML('<strong>world</strong>'), _class='test', _id=0))
<th id="0" class="test">&lt;hello&gt;<strong>world</strong></th>
```

10.2.28 THEAD

?

```
>>> print(THEAD(TR(TH('<hello>')), _class='test', _id=0))
<thead id="0" class="test"><tr><th>&lt;hello&gt;</th></tr></thead>
```

10.2.29 TITLE

?

```
>>> print(TITLE('<hello>', XML('<strong>world</strong>')))
<title>&lt;hello&gt;<strong>world</strong></title>
```

10.2.30 TT

?

```
>>> print(TT('<hello>', XML('<strong>world</strong>'), _class='test', _id=0))
<tt id="0" class="test">&lt;hello&gt;<strong>world</strong></tt>
```

10.2.31 UL

?

```
>>> print(UL(LI('<hello>'), LI(XML('<strong>world</strong>')), _class='test',
_id=0))
<ul class="test" id="0"><li>&lt;hello&gt;</li><li><strong>world</strong></li></ul>
```

10.2.32 URL

URL helper?

10.3 ?

10.3.1 TAG

?TAG?

```
[[=TAG.name('a', 'b', _c='d')]]
```

?

```
<name c="d">ab</name>
```

?XML helper ?TAG ?str() ?

```
[[=TAG['name']('a', 'b', _c='d')]]
```

?

```
[[=TAG['link/'](_href='http://py4web.com')]]
```

?

```
<link ref="http://py4web.com"/>
```

?TAG ?TAG.name ?TAG['name'] ?

10.3.2 BEAUTIFY

BEAUTIFY ?

```
[[=BEAUTIFY({"a": ["hello", STRONG("world")], "b": (1, 2)})]]
```

BEAUTIFY ?

```
{"a": ["hello", STRONG("world")], "b": (1, 2)}
```

?

```
<table><tbody>
<tr><th>a</th><td><ul><li>hello</li><li><strong>world</strong></li></ul></td></tr>
<tr><th>b</th><td>(1, 2)</td></tr>
</tbody></table>
```

10.4 ?*DOM*

?

10.4.1 children

?children ?

```
>>> CAT('hello', STRONG('world')).children
['hello', <yatl.helpers.TAGGER object at 0x7fa533ff7640>]
```

10.4.2 find

?find ?

```
def find(self, query=None, **kargs)
```

?

?query ?

```
>>> a = DIV(DIV(SPAN('x'), 3, DIV(SPAN('y'))))
>>> for c in a.find('span', first_only=True): c[0]='z'
>>> print(a)  # We should .xml() here instead of print
<div><div><span>z</span>3<div><span>y</span></div></div></div>
>>> for c in a.find('span'): c[0]='z'
```

```
>>> print(a)
<div><div><span>z</span>3<div><span>z</span></div></div></div>
```

?jQuery ?

- jQuery ? ?
- jQuery ? ?
- jQuery ? ?
- jQuery ? ?
- jQuery ??

?

```
>>> a = DIV(SPAN(A('hello', **{'_id': '1-1', '_u:v': '$'})), P('world',
_class='this is a test'))
>>> for e in a.find('div a#1-1, p.is'): print(e)
<a id="1-1" u:v="$">hello</a>
<p class="this is a test">world</p>
>>> for e in a.find('#1-1'): print(e)
<a id="1-1" u:v="$">hello</a>
>>> a.find('a[u:v=$]')[0].xml()
'<a id="1-1" u:v="$">hello</a>'
>>> a = FORM(INPUT(_type='text'), SELECT(OPTION(0)), TEXTAREA())
>>> for c in a.find('input, select, textarea'): c['_disabled'] = True
>>> a.xml()
'<form><input disabled="disabled" type="text"/><select
disabled="disabled"><option>0</option></select><textarea
disabled="disabled"></textarea></form>'
>>> for c in a.find('input, select, textarea'): c['_disabled'] = False
>>> a.xml()
'<form><input
type="text"/><select><option>0</option></select><textarea></textarea></form>'
```

?replace ?

```
>>> a = DIV(DIV(SPAN('x', _class='abc'), DIV(SPAN('y', _class='abc'), SPAN('z',
_class='abc'))))
>>> b = a.find('span.abc', replace=P('x', _class='xyz'))
>>> print(a)
<div><div><p class="xyz">x</p><div><p class="xyz">x</p><p
class="xyz">x</p></div></div></div>
```

replace ?

```
>>> a = DIV(DIV(SPAN('x', _class='abc'), DIV(SPAN('y', _class='abc'), SPAN('z',
_class='abc'))))
>>> b = a.find('span.abc', replace=lambda el: P(el[0], _class='xyz'))
>>> print(a)
<div><div><p class="xyz">x</p><div><p class="xyz">y</p><p
class="xyz">z</p></div></div></div>
```

?replace=None ?

```
>>> a = DIV(DIV(SPAN('x', _class='abc'), DIV(SPAN('y', _class='abc'), SPAN('z',
_class='abc'))))
>>> b = a.find('span', text='y', replace=None)
>>> print(a)
<div><div><span class="abc">x</span><div><span class="abc"></span><span
class="abc">z</span></div></div></div>
```

?text ?text ?replace ?text ?find ?

?find ?text ?

```
>>> a = DIV(DIV(SPAN('x', _class='abc'), DIV(SPAN('y', _class='abc'), SPAN('z',
_class='abc'))))
>>> b = a.find(text=re.compile('x|y|z'), replace='hello')
>>> print(a)
<div><div><span class="abc">hello</span><div><span class="abc">hello</span><span
class="abc">hello</span></div></div></div>
```

?text ?

```
>>> a = DIV(DIV(SPAN('x', _class='abc'), DIV(SPAN('y', _class='efg'), SPAN('z',
_class='abc'))))
>>> b = a.find('span.efg', text=re.compile('x|y|z'), replace='hello')
>>> print(a)
<div><div><span class="abc">x</span><div><span class="efg">hello</span><span
class="abc">z</span></div></div></div>
```

10.5 ??

?

?Inject ?

?

```
from py4web.utils.factories import Inject

my_var = "Example variable to be passed to a Template"

...

@unauthenticated("index", "index.html")
@action.uses(Inject(my_var=my_var))
def index():

    ...
```

?index.html ?

```
[[=my_var]]
```

?Inject ?

```
auth.enable(uses=(session, T, db, Inject(TIMEOFFSET=settings.TIMEOFFSET)))
```

? ?sidebar_menu ?controllers.py ?

```
from py4web.utils.factories import Inject
from .libs.helpers import sidebar_menu

@action(...)
@action.uses("index.html", Inject(sidebar_menu=sidebar_menu))
def index(): ....
```

?

```
from py4web.utils.factories import Inject
from .libs import helpers

@action(...)
@action.uses(Inject(**vars(helpers)), "index.html")
```

```
def index(): ....
```

?

```
[[=sidebar_menu]]
```

?

11.1 Pluralize

Pluralize ?

Pluralize ?

```
it.json
it-IT.json
fr.json
fr-FR.json
(etc)
```

?

```
{"dog": {"0": "no cane", "1": "un cane", "2": "{n} cani", "10": "tantissimi
cani"}}
```

?

?

```
{"bed": {"0": "no postel", "1": "postel", "2": "postele", "5": "postelí"}}
```

?

```
>>> from pluralize import Translator
>>> T = Translator('translations')
>>> dog = T("dog")
>>> print(dog)
dog
>>> T.select('it')
>>> print(dog)
un cane
>>> print(dog.format(n=0))
no cane
>>> print(dog.format(n=1))
un cane
>>> print(dog.format(n=5))
5 cani
>>> print(dog.format(n=20))
tantissimi cani
```

?

T(...) ?

T.select(s) ?

11.2 ?

?

```
matches = T.find_matches('path/to/app/folder')
```

?

```
T.update_languages(matches)
```

?

```
T.languages['de'] = {}
```

?

```
known_expressions = set()
for language in T.languages.values():
    for expression in language:
        known_expressions.add(expression)
T.update_languages(known_expressions))
```

?

```
T.save('translations')
```

?

Form ?

?

?

```
@action('create_thing')
@action.uses('generic.html', db, flash)
def create_thing():
    form = Form(db.thing)
    if form.accepted:
        flash.set("record created")
        redirect(URL('other_page'))
    return locals()
```

?

```
@action('update_thing/<thing_id:int>')
@action.uses('generic.html', db, flash)
def update_thing(thing_id):
    form = Form(db.thing, thing_id)
    if form.accepted:
        flash.set("record updated")
        redirect(URL('other_page'))
    return locals()
```

?

```
@action('some_form')
@action.uses('generic.html', flash)
def some_form():
    fields = [
        Field("name", requires=IS_NOT_EMPTY()),
        Field("color", required=IS_IN_SET(["red","blue","green"])),
    ]
    form = Form(fields)
    if form.accepted:
        flash.set("information recorded")
        redirect(URL('other_page'))
    return locals()
```

?`flash` ?`flash` ?`common.py` ?

?

```
db.define_table(
    'thing',
    Field('name', requires=IS_NOT_EMPTY()),
    Field('color', requires=IS_IN_SET(['red','blue','green'])),
```

```
    Field('image', 'upload', download_url=lambda name: URL('download', name)),
)
```

12.1 Form ?

Form ?

```
Form(self,
     table,
     record=None,
     readonly=False,
     deletable=True,
     formstyle=FormStyleDefault,
     dbio=True,
     keep_values=False,
     form_name=False,
     hidden=None,
     validation=None,
     csrf_session=None,
     csrf_protection=True,
     lifespan=None,
     signing_info=None,
     ):
```

?

- table: ?
- record: ?
- readonly: ?
- deletable: ?
- **formstyle: ?**
  ?FormStyleDefault (?FormStyleBulma ?FormStyleBootstrap4 ?FormStyleBootstrap5
- dbio: ?
- keep_values: ?
- form_name: ?
- hidden: ?
- validation: ?Section 12.6.8
- csrf_session: ?
- lifespan: ?
- signing_info: ?

FormStyleDefault ?py4web.utils.form.FormStyleDefault ?formstyle ?
?color ?color ?

12.2 ?

?form_minimal ?

```
# in controllers.py
```

```
from py4web impot action, redirect, URL, Field
from py4web.utils.form import Form
from pydal.validators import *

@action('index', method=['GET', 'POST'])
@action.uses('form_minimal.html')
def index():
    fields = [
        Field('name', requires=IS_NOT_EMPTY()),
        Field('color', requires=IS_IN_SET(['red','blue','green'])),
    ]
    form = Form(fields)
    if form.accepted:
        # Do something with form.vars['name'] and form.vars['color']
        redirect(URL('accepted'))
    if form.errors:
        # do something
        ...
    return dict(form=form)

@action("accepted")
def accepted():
    return "form_example accepted"
```

?templates/form_minimal.html ?

```
[[extend 'layout.html']]
[[=form]]
```

?http://127.0.0.1:8000/form_minimal - ?



?

- Form ?py4web.utils.form ?
- ??IS_NOT_EMPTY ?Section 12.6 ?pydal.validators ?
- ?GET ?POST ?

?

12.3 ?

?form_basic ?

```
# in controllers.py
from py4web import action, redirect, URL, Field
from py4web.utils.form import Form
from pydal.validators import *
from .common import db

# controllers definition
@action("create_form", method=["GET", "POST"])
@action.uses("form_basic.html", db)
def create_form():
    form = Form(db.thing)
    rows = db(db.thing).select()
    return dict(form=form, rows=rows)
```

?requires ?Section 12.6 ?

?templates/form_basic.html ?

```
[[extend "layout.html"]]

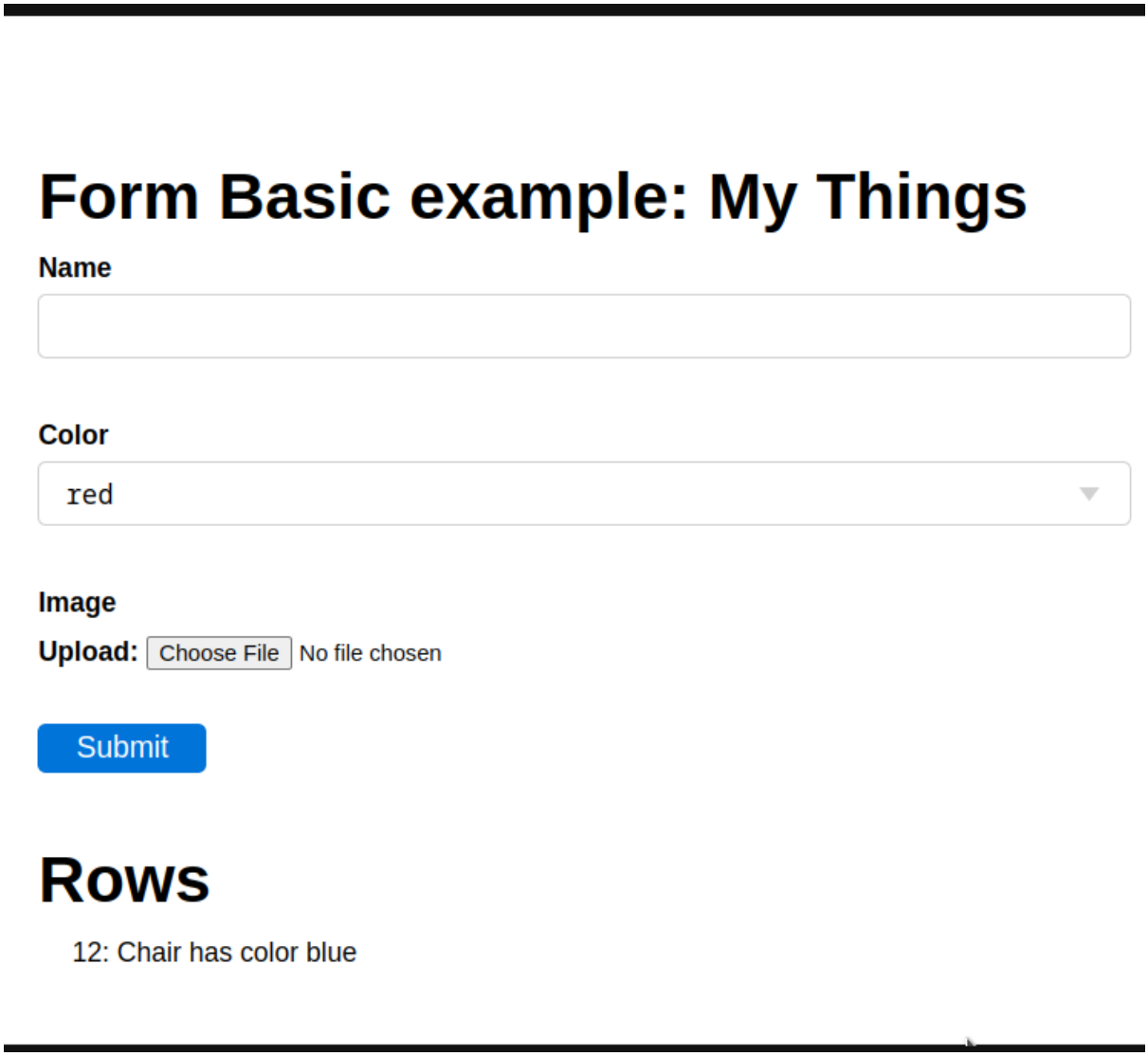
<h2 class="title">Form Basic example: My Things</h2>

[[=form]]

<h2 class="title">Rows</h2>

<ul>
[[for row in rows:]]
<li>[[=row.id]]: [[=row.name]] has color [[=row.color]]</li>
[[pass]]
</ul>
```

?http://127.0.0.1:8000/create_form ?



?

?

```
# controllers definition
@action("update_form/<thing_id:int>", method=["GET", "POST"])
@action.uses("form_basic.html", db)
def update_form(thing_id):
    form = Form(db.thing, thing_id)
    rows = db(db.thing).select()
    return dict(form=form, rows=rows)
```

12.3.1 ?

?

```
db.define_table(
    'thing',
    Field('name', requires=IS_NOT_EMPTY()),
    Field('color', requires=IS_IN_SET(['red','blue','green'])),
    Field('image', 'upload', download_url=lambda image: URL('download', image)),
)
```

?`your_app/upload` ?`download_url` ?`download` url ?`common.py` ?

?form_basic.html ?

```
<h2 class="title">Form upload example: My Things</h2>

[[=form]]

<h2 class="title">Rows</h2>

<ul>
[[for row in rows:]]
<li>[[=row.id]]: [[=row.name]] has color [[=row.color]]
    <img src="[[=URL('download', row.image)]]" />
[[pass]]
</ul>
```

?Section 7.5 ?

12.4 ?

12.4.1 ?

Py4web ?

?

- CheckboxWidget
- DateTimeWidget
- FileUploadWidget
- ListWidget
- PasswordWidget
- RadioWidget
- SelectWidget
- TextareaWidget

?

```
# in controllers.py
from py4web import action, redirect, URL, Field
from py4web.utils.form import Form, FormStyleDefault, RadioWidget
from pydal.validators import *
from .common import db

# controllers definition
@action("create_form", method=["GET", "POST"])
@action.uses("form_widgets.html", db)
def create_form():
    FormStyleDefault.widgets['color']=RadioWidget()
    form = Form(db.thing, formstyle=FormStyleDefault)
    rows = db(db.thing).select()
    return dict(form=form, rows=rows)
```

?

- ?
- ?color ?

```
FormStyleDefault.widgets['color']=RadioWidget()
```

?

?

Important ?Section 7.5 ?

12.4.2 ?

?

```
# in controllers.py
from py4web import action, redirect, URL, Field
from py4web.utils.form import Form, FormStyleDefault, RadioWidget
from pydal.validators import *
from .common import db

# custom widget class definition
class MyCustomWidget:
    def make(self, field, value, error, title, placeholder, readonly=False):
        tablename = field._table if "_table" in dir(field) else "no_table"
        control = INPUT(
            _type="text",
            _id="%s_%s" % (tablename, field.name),
            _name=field.name,
            _value=value,
            _class="input",
            _placeholder=placeholder if placeholder and placeholder != "" else
"..",
            _title=title,
            _style="font-size: x-large;color: red; background-color: black;",
        )
        return control

# controllers definition
@action("create_form", method=["GET", "POST"])
@action.uses("form_custom_widgets.html", db)
def create_form():
    MyStyle = FormStyleDefault.clone()
    MyStyle.classes = FormStyleDefault.classes
    MyStyle.widgets['name']=MyCustomWidget()
    MyStyle.widgets['color']=RadioWidget()

    form = Form(db.thing, deletable=False, formstyle=MyStyle)
    rows = db(db.thing).select()
    return dict(form=form, rows=rows)
```

?

?

12.5 ?

12.5.1 ?

?

```
db.define_table('paint', Field('color'))
```

```
form = Form(db.paint)
form.structure.find('[name=color]')[0]['_class'] = 'my-class'
```

?.find(...) ?find ?[0] ?_class ?

12.5.2 ?

?

```
[[=form.custom.begin ]]
[[=form.custom.submit ]]
[[=form.custom.end ]]
```

?id ?

```
[[extend 'layout.html']]
[[=form.custom.begin ]]
    [[for field in DETAIL_FIELDS: ]]
        [[ if field not in ['id']: ]]
            <div class="select">
                [[=form.custom.widgets[field] ]]
            </div>
        [[pass]]
    [[pass]]
[[=form.custom.submit ]]
[[=form.custom.end ]]
```

?

Warning ?

?

?

```
[[extend 'layout.html']]

[[for field, error form.errors.items:]]
<div class="error">Field [[=field]] [[=error]]</div>
[[pass]]

[[=form.custom.begin ]]

<div class="select">
     <input name="name" value="form.vars.get('name', '')"/>
</div>
<div class="select">
[[for color in ['red', 'blue', 'green']:]]
     <label>[[=color]]</label>
     <input name="color" type="radio" value="[[=color]]"
                [[if form.vars.get('color') == color:]]checked[[pass]]
     />
[[pass]]
</div>
<input type="submit" value="Click me"/>
[[=form.custom.end ]]
```

12.5.3 sidecar ?

sidecar ?

?click me ?

```
form.param.sidecar = DIV(BUTTON("click me", _onclick="alert('doh!')"))
```

?Cancel ?

```
attrs = {
"_onclick": "window.history.back(); return false;",
"_class": "button is-default",
}
form.param.sidecar.append(BUTTON("Cancel", **attrs))
```

12.6 ?

?Field ?requires ?Section 7.5 ?

?

```
db.define_table(
    'person',
    Field('name',requires=IS_NOT_EMPTY(),
    Field('job')
)
```

?

```
db.define_table(
    'person',
    Field('name'),
    Field('job')
)
db.person.name.requires = IS_NOT_EMPTY()
```

?

```
db.person.name.requires = [
    IS_NOT_EMPTY(),
    IS_NOT_IN_DB(db, 'person.name')]
```

?list: ?

- IS_IN_DB(..., multiple=True)
- IS_IN_SET(..., multiple=True)
- IS_NOT_EMPTY()
- IS_LIST_OF_EMAILS()
- IS_LIST_OF(...)

?multiple=(1, 1000) ?

?error_message ?

```
IS_NOT_EMPTY(error_message='cannot be empty!')
```

?error_message ?IS_NOT_EMPTY('cannot be empty!')

?

IS_NOT_EMPTY(error_message=T('cannot be empty!'))

```
IS_NOT_EMPTY('cannot be empty!')
```

?

```
db.person.name.requires = IS_NOT_EMPTY(error_message=T('fill this!'))
```

?T ?T ?

?

```
db.person.name.validate(value)
```

?(value, error)?error ?None .

?

```
>>> from pydal.validators import *
>>> IS_ALPHANUMERIC()('test')
('test', None)
>>> IS_ALPHANUMERIC()('test!')
('test!', 'Enter only letters, numbers, and underscore')
>>> IS_ALPHANUMERIC('this is not alphanumeric')('test!')
('test!', 'this is not alphanumeric')
>>> IS_ALPHANUMERIC(error_message='this is not alphanumeric')('test!')
('test!', 'this is not alphanumeric')
```

Hint ?

```
from pydal import validators
dir(validators) # get the list of all validators
help(validators.IS_URL) # get specific help for the IS_URL validator
```

12.6.1 ?

IS_ALPHANUMERIC

?

```
requires = IS_ALPHANUMERIC(error_message='must be alphanumeric!')
```

IS_LOWER

?

```
requires = IS_LOWER()
```

IS_UPPER

?

```
requires = IS_UPPER()
```

IS_EMAIL

?

```
requires = IS_EMAIL(error_message='invalid email!')
```

IS_MATCH

?

```
requires = IS_MATCH('^\d{5}(-\d{4})?$',
    error_message='not a zip code')
```

?

```
requires = IS_MATCH('^\d{1,3}(\.\d{1,3}){3}$',
        error_message='not an IP address')
```

?

```
requires = IS_MATCH('^1?((-)\d{3}-?|\(\d{3}\))\d{3}-?\d{4}$',
        error_message='not a phone number')
```

?

"IS_MATCH ?strict ?False?True ?

```
>>> IS_MATCH('ab', strict=False)('abc')
('abc', None)
>>> IS_MATCH('ab', strict=True)('abc')
('abc', 'Invalid expression')
```

IS_MATCH ?search ?False?True ?search ?match ?

IS_MATCH('...', extract=True) ?

IS_LENGTH

?

?

- maxsize: ?
- minsize: ?

?

```
>>> IS_LENGTH(15)('example string')
('example string', None)
>>> IS_LENGTH(15)('example long string')
('example long string', 'Enter from 0 to 15 characters')
>>> IS_LENGTH(15)('33')
('33', None)
>>> IS_LENGTH(15)(33)
('33', None)
```

?

```
INPUT(_type='file', _name='name', requires=IS_LENGTH(1048576, 1024))
```

?cgi.FieldStorage ?

IS_URL

?

- ?
- ?
- ?
- ?
- ?

(?RFC 2616)

?

- ?
- ?

- ?

?RFC 2396

?

?

IS_URL ?RFC 3490 ?RFC 3492 ?

?

```
requires = IS_URL())
requires = IS_URL(mode='generic')
requires = IS_URL(allowed_schemes=['https'])
requires = IS_URL(prepend_scheme='https')
requires = IS_URL(mode='generic',
                  allowed_schemes=['ftps', 'https'],
                  prepend_scheme='https')
```

IS_SAFE

```
requires = IS_SAFE(error_message='Unsafe Content')
requires = IS_SAFE(mode="sanitize")
requires = IS_SAFE(sanitizer=lambda text: str(XML(text, sanitize=True)))
```

?

IS_SLUG

```
requires = IS_SLUG(maxlen=80, check=False, error_message='must be slug')
```

?check ?True ?

?check ?False ?

IS_JSON

```
requires = IS_JSON(error_message='Invalid json', native_json=False)
```

?

?native_json ?False ?

12.6.2 ?

IS_TIME

?

```
requires = IS_TIME(error_message='must be HH:MM:SS!')
```

IS_DATE

?

```
requires = IS_DATE(format=T('%Y-%m-%d'),
    error_message='must be YYYY-MM-DD!')
```

?IS_DATETIME ?% ?

IS_DATETIME

?

```
requires = IS_DATETIME(format=T('%Y-%m-%d %H:%M:%S'),
                       error_message='must be YYYY-MM-DD HH:MM:SS!')
```

?

```
%Y  year with century (e.g. '1963')
%y  year without century (e.g. '63')
%d  day of the month (e.g. '28')
%m  month (e.g '08')
%b  abbreviated month name (e.g.'Aug')
%B  full month name (e.g. 'August')
%H  hour (24-hour clock, e.g. '14')
%I  hour (12-hour clock, e.g. '02')
%p  either 'AM' or 'PM'
%M  minute (e.g. '30')
%S  second (e.g. '59')
```

IS_DATE_IN_RANGE

?

```
requires = IS_DATE_IN_RANGE(format=T('%Y-%m-%d'),
                minimum=datetime.date(2008, 1, 1),
                maximum=datetime.date(2009, 12, 31),
                error_message='must be YYYY-MM-DD!')
```

?IS_DATETIME ?% ?

IS_DATETIME_IN_RANGE

?

```
requires = IS_DATETIME_IN_RANGE(format=T('%Y-%m-%d %H:%M:%S'),
                    minimum=datetime.datetime(2008, 1, 1, 10, 30),
                    maximum=datetime.datetime(2009, 12, 31, 11, 45),
                    error_message='must be YYYY-MM-DD HH:MM::SS!')
```

?IS_DATETIME ?% ?

12.6.3 ?

IS_EQUAL_TO

?

```
requires = IS_EQUAL_TO(request.vars.password,
                    error_message='passwords do not match')
```

IS_NOT_EMPTY

?.strip() ?

```
requires = IS_NOT_EMPTY(error_message='cannot be empty!')
```

?

```
requires = IS_NOT_EMPTY(error_message='Enter a value', empty_regex='NULL(?i)')
```

IS_NULL_OR

?

IS_EMPTY_OR

?IS_EMPTY_OR ?

```
requires = IS_EMPTY_OR(IS_DATE())
```

?.strip() ?

?empty_regex ?

?

```
requires = IS_EMPTY_OR(IS_ALPHANUMERIC(), null='anonymous')
```

IS_EXPR

?None ?

```
requires = IS_EXPR(lambda v: T('not divisible by 3') if int(v) % 3 else None)
```

? ?

?True ?

```
requires = IS_EXPR('int(value) % 3 == 0',
                   error_message='not divisible by 3')
```

?

```
requires = [IS_INT_IN_RANGE(0, None),
            IS_EXPR(lambda v: T('not divisible by 3') if v % 3 else None)]
```

IS_DECIMAL_IN_RANGE

```
INPUT(_type='text', _name='name', requires=IS_DECIMAL_IN_RANGE(0, 10, dot="."))
```

?

?

dot ?

IS_FLOAT_IN_RANGE

?0 <= value <= 100 ?

```
requires = IS_FLOAT_IN_RANGE(0, 100, dot=".",
                             error_message='negative or too large!')
```

dot ?

IS_INT_IN_RANGE

? ?0 <= value < 100 ?

```
requires = IS_INT_IN_RANGE(0, 100,
                           error_message='negative or too large!')
```

IS_IN_SET

?

IS_IN_SET ?

```
requires = IS_IN_SET(['a', 'b', 'c'], zero=T('choose one'),
                     error_message='must be a or b or c')
```

zero ?IS_IN_SET ?zero=None ?

?IS_IN_SET ?S_IN_SET ?IS_INT_IN_RANGE ?IS_FLOAT_IN_RANGE ?

```
requires = [ IS_IN_SET([2, 3, 5, 7], error_message='must be prime and less than
10'),
             IS_INT_IN_RANGE(0, None) ]
```

?

?

```
requires=IS_IN_SET(['ON'])
```

?

?

```
# Dictionary example:
requires = IS_IN_SET({'A':'Apple', 'B':'Banana', 'C':'Cherry'}, zero=None)

# List of tuples example:
requires = IS_IN_SET([('A', 'Apple'), ('B', 'Banana'), ('C', 'Cherry')])
```

?

?IS_IN_SET ?sort ?

```
IS_IN_SET([('H', 'Hulk'), ('S', 'Superman'), ('B', 'Batman')], sort=True)
```

IS_IN_SET ?

IS_IN_SET ?multiple=False ?list:integer ?list:string ?Section 7.14.1 ?

? ?multiple=True ?IS_IN_SET ?multiple ?(a, b) ?a ?b ?

12.6.4 ?

IS_STRONG

?

?

```
requires = IS_STRONG(min=10, special=2, upper=2)
```

?

- min ?
- special ?^!!@#$%^&*()_+-=?<>,.:;{}[]| ?specials = '...' ?
- upper ?

?

- invalid ?invalid=' "'
- max ?
- lower ?
- number ?

?error_message ?IS_STRONG ?

?entropy ?

```
>>> IS_STRONG(entropy=100.0)('hello')
('hello', Entropy (24.53) less than required (100.0))
```

? ?entropy ?IS_STRONG ?min = 8, upper = 1, lower = 1, number = 1, special = 1?

CRYPT

?

```
requires = CRYPT()
```

?

?

```
requires = CRYPT(key='sha512:thisisthekey')
```

?

```
requires = CRYPT(salt='mysaltvalue')
```

?

```
requires = CRYPT(salt=False)
```

CRYPT ?

```
requires = [IS_STRONG(), CRYPT(key='sha512:thisisthekey')]
```

CRYPT ?min_length ?

?alg$salt$hash ?alg ?salt ?hash ?

12.6.5 ?

IS_LIST_OF

?minimum` `? ?error_message ?

```
requires = IS_LIST_OF(minimum=2)
```

?

```
>>> IS_LIST_OF()('hello')
(['hello'], None)
```

?list: ?

```
Field('emails', 'list:string', requires=IS_LIST_OF(IS_EMAIL()), ...)
```

IS_LIST_OF_EMAILS

?

```
Field('emails', 'list:string',
      widget=SQLFORM.widgets.text.widget,
    requires=IS_LIST_OF_EMAILS(),
    represent=lambda v, r:
        XML(', '.join([A(x, _href='mailto:'+x).xml() for x in (v or [])]))
    )
```

?widget ?, ?; ?requires ?filter_in ?accepts ?process ?validate ?onvalidation ?

```
def emails_onvalidation(form):
    form.vars.emails = IS_LIST_OF_EMAILS.split_emails.findall(form.vars.emails)
```

represent ?

ANY_OF

?

```
requires = ANY_OF([IS_ALPHANUMERIC(), IS_EMAIL()])
```

?

```
>>> ANY_OF([IS_ALPHANUMERIC(), IS_EMAIL()])('@ab.co')
('@ab.co', 'Enter a valid email address')
>>> ANY_OF([IS_ALPHANUMERIC(), IS_EMAIL()],
...        error_message='Enter login or email')('@ab.co')
('@ab.co', 'Enter login or email')
```

IS_IMAGE

?

?

?

?

?

?

```
requires = IS_IMAGE()
```

- ?

```
requires = IS_IMAGE(extensions=('jpeg', 'png'))
```

- ?

```
requires = IS_IMAGE(extensions=('png'), maxsize=(200, 200))
```

?`requires=IS_IMAGE()``?`?`delete`?

```
requires = IS_EMPTY_OR(IS_IMAGE())
```

IS_FILE

?

?

?

- filename: ?
- extension: ?
- lastdot: ?`True` ?`False`?
- case: 0 ?

?

?

```
INPUT(_type='file', _name='name',
        requires=IS_FILE(extension='pdf'))
```

?

```
INPUT(_type='file', _name='name',
        requires=IS_FILE(filename='thumbnail',
        extension=['jpg', 'png']))
```

?

```
INPUT(_type='file', _name='name',
        requires=IS_FILE(filename=re.compile('backup.*'),
        extension='tar.gz', lastdot=False))
```

?

```
INPUT(_type='file', _name='name',
    requires=IS_FILE(filename='README',
    extension='', case=0)
```

IS_UPLOAD_FILENAME

?`IS_FILE()` ?

?

?

?

- filename: ?
- extension: ?
- lastdot: ?`True` ?`False` ?
- case: 0 ?

?

?

?

```
requires = IS_UPLOAD_FILENAME(extension='pdf')
```

?

```
requires = IS_UPLOAD_FILENAME(filename='backup.*', extension='tar.gz',
lastdot=False)
```

?

```
requires = IS_UPLOAD_FILENAME(filename='^README$', extension='^$', case=0)
```

IS_IPV4

?

IPv4 ?`regexlib` ?`IS_IPV4` ?

```
IS_IPV4(minip='0.0.0.0', maxip='255.255.255.255', invert=False,
        is_localhost=None, is_private=None, is_automatic=None,
        error_message='Enter valid IPv4 address')
```

?

- `minip` ?
- `maxip` ?
- `invert` ?

?

?`minip` ?`maxip` ?

```
requires = IS_IPV4(minip=('192.168.20.10', '192.168.30.100'),
                   maxip=('192.168.20.19', '192.168.30.199'))
```

?minip ?maxip ?

?is_localhost ?is_private ?is_automatic ?

- None ?
- True ?
- False ?

?

- is_localhost: ?
- is_private: ?
- is_automatic: ?

?

?

```
requires = IS_IPV4()
```

?

```
requires = IS_IPV4(minip='192.168.0.1', maxip='192.168.255.255')
```

IS_IPV6

?

IS_IPV6 ?

```
IS_IPV6(is_private=None,
        is_link_local=None,
        is_reserved=None,
        is_multicast=None,
        is_routeable=None,
        is_6to4=None,
        is_teredo=None,
        subnets=None,
        error_message='Enter valid IPv6 address')
```

?is_private ?is_link_local ?is_reserved ?is_multicast ?is_routeable ?is_6to4 ?is_teredo ?

- None ?
- True ?
- False ?is_routeable ?

?

- is_private: ?
- is_link_local: ?is_private ?
- is_reserved: ?
- is_multicast: ?
- is_6to4: ?
- is_teredo: ?

?is_routeable ?is_private ?is_reserved ?is_multicast ?

?subnets ?

?

?

```
requires = IS_IPV6()
```

?

```
requires = IS_IPV6(is_link_local=True)
```

?

```
requires = IS_IPV6(subnets='fb00::/8')
```

IS_IPADDRESS

?IS_IPV4 ?IS_IPV6 ?

IS_IPADDRESS ?

```
IS_IPADDRESS(minip='0.0.0.0', maxip='255.255.255.255', invert=False,
             is_localhost=None, is_private=None, is_automatic=None,
             is_ipv4=None,
             is_link_local=None, is_reserved=None, is_multicast=None,
             is_routeable=None, is_6to4=None, is_teredo=None,
             subnets=None, is_ipv6=None,
             error_message='Enter valid IP address')
```

?IS_IPV4 ?IS_IPV6 ?

- is_ipv4,?
- is_ipv6,?

?

?

?

```
requires = IS_IPADDRESS()
```

?

```
requires = IS_IPADDRESS(is_ipv6=True)
```

12.6.6 ?

CLEANUP

?

```
requires = CLEANUP()
```

?

```
>>> CLEANUP('[^\\d]')('Hello 123 world 456')
('123456', None)
```

12.6.7 ?

IS_NOT_IN_DB

?IS_NOT_IN_DB(db|set, 'table.field')

?

```
db.define_table('person', Field('name'))
db.person.name.requires = IS_NOT_IN_DB(db, 'person.name')
```

?db ?person.name ?

?db

?

```
db.define_table('person', Field('name', unique=True))
db.person.name.requires = IS_NOT_IN_DB(db, 'person.name')
```

?

IS_NOT_IN_DB ?

IS_NOT_IN_DB()` ?

```
IS_NOT_IN_DB(dbset, field, error_message='value already in database or empty',
             allowed_override=[], ignore_common_filters=True)
```

?

```
import datetime
now = datetime.datetime.today()
db.define_table('person',
    Field('name'),
    Field('registration_stamp', 'datetime', default=now))
recent = db(db.person.registration_stamp > now-datetime.timedelta(10))
db.person.name.requires = IS_NOT_IN_DB(recent, 'person.name')
```

IS_IN_DB

?IS_IN_DB(db|set, 'table.value_field', '%(representing_field)s', zero='choose one') ?

?multiple= ?multiple=(1, 10) ?

?

?

```
db.define_table('person', Field('name', unique=True))
db.define_table('dog', Field('name'), Field('owner', db.person))
db.dog.owner.requires = IS_IN_DB(db, 'person.id', '%(name)s',
                                 zero=T('choose one'))
```

IS_IN_DB ?

```
db.dog.owner.requires = IS_IN_DB(db(db.person.id > 10), 'person.id', '%(name)s',
                                 zero=T('choose one'))
```

?dog.owner ?db ?person.id ?dog.owner ?label ?%(name)s ?%(id)s ?%(...)s ?label ?

zero ?IS_IN_SET ?

IS_IN_DB ?orderby ?groupby ?distinct ?cache ?left ?Section 7.9.6 ?

? ?groupby ?distinct ?left ?

?sort ?True ?

?IS_NOT_IN_DB ?

```
def index():
    (...)
    query = (db.table.field == 'xyz') # in practice 'xyz' would be a variable
    db.table.field.requires = IS_IN_DB(db(query), ...)
    form = Form(...)
    if form.process().accepted: ...
    (...)
```

?

```
db.dog.owner.requires = [IS_IN_DB(db, 'person.id', '%(name)s')]
```

?IS_IN_DB ?_and ?IS_IN_DB ?

```
subset = db(db.person.id > 100)
db.dog.owner.requires = IS_IN_DB(db, 'person.id', '%(name)s',
                                 _and=IS_NOT_IN_DB(subset, 'person.id'))
```

IS_IN_DB ?

IS_IN_DB ?multiple=False?True ?list:reference ?Section 7.14.1 ?

12.6.8 ?

?validation ?

?

```
from py4web import Field
from py4web.utils.form import Form, FormStyleBulma
from pydal.validators import IS_INT_IN_RANGE

def custom_check(form):
    if not 'name' in form.errors and len(form.vars['name']) < 4
        form.errors['name'] = T("too short")

@action('form_example', method=['GET', 'POST'])
@action.uses('form_example.html', session)
def form_example():
    form = Form(db.thing, validation=custom_check)
    if form.accepted:
        redirect(URL('index'))
    return dict(form=form)
```

?

?

- ?
- ?

13.1 ?

py4web ?`Auth` ?

_scaffold ?

?

```
from py4web.utils.auth import Auth
auth = Auth(session, db)
# (configure here)
auth.enable()
```

?`session['user']` ?`session['user']['id']` ?db ?`auth_user` ?`auth_user` ?

- username
- email
- password
- first_name
- last_name
- sso_id ?
- action_token ?

`auth.enable()` ?

- {appname}/auth/api/register (POST)
- {appname}/auth/api/login (POST)
- {appname}/auth/api/request_reset_password (POST)
- {appname}/auth/api/reset_password (POST)
- {appname}/auth/api/verify_email (GET, POST)
- {appname}/auth/api/logout (GET, POST) (+)
- {appname}/auth/api/profile (GET, POST) (+)
- {appname}/auth/api/change_password (POST) (+)

- {appname}/auth/api/change_email (POST) (+)

?

13.1.1 Auth UI

?

- _scaffold/templates/auth.html
- _scaffold/templates/layout.html

?`layout.html` ?

```
<ul>
   [[if globals().get('user'):]]
   <li>
   <a class="navbar-link is-primary">
      [[=globals().get('user',{}).get('email')]]
   </a>
   <ul>
      <li><a href="[[=URL('auth/profile')]]">Edit Profile</a></li>
      [[if 'change_password' in
globals().get('actions',{}).get('allowed_actions',{}):]]
         <li><a href="[[=URL('auth/change_password')]]">Change Password</a></li>
      [[pass]]
      <li><a href="[[=URL('auth/logout')]]">Logout</a></li>
   </ul>
   </li>
   [[else:]]
   <li>
   Login
   <ul>
      <li><a href="[[=URL('auth/register')]]">Sign up</a></li>
      <li><a href="[[=URL('auth/login')]]">Log in</a></li>
   </ul>
   </li>
   [[pass]]
</ul>
```

?`Edit Profile` ?`Change Password` ?`Logout` ?`Sign up` ?`Log in` ?

?

13.1.2 ?

?

?`@action.uses(auth)` ?

```
@action('index')
@action.uses(auth)
def index():
    user = auth.get_user()
    return 'hello {first_name}'.format(**user) if user else 'not logged in'
```

?

```
@action('index')
@action.uses(auth.user)
def index():
    user = auth.get_user()
    return 'hello {first_name}'.format(**user)
```

?`@action.uses(auth.user)` ?

13.1.3 ?

?

?

?

- two_factor_required
- two_factor_send
- two_factor_validate

two_factor_required

?two_factor_required ?

two_factor_required ?

?

```
def user_outside_network(user, request):
    import ipaddress

    networks = ["10.10.0.0/22"]

    ip_list = []
    for range in networks:
        ip_list.extend(ipaddress.IPv4Network(range))

    if ipaddress.IPv4Address(request.remote_addr) in ip_list:
        #  if the client address is in the network address list, then do NOT
require MFA
        return False

    return True
```

two_factor_send

?

?

```
def send_two_factor_email(user, code):
    try:
        auth.sender.send(
            to=[user.email],
            subject=f"Two factor login verification code",
            body=f"You're verification code is {code}",
            sender="from_address@youremail.com",
        )
    except Exception as e:
        print(e)
    return code
```

?

```
auth.param.two_factor_required = user_outside_network
auth.param.two_factor_send = send_two_factor_email
```

two_factor_validate

?two_factor_send ?two_factor_send ?

?two_factor_validate ?

- user ?
- code ?

?

?

```
def validate_code(user, code):
    try:
        # get the correct code from an external function
        correct_code = generate_time_based_code(user_id)
    except Exception as e:
        # return None to indicate that validation could not be performed
        return None

    # compare the value entered in the auth form with the correct code
    if code == correct_code:
        return True
    else:
        return False
```

validate_code ?

- True - ?
- False - ?
- None - ?

?True ?False ?None ?two_factor_send ?two_factor_send ?

```
auth.param.two_factor_validate = validate_code
```

two_factor_tries

?

```
auth.param.two_factor_tries = 5
```

?

- ?
- **?**
  - ?
  - **?two_factor_send ?**
    - ?
    - ?two_factor_send ?
  - ?
  - ?two_factor_validate ?
  - ?

?ALLOWED_ACTIONS ?

13.1.4 ?

?

<auth/> ?

?

PAM

?

```
from py4web.utils.auth_plugins.pam_plugin import PamPlugin
auth.register_plugin(PamPlugin())
```

?

`auth.register_plugin(...)` ? ?`auth.enable()` ?

Note ?

LDAP

?

```
from py4web.utils.auth_plugins.ldap_plugin import LDAPPlugin
LDAP_SETTING = {
    'mode': 'ad',
    'server': 'my.domain.controller',
    'base_dn': 'cn=Users,dc=domain,dc=com'
}
auth.register_plugin(LDAPPlugin(**LDAP_SETTINGS))
```

Warning ?`sudo apt-get install libldap2-dev libsasl2-dev` ?

?

```
from py4web.utils.auth_plugins.oauth2google import OAuth2Google # TESTED
auth.register_plugin(OAuth2Google(
    client_id=CLIENT_ID,
    client_secret=CLIENT_SECRET,
    callback_url='auth/plugin/oauth2google/callback'))
```

?

?

```
from py4web.utils.auth_plugins.oauth2google import OAuth2Google  # TESTED
...
auth = Auth(session, db, define_tables=False)
auth.extra_auth_user_fields = [
   Field('profile_picture', 'text', readable=False, writable=False)
]
...
OAuth2Google.maps['profile_picture'] = 'picture'
```

?

?

```
from py4web.utils.auth_plugins.oauth2facebook import OAuth2Facebook # UNTESTED
auth.register_plugin(OAuth2Facebook(
    client_id=CLIENT_ID,
    client_secret=CLIENT_SECRET,
    callback_url='auth/plugin/oauth2google/callback'))
```

?

?

```
from py4web.utils.auth_plugins.oauth2discord import OAuth2Discord
auth.register_plugin(OAuth2Discord(
    client_id=DISCORD_CLIENT_ID,
    client_secret=DISCORD_CLIENT_SECRET,
    callback_url="auth/plugin/oauth2discord/callback"))
```

?https://discord.com/developers/applications ?`http(s)://<your host>/<your app name>/auth/plugin/oauth2discord/callback` ?

Note ?

13.1.5 Auth API ?

?

?

Py4web ?

?

- ?
- ?

SimpleTokenPlugin ?

- ?
- ?
- ?

SimpleTokenPlugin ?

- ?
- ?
- ?
- ?
- ?

JvtokenPlugin ?

- ?
- ?
- ?
- ?
- ?

SimpleTokenPlugin ?

SimpleTokenPlugin ?

?

```
from py4web.utils.auth import SimpleTokenPlugin
simple_token_plugin = SimpleTokenPlugin(auth)
auth.token_plugins.append(simple_token_plugin)
```

?table=db.mytable ?

?

```
@action("test_api")
@action.uses(auth.user)
def test_api():
    return {"hello": "world"}
```

?

```
curl http://127.0.0.1:8000/test1/test_api -H "Authorization: Bearer {token}"
```

?

```
@action("tokens")
def _():
   db.auth_simple_token.user_id.default = auth.user_id
   grid = Grid(db.auth_simple_token.user_id==auth.user_id, create=True,
deletable=True)
   return dict(grid=grid)
```

JwtTokenPlugin ?

?

```
from py4web.utils.auth import SimpleTokenPlugin
jwt_token_plugin = JwtTokenPlugin(auth)
auth.token_plugins.append(jwt_token_plugin)
```

?

```
@action("test_api")
@action.uses(auth.user)
def test_api():
    return {"hello": "world"}
```

?

```
curl http://127.0.0.1:8000/test1/test_api -H "Authorization: Bearer {token}"
```

?

```
@action("make_token")
@action.uses("generic.html", auth.user)
def make_token():
     return dict(token=jwt_token_plugin.make(
        auth.current_user,
        expiration=utcnow()+datetime.timedelta(days=10)))
```

?

?

?

```
class MyCustomTokenPlugin:
    def get_user(self):
        authorization = request.headers.get("Authentication")
        if authorization:
            return db.auth_user(1)
        return None

auth.token_plugins.append(MyCustomTokenPlugin())
```

13.2 ?

?`Tags` ?Section 7.7.7 ?

13.2.1 Tags ?

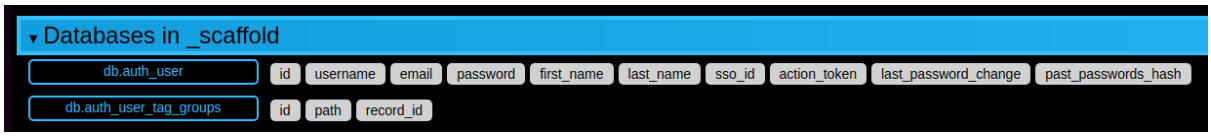
Py4web ?

Note Py4web ?

?`pydal.tools` ?

```
from pydal.tools.tags import Tags
groups = Tags(db.auth_user, 'groups')
```

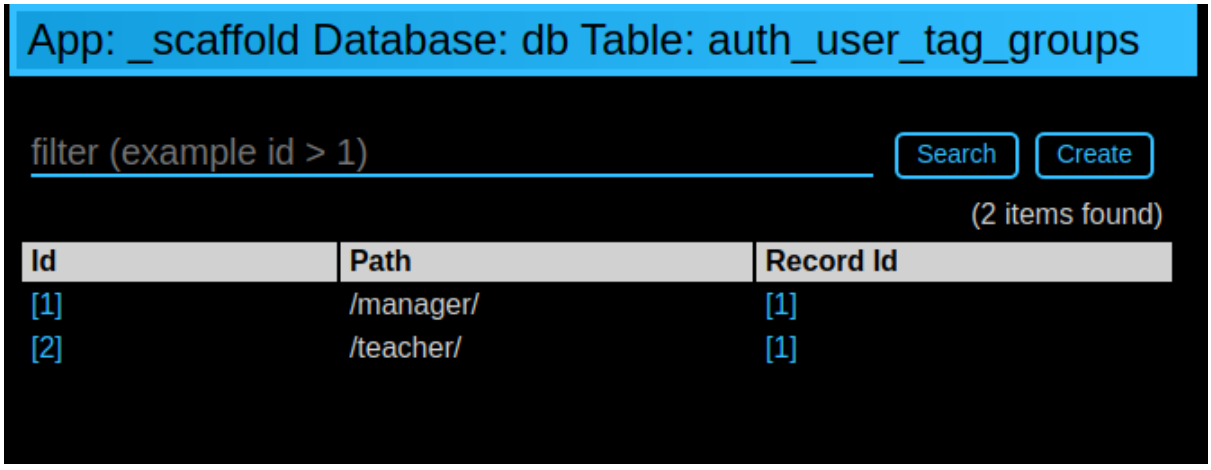
tail_name ?`tagged_db.table + '_tag_' + tail_name` ?`auth_user_tag_groups` ?



?

```
groups.add(user.id, 'manager')
groups.add(user.id, ['dancer', 'teacher'])
groups.remove(user.id, 'dancer')
```

?`auth_user_tag_groups` ?



?

?**?** ?`'manager'` ?`'not authorized url'` ?

```
@action('index')
@action.uses(auth.user)
def index():
    if not 'manager' in groups.get(auth.get_user()['id']):
        redirect(URL('not_authorized'))
    return 'hello manager'
```

?

```
@action('find_by_tag/{group_name}')
```

```
@action.uses(db)
def find(group_name):
    users = db(groups.find([group_name])).select(orderby=db.auth_user.first_name |
db.auth_user.last_name)
    return {'users': users}
```

?The Condition fixture ?requires_membership ?

```
groups = Tags(db.auth_user)

class requires_membership(Fixture):
    def __init__(self, group):
        self.__prerequisites__ = [auth.user] # you must have a user before you can
check
        self.group  = group # store the group when action defined
    def on_request(self, context): # will be called if the action is called
        if self.group not in groups.get(auth.user_id):
            raise HTTP(401) # check and do something

@action('index')
@action.uses(requires_membership('teacher'))
def index():
    return 'hello teacher'
```

?has_membership ?

```
@action('index')
@action.uses(has_membership(groups, 'teacher'))
def index():
    return 'hello teacher'
```

?Tags ?

- groups.find('teacher/high-school/physics')
- groups.find('teacher/high-school')
- groups.find('teacher')

?

13.2.2 ?

Note ?Tags ?Tags ?

?auth_group ?

```
db.define_table('auth_group', Field('name'), Field('description'))
```

?

```
groups = Tags(db.auth_user)
permissions = Tags(db.auth_groups)
```

?auth_group ?

```
zap_id = db.auth_group.insert(name='zapper', description='can zap database')
permissions.add(zap_id, 'zap database')
groups.add(user.id, 'zapper')
```

?

```
@action('zap')
@action.uses(auth.user)
```

```
def zap():
    user = auth.get_user()
    permission = 'zap database'
    if db(permissions.find(permission))(
          db.auth_group.name.belongs(groups.get(user['id']))
          ).count():
        # zap db
        return 'database zapped'
    else:
        return 'you do not belong to any group with permission to zap db'
```

?`permissions.find(permission)` ?

13.2.3 ?

Auth ?

```
@action("impersonate/{user_id:int}", method="GET")
@action.uses(auth.user)
def start_impersonating(user_id):
    if (not auth.is_impersonating() and
        user_id and
        user_id != auth.user_id and
        db(db.auth_user.id==user_id).count()):
        auth.start_impersonating(user_id, URL("index"))
    raise HTTP(404)

 @action("stop_impersonating", method="GET")
 @action.uses(auth)
 def stop_impersonating():
    if auth and auth.is_impersonating():
        auth.stop_impersonating(URL("index"))
    redirect(URL("index"))
```

?

py4web?

14.1 ?

- ?
- ?
- ?
- ?
- ?
- ?
- ?
- ?

Hint Jim Steil ?https://github.com/jpsteil/grid_tutorial ?

14.2 grid ?

?

?`grid`?

```
# in grid/__init__.py
import os
from py4web import action, Field, DAL
from py4web.utils.grid import *
from py4web.utils.form import *
from yatl.helpers import A


# database definition
DB_FOLDER = os.path.join(os.path.dirname(__file__), 'databases')
if not os.path.isdir(DB_FOLDER):
    os.mkdir(DB_FOLDER)
db = DAL('sqlite://storage.sqlite', folder=DB_FOLDER)
db.define_table(
    'person',
    Field('superhero'),
```

```
    Field('name'),
    Field('job'))

# add example entries in db
if not db(db.person).count():
    db.person.insert(superhero='Superman', name='Clark Kent', job='Journalist')
    db.person.insert(superhero='Spiderman', name='Peter Park', job='Photographer')
    db.person.insert(superhero='Batman', name='Bruce Wayne', job='CEO')
    db.commit()

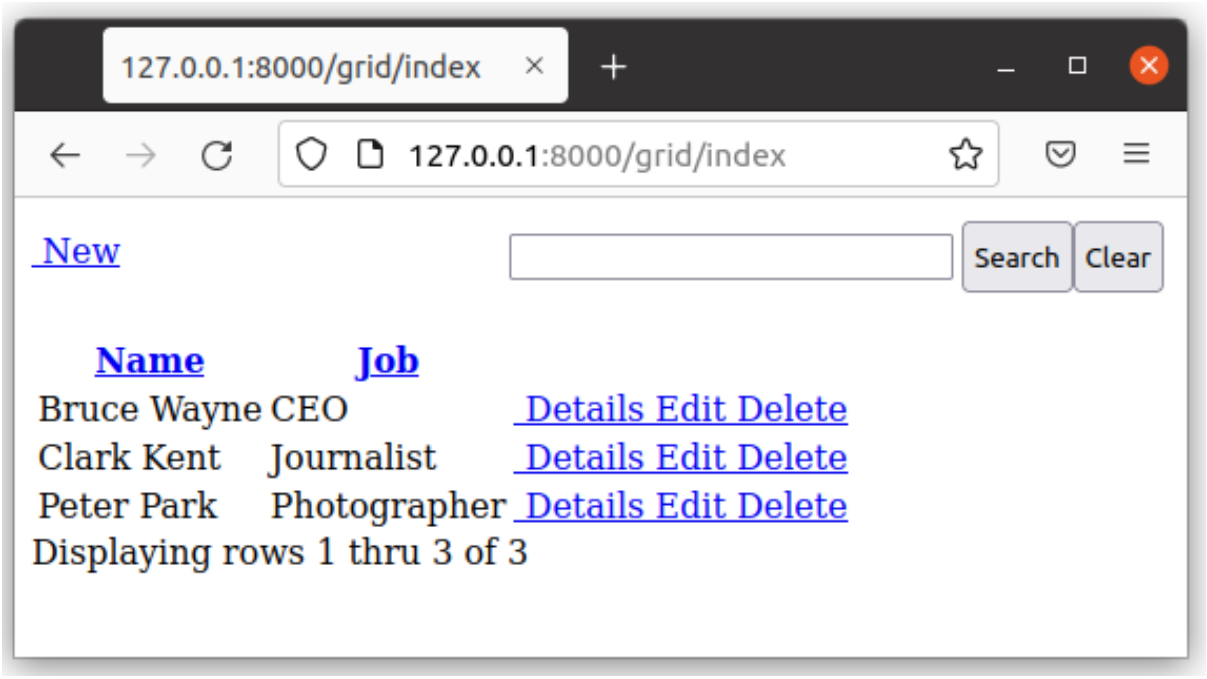
@action('index', method=['POST', 'GET'])
@action.uses('grid.html', db)
def index():
    grid = Grid(
            formstyle=FormStyleDefault, # FormStyleDefault, FormStyleBulma,
FormStyleBootstrap4, or FormStyleBootstrap5
            grid_class_style=GridClassStyle, # GridClassStyle or
GridClassStyleBulma or GridClassStyleBootstrap5
            icon_style=IconStyleFontawesome, # IconStyle, IconStyleFontawesome, or
IconStyleBootstrapIcons
            query=(db.person.id > 0),
            orderby=[db.person.name],
            search_queries=[['Search by Name', lambda val:
db.person.name.contains(val)]])

    return dict(grid=grid)
```

?

```
[[=grid.render()]]
```

?http://127.0.0.1:8000/grid/index ?



?

?here ?

```
<!DOCTYPE html>
<html>
    <head>
        <link rel="stylesheet"
```

```
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.14.0/css/all.min.css"
/>
   </head>
   <body>
      [[=grid.render()]]
   <body>
</html>
```

?



?

?bulma.css ?here ?

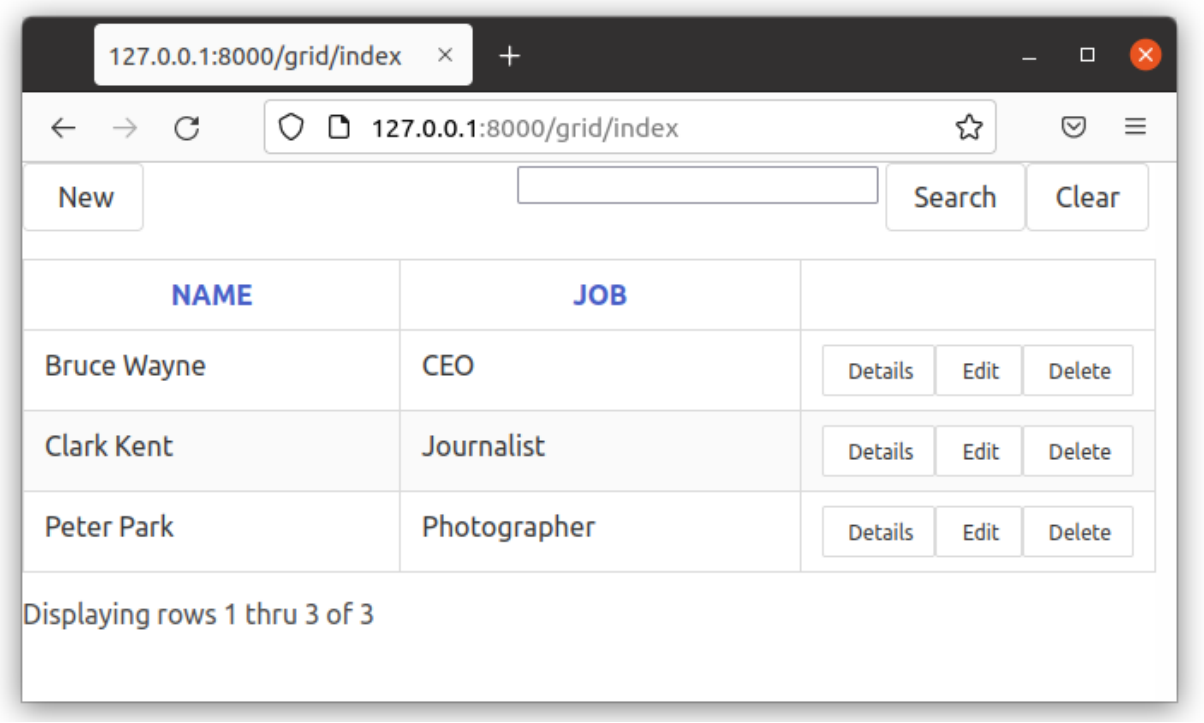
```
formstyle=FormStyleBulma, # FormStyleDefault or
FormStyleBulma,FormStyleBootstrap4, or FormStyleBootstrap5
grid_class_style=GridClassStyleBulma, # GridClassStyle or GridClassStyleBulma or
GridClassStyleBootstrap5
```

?

?

```
<!DOCTYPE html>
<html>
   <head>
      <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/bulma/0.9.3/css/bulma.min.css">
   </head>
   <body>
         [[=grid.render()]]
   <body>
</html>
```

?



?

Bootstrap4 ?Section 14.8 ?

Note ?`grid` ?`_scaffold` ?

?http://127.0.0.1:8000/grid ?

?Chapter 16 ?**htmx** ?https://github.com/jpsteil/grid_tutorial ?

14.3 Grid ?

```
class Grid:
    def __init__(
        self,
        query,
        search_form=None,
        search_queries=None,
        columns=None,
        field_id=None,
        show_id=False,
        orderby=None,
        left=None,
        groupby=None,
        headings=None,
        create=True,
        details=True,
        editable=True,
        deletable=True,
        validation=None,
        pre_action_buttons=None,
        post_action_buttons=None,
        auto_process=True,
        rows_per_page=15,
```

```
        include_action_button_text=True,
        search_button_text="Filter",
        formstyle=FormStyleDefault,
        grid_class_style=GridClassStyle,
        icon_style=IconStyleFontawsome,
        T=lambda text: text,
    ):
```

- query?
- search_form?
- search_queries?
- columns?Section 14.4 ?
- show_id?
- orderby?
- left?
- groupby?
- headings?
- create?
- details?
- editable: ?
- deletable?
- required_fields?
- validation?
- pre_action_buttons?
- post_action_buttons: ?
- auto_process?
- rows_per_page?
- include_action_button_text?
- search_button_text?
- formstyle?
- grid_class_style: ?
- icon_style: ?
- T: ?

(*) ?`details` ?`editable` ?`deletable` ?

```
deletable=lambda row: False if row.job=="CEO" else True,
```

?Section 14.10

14.3.1 ?

?

- ?
- ?
- ?

?

14.3.2 CRUD ?

grid ?
?

14.4 ?

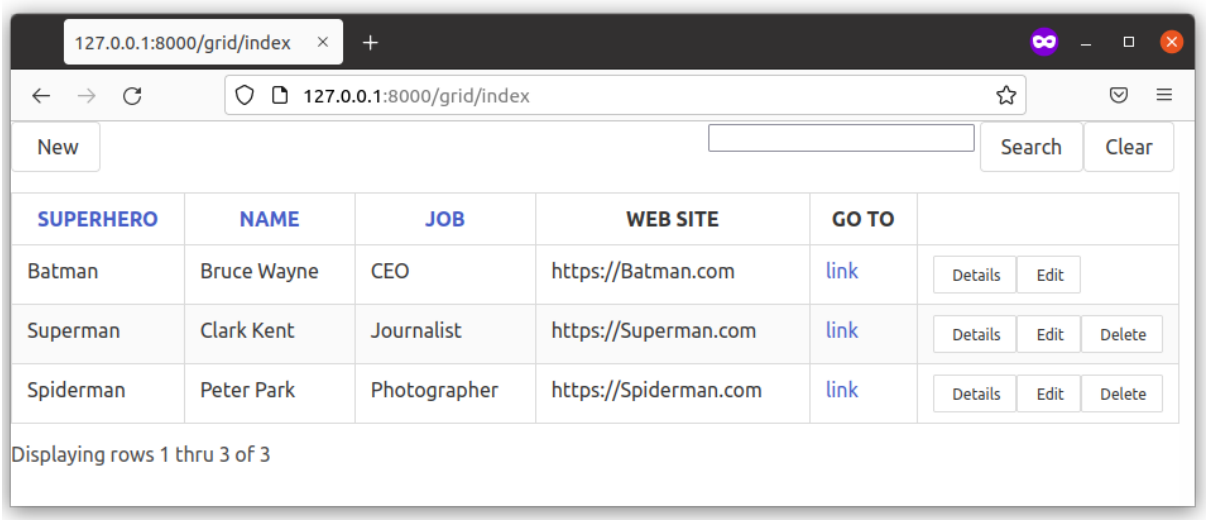
?

```
from py4web.utils.grid import Column
from yatl.helpers import A

columns = [
    db.person.id,
    db.person.superhero,
    db.person.name,
    db.person.job,
    Column("Web Site", lambda row: f"https://{row.superhero}.com"),
    Column("Go To", lambda row: A("link", _href=f"https://{row.superhero}.com"))
]

grid = Grid(... columns=columns ...)
```

?`A` helper ?



?`deletable` ?

Warning ?
?

14.5 ?

?

```
[[=grid.render()]]
```

?Section 12.5.2 ?

```
[[form = grid.render() ]]
[[form.custom["begin"] ]]
...
[[form.custom["submit"]
[[form.custom["end"]
```

?

```
[[if grid.action in ['details', 'edit']: ]]
    #  Display the custom form
    [[form = grid.render() ]]
    [[form.custom["begin"] ]]
    ...
    [[form.custom["submit"]
    [[form.custom["end"]
[[else:]]
    [[grid.render() ]]
[[pass]]
```

14.6 Grid ?

?*Grid* ?*path* ?

?

```
grid = Grid(
   path=request.path,
   query=(db.person.id > 0),
   # ...other parameters...
)
```

?*path* ?*path* ?

?

```
grid = Grid(
   query=(db.person.id > 0),
   # ...other parameters...
)
```

?

?

14.7 Grid ?

Grid ?*mode* ?*request.query* ?*mode* ?

- *select*: ?
- *new*: ?
- *details*: ?
- *edit*: ?
- *delete*: ?

grid ?*mode* ?*/myapp/mygrid?mode=edit&id=5* ?*mode* ?*select* ?

?

14.8 ?

?

formstyle ?

?`GridClassStyle` ?**no.css** ?`GridClassStyleBulma` ?`GridClassStyleBootstrap5`?

?

?

?fontawesome ?`icon-edit-button` ?Bootstrap Icons ?

14.9 ?

?**init** ?`pre_action_buttons` ?`post_action_buttons` ?

?

?

?

```
pre_action_buttons = [
    lambda row: dict(
        url = f"https://www.google.com/search?q={row.superhero}",
        text = f"Google for {row.superhero}",
        _style = "font-weight: 200"
    )
]

grid = Grid(... pre_action_buttons = pre_action_buttons  ...)
```

?

- url?
- text?
- icon?`fa-gear`
- lasses?
- kind?
- _{name}?`A(..., _{name}=...)` helper ?

?

```
pre_action_buttons = [
    lambda row: A(
        f"Google for {row.superhero}",
        _href = f"https://www.google.com/search?q={row.superhero}",
        _style = "font-weight: 200",
    )
]

grid = Grid(... pre_action_buttons = pre_action_buttons  ...)
```

14.10 ?

?**?** ?

?

- details
- editable
- deletable
- additional_classes
- additional_styles
- override_classes
- override_styles

?

```
@action("example/<path:path>")
def example(path=None):

   pre_action_buttons = [
      lambda row: GridActionButton(
            URL("test", row.id),
            text="Click me",
            icon=IconStyleFontawsome.add_button, # same as "fa-plus"
            additional_classes=row.id,
            additional_styles=["height: 10px" if row.bar else None],
      )
   ]

   post_action_buttons = [
      lambda row: GridActionButton(
            URL("test", row.id),
            text="Click me!!!",
            icon="fa-plus",
            additional_classes=row.id,
            additional_styles=["height: 10px" if row.bar else None],
      )
   ]

   grid = Grid(
      path=path,
      query=db.foo,
      pre_action_buttons=pre_action_buttons,
      post_action_buttons=post_action_buttons,
   )

   return dict(grid=grid.render())
```

14.11 ?

?

PyDAL ?`filter_out` - ?

```
Field('company', 'reference company',
```

```
      requires=IS_NULL_OR(IS_IN_DB(db, 'company.id',
                                   '%(name)s',
                                   zero='..')),
      filter_out=lambda x: x.name if x else ''),
```

?

?

left join ?

```
db.company.on(db.employee.company == db.company.id)
```

?

?

```
queries.append((db.employee.last_name.contains(search_text)) |
(db.employee.first_name.contains(search_text)) |
db.company.name.contains(search_text))
```

?

?

14.12 ?

?<form> ?

```
column = Column("select", lambda row:
INPUT(_type="checkbox",_name="selected_id",_value=row.id))

@action("manage")
@action.uses("manage.html", db)
def manage():

   grid = Grid(db.thing, columns=[column, db.thing.name])

   # if we are displaying a "select" grid page (not a form)
   if not grid.form:
      grid = grid.render()
      # if checkboxes selection was submitted
      if request.method == "POST":
         # do something with the selected ids
         print("you selected", request.POST.get("selected_id"))
      # inject a ``<form>`` and a ``submit`` button
      grid.children[1:] = [FORM(
            *grid.children[1:],
            DIV(INPUT(_type="submit",_value="do it!")),
            _method="POST",
            _action=request.url)]
   return locals()
```

?request.POST.get("selected_id") ?

Chapter 15

?

?

Web2py ?`{{...}}` ?`[[...]]` ?*XML* ?*HTTP* ?*Auth* ?

?

- web2py ?
- web2py ?
- ?`controllers/*.py` ?`__init__.py` ?`static` ?
- ?*__init__.py* ?
- ?`controllers/*.py` ?`@action("...")` ?`@action("...")` ?
- ?*@action('my_url_path')* ?
- ?http://*.html" ?http://*.json" ?
- ?`@action.uses(...)` ?`@action.uses(...)` ?`@action.uses(session, T, db, flash)` ?
- web2py ?
- web2py ?Section 7.3.5 ?
- web2py ?*auth.requires_** ?
- Web2py ?Rocket3 ?

15.1 ?

15.1.1 "Hello world" ?

web2py

```
# in controllers/default.py
def index():
   return "hello world"
```

--> py4web

```
# file imported by __init__.py
@action('index')
def index():
    return "hello world"
```

15.1.2 “?

web2py

```
request.get_vars.name
request.post_vars.name
request.env.name
raise HTTP(301)
redirect(url)
URL('c','f',args=[1,2],vars={})
```

--> py4web

```
request.query.get('name')
request.forms.get('name') or request.json.get('name')
request.environ.get('name')
raise HTTP(301)
redirect(url)
URL('c', 'f', 1, 2, vars={})
```

15.1.3 “?

web2py

```
def index():
    a = request.get_vars.a
    return locals()
```

--> py4web

```
@action("index")
def index():
    a = request.query.get('a')
    return locals()
```

15.1.4 “?

web2py

```
def index():
    a, b, c = request.args
    b, c = int(b), int(c)
    return locals()
```

--> py4web

```
@action("index/<a>/<b:int>/<c:int>")
def index(a,b,c):
    return locals()
```

15.1.5 “?

web2py

```
def index():
    if request.method == "GET":
        return "GET"
    if request.method == "POST":
        return "POST"
    raise HTTP(400)
```

--> py4web

```
@action("index", method="GET")
def index():
    return "GET"

@action("index", method="POST")
def index():
    return "POST"
```

15.1.6 “?

web2py

```
def counter():
    session.counter = (session.counter or 0) + 1
    return str(session.counter)
```

--> py4web

```
def counter():
    session['counter'] = session.get('counter', 0) + 1
    return str(session['counter'])
```

15.1.7 “?

web2py

```
{{ extend 'layout.html' }}
<div>
{{ for k in range(1): }}
<span>{{= k }}<span>
{{ pass }}
</div>
```

--> py4web

```
[[ extend 'layout.html' ]]
<div>
[[ for k in range(1): ]]
<span>[[= k ]]<span>
[[ pass ]]
</div>
```

15.1.8 “Form ?

web2py

```
db.define_table('thing', Field('name'))

def index():
    form = SQLFORM(db.thing)
    form.process()
    if form.accepted:
        flash = 'Done!'
    rows = db(db.thing).select()
    return locals()
```

--> py4web

```
db.define_table('thing', Field('name'))

@action("index")
```

```
@action.uses(db, flash)
def index():
    form = Form(db.thing)
    if form.accepted:
        flash.set("Done!", "green")
    rows = db(db.thing).select()
    return locals()
```

?

```
<div class="flash">[[=globals().get('flash','')]]</div>
```

?

```
<flash-alerts class="padded " data-alert="[[=globals().get( 'flash',
'')]]"></flash-alerts>
```

?utils.js ?

?Flash ?

15.1.9 “grid” ?

web2py

```
def index():
    grid = SQLFORM.grid(db.thing, editable=True)
    return locals()
```

--> py4web

```
@action("index")
@action.uses(db, flash)
def index():
    grid = Grid(db.thing)
    form.param.editable = True
    return locals()
```

15.1.10 “?

web2py

```
file_path = os.path.join(request.folder, 'file.csv')
```

--> py4web

```
from .settings import APP_FOLDER
file_path = os.path.join(APP_FOLDER, 'file.csv')
```

15.1.11 “auth” ?

web2py

```
auth = Auth()
auth.define_tables()

@requires_login()
def index():
    user_id = auth.user.id
    user_email = auth.user.email
    return locals()

def user():
```

```
    return dict(form=auth())
```

?http://.../user/login ?

--> **py4web**

```
auth = Auth(define_table=False)
auth.define_tables()
auth.enable(route='auth')

@action("index")
@action.uses(auth.user)
def index():
    user_id = auth.user_id
    user_email = auth.get_user().get('email')
    return locals()
```

?http://.../auth/login ?auth.user ?auth.user ?@requires_login ?user_id ?auth.user_id ?auth.get_user() ?

?

```
@action.uses(auth)
```

?

```
@action.uses(auth.user)
```

?auth.user_id ?auth.user_id ?

?

?

16.1 ?

Py4web ?

?

?`task_run` ?`task_run` ?`db.task_run.log` ?`db.task_run.output` ?

py4web ?

?

?`task_run` ?

?

```
from pydal.tools.scheduler import Scheduler, delta, now
from .common import db

# create and start the scheduler
scheduler = Scheduler(db, sleep_time=1, max_concurrent_runs=1)
scheduler.start()

# register your tasks
scheduler.register_task("hello", lambda **inputs: print("hi!"))
scheduler.register_task("slow", lambda: time.sleep(10))
scheduler.register_task("periodic", lambda **inputs: print("I am periodic!"))
scheduler.register_task("fail", lambda x: 1 / x)

# enqueue some task runs:

scheduler.enqueue_run(name="hello")
scheduler.enqueue_run(name="hello", scheduled_for=now() + delta(10) # start in 10
secs
scheduler.enqueue_run(name="slow", timeout=1) # 1 secs
scheduler.enqueue_run(name="periodic", period=10) # 10 secs
scheduler.enqueue_run(name="fail", inputs={"x": 0})
```

?`settings.py` ?`USE_SCHEDULER=True` ?

?`Grid(db.task_run)` ?

?

- ?
- ?`db.commit()`
- ?`try...except` ?

```
def my_task():
    try:
        # do something
        db.commit()
    except Exception:
        db.rollback()
```

16.2 ?

?https://www.twilio.com/docs/sendgrid/for-developers/sending-email/quickstart-python) ?

?

```
import sendgrid
from sendgrid.helpers.mail import Mail, Email, To, Content

def sendmail_task(from_addr, to_addrs, subject, body):
    ""
    # build the messages using sendgrid API
    from_email = Email(from_addr)  # Must be your verified sender
    content_type = "text/plain" if body[:6] != "<html>" else "text/html"
    content = Content(content_type, body)
    mail = Mail(from_email, To(to_addrs), subject, content)
    # ask sendgrid to deliver it
    sg = sendgrid.SendGridAPIClient(api_key=settings.SENDGRID_API_KEY)
    response = sg.client.mail.send.post(request_body=mail.get())
    # check if worked
    assert response.status_code == "200"

# register the above task with the scheduler
scheduler.register_task("sendmail", sendmail_task)
```

?

```
email = {
    "from_addr": "me@example.com",
    "to_addrs": ["me@example.com"],
    "subject": "Hello World",
    "body": "I am alive!",
}
scheduler.enqueue_run(name="sendmail", inputs=email, scheduled_for=None)
```

?`scheduled_for` ?`sendmail` ?`task_run` ?

?

```
class MySendGridSender:
    def __init__(self, from_addr):
        self.from_addr = from_adds
    def send(self, to_addr, subject, body):
        email = {
            "from_addr": self.from_addr,
            "to_addrs": [to_addr],
            "subject": subject,
            "body": body,
        }
        scheduler.enqueue_run(name="sendmail", inputs=email)

auth.sender = MySendGridSender(from_addr="me@example.com")
```

?`auth.sender` ?`send` ?

?`auth_user` ?`Auth.send` ?

16.3 Celery

?

16.4 py4web ?

Asyncio ?

?

16.5 htmx

?

?[CIT1601]

?https://htmx.org ?Simple, Fast Frontends With htmx ?

py4web ?

1. ?
2. ?

16.5.1 ?

py4web Form ?

```
attrs = {
    "_hx-post": URL("url_to_post_to/%s" % record_id),
    "_hx-target": "#detail-target",
}
form = Form(
    db.tablename,
    record=record_id,
    **attrs,
)
```

?

?

controllers.py

```
import datetime

@action("htmx_form_demo", method=["GET", "POST"])
@action.uses("htmx_form_demo.html")
def htmx_form_demo():
    return dict(timestamp=datetime.datetime.now())


@action("htmx_list", method=["GET", "POST"])
@action.uses("htmx_list.html", db)
def htmx_list():
    superheros = db(db.superhero.id > 0).select()
```

[CIT1601] ?https://htmx.org ?

```
    return dict(superheros=superheros)


@action("htmx_form/<record_id>", method=["GET", "POST"])
@action.uses("htmx_form.html", db)
def htmx_form(record_id=None):
    attrs = {
        "_hx-post": URL("htmx_form/%s" % record_id),
        "_hx-target": "#htmx-form-demo",
    }
    form = Form(db.superhero, record=db.superhero(record_id), **attrs)
    if form.accepted:
        redirect(URL("htmx_list"))

    cancel_attrs = {
        "_hx-get": URL("htmx_list"),
        "_hx-target": "#htmx-form-demo",
    }
    form.param.sidecar.append(A("Cancel", **cancel_attrs))

    return dict(form=form)
```

templates/htmx_form_demo.html

```
[[extend 'layout.html']]

[[=timestamp]]
<div id="htmx-form-demo">
    <div hx-get="[[=URL('htmx_list')]]" hx-trigger="load"
hx-target="#htmx-form-demo"></div>
</div>

<script src="https://unpkg.com/htmx.org@1.3.2"></script>
```

templates/htmx_list.html

```
<ul>
[[for sh in superheros:]]
    <li><a hx-get="[[=URL('htmx_form/%s' % sh.id)]]"
hx-target="#htmx-form-demo">[[=sh.name]]</a></li>
[[pass]]
</ul>
```

templates/htmx_form.html

```
[[=form]]
```

?

?

?

?

?

16.5.2 ?

py4web ?

?

controller.py

```
@action("htmx_form/<record_id>", method=["GET", "POST"])
@action.uses("htmx_form.html", db)
```

```
def htmx_form(record_id=None):
    attrs = {
        "_hx-post": URL("htmx_form/%s" % record_id),
        "_hx-target": "#htmx-form-demo",
    }
    form = Form(db.superhero, record=db.superhero(record_id), **attrs)
    if form.accepted:
        redirect(URL("htmx_list"))

    cancel_attrs = {
        "_hx-get": URL("htmx_list"),
        "_hx-target": "#htmx-form-demo",
    }
    form.param.sidecar.append(A("Cancel", **cancel_attrs))

    return dict(form=form)

@action("htmx_grid")
@action.uses( "htmx_grid.html", session, db)
def htmx_grid():
    grid = Grid(db.superhero, auto_process=False)

    grid.attributes_plugin = AttributesPluginHtmx("#htmx-grid-demo")
    attrs = {
        "_hx-get": URL(
            "htmx_grid",
        ),
        "_hx-target": "#htmx-grid-demo",
    }
    grid.param.new_sidecar = A("Cancel", **attrs)
    grid.param.edit_sidecar = A("Cancel", **attrs)

    grid.process()

    return dict(grid=grid)
```

templates/htmx_form_demo.html

```
[[extend 'layout.html']]

[[=timestamp]]
<div id="htmx-form-demo">
    <div hx-get="[[=URL('htmx_list')]]" hx-trigger="load"
hx-target="#htmx-form-demo"></div>
</div>

<div id="htmx-grid-demo">
    <div hx-get="[[=URL('htmx_grid')]]" hx-trigger="load"
hx-target="#htmx-grid-demo"></div>
</div>

<script src="https://unpkg.com/htmx.org@1.3.2"></script>
```

?

templates/htmx_grid.html

```
[[=grid.render()]]
```

?

16.5.3 ?

htmx ??

?

?

```
import json
from functools import reduce

from yatl import DIV, INPUT, SCRIPT

from py4web import action, request, URL
from ..common import session, db, auth


@action(
    "htmx/autocomplete",
    method=["GET", "POST"],
)
@action.uses(
    "htmx/autocomplete.html",
    session,
    db,
    auth.user,
)
def autocomplete():
    tablename = request.params.tablename
    fieldname = request.params.fieldname
    autocomplete_query = request.params.query

    field = db[tablename][fieldname]
    data = []

    fk_table = None

    if field and field.requires:
        fk_table = field.requires.ktable
        fk_field = field.requires.kfield

        queries = []
        if "_autocomplete_search_fields" in dir(field):
            for sf in field._autocomplete_search_fields:
                queries.append(
                    db[fk_table][sf].contains(
                        request.params[f"{tablename}_{fieldname}_search"]
                    )
                )
            query = reduce(lambda a, b: (a | b), queries)
        else:
            for f in db[fk_table]:
                if f.type in ["string", "text"]:
                    queries.append(
                        db[fk_table][f.name].contains(
                            request.params[f"{tablename}_{fieldname}_search"]
                        )
                    )

            query = reduce(lambda a, b: (a | b), queries)

        if len(queries) == 0:
            queries = [db[fk_table].id > 0]
            query = reduce(lambda a, b: (a & b), queries)

        if autocomplete_query:
            query = reduce(lambda a, b: (a & b), [autocomplete_query, query])
```

```
        data = db(query).select(orderby=field.requires.orderby)

    return dict(
        data=data,
        tablename=tablename,
        fieldname=fieldname,
        fk_table=fk_table,
        data_label=field.requires.label,
    )

class HtmxAutocompleteWidget:
    def __init__(self, simple_query=None, url=None, **attrs):
        self.query = simple_query
        self.url = url if url else URL("htmx/autocomplete")
        self.attrs = attrs

        self.attrs.pop("simple_query", None)
        self.attrs.pop("url", None)

    def make(self, field, value, error, title, placeholder="", readonly=False):
        #  TODO: handle readonly parameter
        control = DIV()
        if "_table" in dir(field):
            tablename = field._table
        else:
            tablename = "no_table"

        #  build the div-hidden input field to hold the value
        hidden_input = INPUT(
            _type="text",
            _id="%s_%s" % (tablename, field.name),
            _name=field.name,
            _value=value,
        )
        hidden_div = DIV(hidden_input, _style="display: none;")
        control.append(hidden_div)

        #  build the input field to accept the text

        #  set the htmx attributes

        values = {
            "tablename": str(tablename),
            "fieldname": field.name,
            "query": str(self.query) if self.query else "",
            **self.attrs,
        }
        attrs = {
            "_hx-post": self.url,
            "_hx-trigger": "keyup changed delay:500ms",
            "_hx-target": "#%s_%s_autocomplete_results" % (tablename, field.name),
            "_hx-indicator": ".htmx-indicator",
            "_hx-vals": json.dumps(values),
        }
        search_value = None
        if value and field.requires:
            row = (
                db(db[field.requires.ktable][field.requires.kfield] == value)
                .select()
                .first()
            )
            if row:
                search_value = field.requires.label % row
```

```
        control.append(
            INPUT(
                _type="text",
                _id="%s_%s_search" % (tablename, field.name),
                _name="%s_%s_search" % (tablename, field.name),
                _value=search_value,
                _class="input",
                _placeholder=placeholder if placeholder and placeholder != "" else
"..",
                _title=title,
                _autocomplete="off",
                **attrs,
            )
        )

        control.append(DIV(_id="%s_%s_autocomplete_results" % (tablename,
field.name)))

        control.append(
            SCRIPT(
                """
        htmx.onLoad(function(elt) {
            document.querySelector('#%(table)s_%(field)s_search').onkeydown =
check_%(table)s_%(field)s_down_key;
            \n
            function check_%(table)s_%(field)s_down_key(e) {
                if (e.keyCode == '40') {

document.querySelector('#%(table)s_%(field)s_autocomplete').focus();

document.querySelector('#%(table)s_%(field)s_autocomplete').selectedIndex = 0;
                }
            }
        })
            """
                % {
                    "table": tablename,
                    "field": field.name,
                }
            )
        )

        return control
```

?

```
formstyle = FormStyleFactory()
formstyle.classes = FormStyleBulma.classes
formstyle.class_inner_exceptions = FormStyleBulma.class_inner_exceptions
formstyle.widgets["vendor"] = HtmxAutocompleteWidget(
    simple_query=(db.vendor.vendor_type == "S")
)

form = Form(
    db.product,
    record=product_record,  # defined earlier in controller
    formstyle=formstyle,
)
```

?

?

?

16.6 utils.js

?

16.6.1 string.format

?

```
var a = "hello {name}".format(name="Max");
```

16.6.2 Q ?

Q ?

```
var element = Q("#element-id")[0];
var selected_elements = Q(".element-class");
```

?querySelectorAll ?

Q ?

?

Q.clone

?

```
var b = {any: "object"}
var a = Q.clone(b);
```

Q.eval

?

```
var a = Q.eval("2+3+Math.random()");
```

Q.ajax

JS fetch ?

```
var data = {};
var headers = {'custom-header-name': 'value'}
var success = response => { console.log("recereived", response); }
var failure = response => { console.log("recereived", response); }
Q.ajax("POST", url, data, headers).then(success, failure);
```

Q.get_cookie

?

```
var a = Q.get_cookie("session");
```

Q.register_vue_component

?

?

```
Vue.component('button-counter', {
data: function () {
    return {
        count: 0
    }
},
template: '<button v-on:click="count++">You clicked me {{ count }}
```

```
times.</button>'
});
```

?

```
Q.register_vue_component("button-counter", "button-counter.html", function(res) {
    return {
        data: function () {
            return {
                count: 0
            };
        };
});
```

Q.upload_helper

?

?

?

```
<input type="file" id="my-id" />
```

and

```
var file_name = ""
var file_content = "";
Q.upload_helper("my_id", function(name, content) {
   file_name = name;
   file_content = content; // base 64 encoded;
}
```

16.6.3 T ?

?

```
T.translations = {'dog': {0: 'no cane', 1: 'un case', 2: '{n} cani', 10: 'tanti
cani'}};
var message = T('dog').format({n: 5}); // "5 cani"
```

?

Q.debounce

?

```
setInterval(500, Q.debounce(function(){console.log("hello!")}, 200);
```

?

Q.debounce

?

```
Q("#element").onclick = Q.debounce(function(){console.log("clicked!")}, 1000);
```

?

Q.tags_inputs

?

```
<input name="browsers"/>
```

?

```
Q.tags_input('[name=zip_codes]')
```

?

```
Q.tags_input('[name=zip_codes]', {
   freetext: false,
   tags: ['Chrome', 'Firefox', 'Safari', 'Edge']
});
```

?-*list* ?

```
<input name="browsers"/>
<datalist id="browses-list">
   <option>Chrome</option>
   <option>Firfox</option>
   <option>Safari</option>
   <option>Edge</option>
</datalist>
```

?

```
Q.tags_input('[name=zip_codes]', {freetext: false});
```

?

```
ul.tags-list {
  padding-left: 0;
}
ul.tags-list li {
  display: inline-block;
  border-radius: 100px;
  background-color: #111111;
  color: white;
  padding: 0.3em 0.8em 0.2em 0.8em;
  line-height: 1.2em;
  margin: 2px;
  cursor: pointer;
  text-transform: capitalize;
}
ul.tags-list li[data-selected=true] {
  opacity: 1.0;
}
```

?.*type-list-string* ?.*type-list-integer* ?*tag_input* ?

Q.score_input

```
Q.score_input(Q('input[type=password]')[0]);
```

?

?

?

?

```
<ajax-component id="component_1" url="[[=URL('mycomponent')]]">
    <blink>Loading...</blink>
</ajax-component>
```

?

```
@action("mycomponent", method=["GET", "POST"])
@action.uses(flash)
def mycomponent():
    flash.set("Welcome")
    form = Form([Field("your_name")])
```

```
    return DIV(
        "Hello " + request.forms["your_name"]
        if form.accepted else form).xml()
```

?*<html><body>...</body></html>* ?

?

?*redirect("other_page")* ?

?*<script> ... </script>* ?

16.7 ?

?**Altcha** ?https://altcha.org

16.7.1 ?

?

```
python3 -m pip install --upgrade altcha
```

?`.settings.py` ?

```
# .settings.py
ALTCHA_HMAC_KEY = "your-very-secret-key-here"
```

16.7.2 ?

?`altcha` ?`some_form` ?

```
# controllers/default.py
from altcha import (
    create_challenge,
    verify_solution,
    ChallengeOptions,
)
from py4web import action, response, request, URL, Field, flash, Form
from py4web.utils.form import XML, T
from .settings import ALTCHA_HMAC_KEY

@action("altcha", method=["GET"])
def get_altcha():
    """Generates and returns an Altcha challenge."""
    try:
        challenge = create_challenge(
            ChallengeOptions(
                hmac_key=ALTCHA_HMAC_KEY,
                max_number=50000,
            )
        )
        response.headers["Content-Type"] = "application/json"
        return challenge.__dict__
    except Exception as e:
        response.status = 500
        return {"error": f"Failed to create challenge: {str(e)}"}

@action.uses("form_altcha.html", session, flash)
def some_form():
    """An example form that uses the Altcha captcha."""
    fields = [
        Field("name", requires=IS_NOT_EMPTY()),
        Field("color", type="string", requires=IS_IN_SET(["red", "blue",
```

```
"green"])),
    ]
    form = Form(fields,
                csrf_session=session,
                submit_button=T("Submit"))

    # Insert the Altcha widget HTML before the submit button
    form.structure.insert(-1, XML('<altcha-widget></altcha-widget></br>'))

    if form.accepted:
        altcha_payload = request.POST.get("altcha")
        if not altcha_payload:
            response.status = 400
            flash.set("NO ALTCHA payload")
            print("NO ALTCHA payload")
        else:
            ok, error = verify_solution(altcha_payload, ALTCHA_HMAC_KEY)
            if not ok:
                response.status = 400
                flash.set(f"ALTCHA verification fail: {error}")
                print("ALTCHA verification fail:", error)
            else:
                flash.set("ALTCHA verified.")

    return dict(form=form)
```

16.7.3 ?

?

form_altcha.html

?some_form action ?challengeurl ?altcha action ?

```
[[extend 'layout.html']]

<script async defer
src="https://cdn.jsdelivr.net/gh/altcha-org/altcha/dist/altcha.min.js"
type="module"></script>
<script>
    document.addEventListener("DOMContentLoaded", function () {
        const altchaWidget = document.querySelector("altcha-widget");
        if (altchaWidget) {
            altchaWidget.setAttribute("challengeurl", "[[=URL('altcha')]]");
        }
    });
</script>
<div class="section">
    <div class="vars">[[=form]]</div>
</div>
```

16.7.4 ?

?<altcha-widget> ?

```
[[extend "layout.html"]]
<script async defer
src="https://cdn.jsdelivr.net/gh/altcha-org/altcha/dist/altcha.min.js"
type="module"></script>

<script>
    document.addEventListener("DOMContentLoaded", function () {
        const altchaWidget = document.querySelector("altcha-widget");
```

```
        if (altchaWidget) {
            altchaWidget.setAttribute("challengeurl", "[[=URL('altcha')]]");
        }
    });
</script>
<style>
    .auth-container {
        max-width: 80%;
        min-width: 400px;
        margin-left: auto;
        margin-right: auto;
        border: 1px solid #e1e1e1;
        border-radius: 10px;
        padding: 20px;
    }
</style>

[[form.structure.insert(-1,XML('<altcha-widget></altcha-widget></br>'))]]

<div class="auth-container">[[=form]]</div>
```

?

```
class AltchaServerFixture(Fixture):
    def __init__(self, hmac_key=ALTCHA_HMAC_KEY):
        super().__init__()
        self._name = "altcha_server"
        self.hmac_key = hmac_key

    def on_success(self, context):
        # Only verify Altcha for POST requests
        if request.method != "POST":
            return
        payload = request.POST.get("altcha")
        if not payload:
            raise HTTP(400, "ALTCHA payload not received")
        try:
            verified, err = verify_solution(payload, self.hmac_key, True)
            if not verified:
                raise HTTP(400, "Invalid ALTCHA")
            return {"success": True, "message": "Altcha verification passed"}
        except Exception as e:
            raise HTTP(500, "Exception in Altcha verification")

    @property
    def name(self):
        return self._name
```

?AltchaServerFixture ?auth ?common.py ?

```
from .fixtures import AltchaServerFixture
auth.enable(uses=(session, T, db, AltchaServerFixture()), env=dict(T=T))
```

?

?AltchaServerFixture

```
@action('other_form', method=['GET', 'POST'])
@action.uses('form_altcha.html', session, flash, AltchaServerFixture())
def other_form():
    fields = [
        Field("name", requires=IS_NOT_EMPTY()),
        Field("color", type="string", requires=IS_IN_SET(["red","blue","green"])),
    ]
```

```
    form = Form(fields,
                csrf_session=session,
                submit_button=T("Submit"))
    antes_submit = len(form.structure) - 3
    form.structure.insert(antes_submit,
XML('<altcha-widget></altcha-widget></br>'))
    if form.accepted:
        # You can assume here that the Altcha payload was verified by the fixture
        flash.set("Form and Altcha successfully verified.")
        # Process the form data here
    return dict(form=form)
```

- genindex
- modindex
- search